Aalto University
School of Science
Master's Programme in Computer, Communication and Information Sciences

Measuring Static UI Transition Graph Extraction in Single-Page Applications, with Scope Attribution

Duy Khanh Tran

Master's Thesis
Espoo, November 6, 2026

Supervisor: Prof. Petri Vuorimaa
Major: Computer Science

Abstract

A user-interface (UI) transition graph represents application screens and navigation between them. Such graphs are usually obtained by executing single-page applications. This thesis measures static extraction on six open-source Angular applications using five evaluation methods and a construct-level classification of unrecovered navigation. It proposes scope attribution, a dataflow analysis of the screens from which a navigation site may fire.

Under the study's demotion rules and execution assumptions, 25 constructs yield 159 must edges. Of these, 125 come from one toolbar whose scope has not been validated at run time; 33 remain without the route-order assumption. The small must tier largely reflects the classification rules rather than an inherent limit of static analysis.

Scope attribution recovers 66.7 % of the constructs assigned to attribution loss, or 38.6 % without the largest application. Most recovered constructs yield possible transitions through mechanisms from earlier work. The added step places navigation in services, guards and effects, contributing 36 sites (2.0 %) above outlet attribution. Its member-level call graph generalises Rima et al.'s bounded router-service tracing and adds 7 sites (0.4 %) over a per-class join.

On the 16 determined sites of an AI-built Alfresco reference, full attribution increases recall from 13.8 % to 23.6 % while reducing precision from 92.3 % to 82.0 %. The reference applies the analysis's own relations, so agreement does not establish true runtime scope. A comparison with Rima et al.'s tool also shows a coverage–precision trade-off, without an overall ranking. It uses small AI-read samples; its five-application subgroup was selected after inspecting the results. This extractor's estimated precision on the largest application is 6.5 %. Mutation testing provides only a conditional sanity check of source-to-witness mapping.

Route tables provide a starting point for static extraction, but source attribution and target resolution remain necessary. The main corpus results are in-sample because the extractor was developed on these applications. Consumer benefit is not measured.

Keywords: static analysis; UI transition graph; single-page application; Angular; dataflow analysis; program analysis; software testing

Use of AI tools

I wrote the text of this thesis and made the revisions myself. Artificial intelligence (AI) tools were used to support implementation, the search for relevant projects and papers, and review. I assessed the suggestions and decided which changes to make. The research questions, methodological choices and interpretation of the results are my own.

An AI coding assistant supported the implementation of the algorithms presented in this thesis, including the extractor and evaluation code. I reviewed the implementation and remain responsible for its correctness.

AI tools also helped identify candidate open-source projects and research papers. I assessed their relevance and selected the projects and sources used in the study.

AI was used to review the implementation, evaluation and manuscript, and to identify possible errors, inconsistencies and gaps in the arguments. I evaluated that feedback and revised the text myself. The corrections made in response to reviews are recorded in Section 4.3.2.

AI-assisted readings of application source code also form part of the evaluation evidence, including the Alfresco structural reference, the must-tier readings and the transition-pair judgements described in Chapters 4 to 6. Their role and limitations are stated there; they are not treated as independent human validation.

I take responsibility for the final text, implementation, results and conclusions.

Abbreviations

AI	artificial intelligence
Ajax	asynchronous JavaScript and XML
API	application programming interface
DOM	Document Object Model
GUI	graphical user interface
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
ICU	International Components for Unicode (message format)
JSON	JavaScript Object Notation
JSP	JavaServer Pages
JSX	JavaScript XML, the markup syntax of React
NgRx	Angular state-management library built on RxJS
RQ	research question
RxJS	Reactive Extensions for JavaScript
SPA	single-page application
SPIN	model checker for Promela models
UI	user interface
URL	Uniform Resource Locator
ZAP	Zed Attack Proxy, a web security scanner

Contents

Abstract **1**

Use of AI tools **2**

Abbreviations **3**

1 Introduction **7**
1.1 Motivation 7
1.2 Research questions and scope 9
1.3 Thesis structure 9

2 Background **11**
2.1 User-interface transition graphs 11
2.2 Static extraction for Android 11
2.3 Navigation models for web applications 12
2.4 Dynamic state models for single-page applications 13
2.5 Static analysis of JavaScript and TypeScript source 14
2.6 Formal foundations 15
2.6.1 Modal transition systems 15
2.6.2 Soundiness 15
2.6.3 Dataflow analysis and abstract interpretation 15
2.7 Angular routing, dependency injection and reactive state 16
2.7.1 Routes, components and navigation sites 16
2.7.2 Dependency injection 17
2.7.3 Reactive state 18
2.8 Reactive programming and the limits of static dataflow 18
2.9 Mutation testing 18
2.10 Summary and the gap 19

3 Static navigation analysis and scope attribution **22**
3.1 Graph model and soundness 22

3.2 Scope attribution . . . 27
3.2.1 The defect being addressed . . . 27
3.2.2 Scope . . . 28
3.2.3 Input relations . . . 30
3.2.4 Guards as program points . . . 30
3.2.5 Boundary . . . 30
3.2.6 Anchored edges . . . 31
3.2.7 Extensions not evaluated . . . 31
3.3 Implementation . . . 31
3.4 Summary . . . 34

4 Evaluation setup and research material **36**
4.1 Corpus selection and validity . . . 36
4.1.1 Selection . . . 36
4.1.2 Validity of an open-source corpus . . . 37
4.2 Metrics . . . 37
4.3 Evaluation methods . . . 41
4.3.1 Preconditions . . . 44
4.3.2 Corrections made to the protocol . . . 44
4.3.3 Reproducibility . . . 45

5 Results **49**
5.1 The two levels of loss (RQ1) . . . 49
5.2 Which failure is worth an analysis (RQ2) . . . 51
5.3 Scope attribution (RQ3) . . . 52
5.3.1 Checking the scopes against the running interface . . . 55
5.3.2 What the attribution discriminates . . . 57
5.3.3 What each mechanism contributes . . . 58
5.3.4 Agreement with a structural reference . . . 59
5.3.5 Agreement with the applications' own suites . . . 61
5.3.6 What scope attribution adds . . . 62
5.3.7 On the applications of Rima et al. . . . 63
5.3.8 The tool of Rima et al. on the same applications . . . 65
5.4 Change sensitivity (RQ4) . . . 67
5.5 What the losses were (RQ5) . . . 68
5.5.1 What the losses were . . . 68
5.5.2 Construct accounting . . . 69
5.5.3 The final tier and the node set . . . 70

5.6 Dead links, found from the source 71

6 Discussion **73**

6.1 Key findings 73

6.2 Limitations and threats to validity 75

6.3 Declared approximations 80

6.3.1 Why the may tier is incomplete 80

6.4 Transfer to other frameworks 81

6.5 Future work 82

6.6 Summary 83

7 Conclusion **84**

Bibliography **87**

A Corrections made to the protocol **95**

B Declared approximations **102**

Chapter 1

Introduction

A user interface (UI) transition graph represents application screens and the transitions caused by user actions. Each edge records a triggering event, a guard and a destination. The graph supports reachability queries and helps identify screens affected by a component change.

Transition graphs support Android test generation and exploration [79, 39, 14] and crawling-based web testing [50, 51]. Their links to source code and tests also support change impact analysis [4, 12].

1.1 Motivation

Dynamic extraction executes an application and records the states and transitions it observes. Its coverage depends on test data, authentication and the exploration budget, and each new version requires another run. Static extraction derives a graph from source code without executing the application. It can run on each commit, but recovers only the constructs its analysis supports and must approximate targets it cannot resolve.

These approaches are complementary. Much of the navigation in the applications studied here is behind authentication guards. A static graph can identify candidate screens, transitions and guards for a crawler, while execution can confirm observed transitions. This thesis evaluates only static extraction; the combination is left for future work (Section 6.5).

Static extraction is established for Android. Window transition graph analyses derive windows and transitions from bytecode and link callback control flow to the widgets registering each handler [79, 78, 77]. Early web navigation models combined templates and configuration with crawling or server execution [62, 9]; a patent describes a fully static method [69].

Modern single-page applications have mostly been modelled dynamically. Crawljax builds a state flow graph by firing events against the running Document Object Model (DOM) [50]. FeedEx and FRAGGEN improve exploration while retaining execution as the source of the model [51, 74]. A survey covering 2014–2025 reviews model-based web testing [36]; every listed tool whose model construction was examined for this thesis executes the application. Framework-specific static analyses

remain few. One constructs a React component dependency graph without routing [38]. Three extract navigation:

- Oshima and Kishi extract Angular page transitions for model checking. Their stated restrictions exclude nested routes and route parameters, and their model omits page transitions caused by data-binding changes [56].
- Rima et al. extract Angular workflows as a coverage denominator for generated end-to-end tests [64].
- Zhang et al. analyse compiled single-page application bundles to check access-control policies against reachable screens [81].

Section 2.4 compares these approaches.

Early Ajax applications lacked declarative route tables, so crawlers used DOM states to observe navigation. Angular, React Router and Vue Router now expose route tables, template navigation and service calls in source code; routes can also name guards [27]. These declarations provide inputs for static extraction. Existing studies evaluate extraction against hand-built references on their chosen applications, but do not measure losses construct by construct.

Navigation graphs also motivate applications beyond model-based testing. GPT-Droid uses a question-and-answer loop over the current mobile screen and reports higher activity coverage and more detected bugs than its best baseline on 93 applications [46]. AutoDroid supplies a model with an application-specific transition graph built by offline dynamic exploration [73]; LLMDroid uses a model to guide exploration towards uncovered behaviour [70]. Related work explores web interfaces using vision-language models [72], and a survey covers large language models in software testing [71]. Coding agents may also use navigation structure: SWE-bench tasks require coordinated edits across repository files [31], and repository interfaces affect agent performance [76]. Whether navigation graphs benefit these consumers is not tested here.

This thesis builds an Angular extractor to measure recovery and classify losses. It proposes *scope attribution*, a dataflow analysis over screen sets, for the largest loss category under the study's rule order. The analysis is specified independently of the extractor. Measurement is the main contribution; scope attribution is a modest extension of earlier mechanisms. Its added step places navigation in services, guards and effects, recovering 36 sites (2.0 %) above outlet attribution. This step combines a member-level service call graph, a fallback join and stand-ins for guards and effects. The call graph alone adds 7 sites (0.4 %) over a per-class join (Section 5.3.3).

The extractor supports nested, parameterised and lazily loaded route trees as well as scope attribution. The stated restrictions of the earlier Angular model-checking approach exclude the first two (Section 5.3.6). The extractor was developed and evaluated on the same corpus, so its measurements describe these applications rather than estimate performance on Angular applications generally. Section 4.3.2 records the development corrections; Section 6.2 discusses their implications. Chapter 6 separates implementation gaps from limits of the approach.

1.2 Research questions and scope

The aim is to measure how far static extraction of a UI transition graph gets on real Angular applications, why it fails where it does, and what scope attribution adds. Five research questions (RQs) follow from it; the measures are defined in Section 4.2 and answered in Chapter 5.

RQ1 How much of the navigation in Angular applications can static analysis recover as deterministic transitions?

RQ2 Where does static extraction fail, and why?

RQ3 How does scope attribution affect the recovered share and precision of static navigation analysis?

RQ4 Does the extracted graph respond to source changes in the constructs it represents, as change impact analysis requires?

RQ5 Are the losses a property of the applications or of the tool?

The questions are answered on six open-source Angular applications pinned to fixed commits (Chapter 4), with the final version of the extractor. The thesis does not claim completeness: it states a soundness property for the must tier that holds only relative to stated assumptions (Claim 3.1) and lists what it approximates (Section 2.6.2). It does not measure the graph's effect on any consumer, it evaluates Angular only, and its figures on the corpus are in-sample, since the extractor was developed on it.

1.3 Thesis structure

Figure 1.1 summarises the study.

Chapter 2 reviews transition graphs, related extraction methods, Angular and the analysis foundations. Chapter 3 defines the model, its conditional soundness claim, scope attribution and the implementation. Chapter 4 describes the corpus, metrics, evaluation methods and protocol corrections. Chapter 5 answers the research questions and reports statically detected dead-link candidates. Chapter 6 interprets the results and discusses limitations, framework transfer and future work. Chapter 7 summarises the contributions.

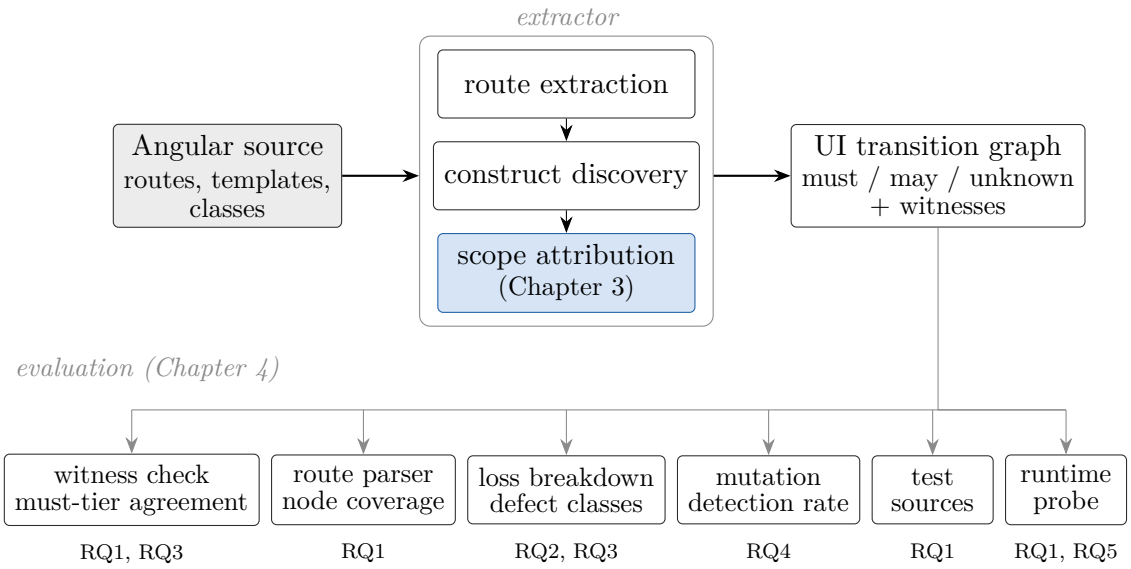


Figure 1.1: Overview of the study: extractor, scope attribution, the main measurements and the research questions they answer.

Note. The extractor turns Angular source into a UI transition graph; scope attribution is the analysis this thesis adds. The loss breakdown classifies what the other methods find missing rather than being one of the five methods itself.

Chapter 2

Background

This chapter defines transition-graph terminology (Section 2.1) and reviews static extraction for Android, web navigation models and dynamic models for single-page applications (SPAs) (Sections 2.2 to 2.4). It then covers source-level static analysis (Section 2.5), formal foundations (Section 2.6), Angular mechanisms (Section 2.7) and reactive programming's limits on static analysis (Section 2.8). Section 2.9 reviews mutation testing, used to answer RQ4 in Chapter 4. Section 2.10 identifies the research gap.

2.1 User-interface transition graphs

A UI transition graph is a directed graph whose nodes represent user-visible application states and whose edges represent transitions caused by user actions. Figure 2.1 illustrates the model used here.

Research communities use different names for these graphs. In Android static analysis it is a *window transition graph* [79], *activity transition graph* [14] or *screen transition graph* [39]. For Ajax (asynchronous JavaScript and XML) crawlers it is a *state flow graph* over DOM states [50]. In web engineering it is a *navigation model* or *navigation state machine* [62, 18]. This thesis uses *UI transition graph* for all of them, while recognising differences in state definitions and graph construction.

Two distinctions matter more than terminology: whether extraction is *static*, from source code or binaries without execution, or *dynamic*, from observed runs; and whether edges record *guards*, the conditions under which transitions are available. Table 2.1 classifies the reviewed work by both distinctions.

2.2 Static extraction for Android

Most static transition-graph research concerns Android. Yang et al. introduced the window transition graph, with windows (activities, dialogs and menus) as nodes and UI-thread callback transitions as edges. The analysis models the active window stack, including back-button behaviour [79, 77]. A related analysis builds a callback control-flow graph over handler–widget pairs [78]. StoryDroid builds an activity

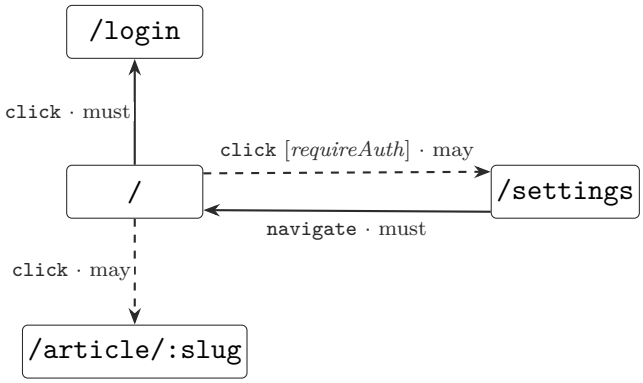


Figure 2.1: A small UI transition graph in the model of this thesis, with must and may edges.

Note. Each edge carries an event, an optional guard in brackets, and a modality: *must* always fires when its code runs, *may* is conditional (here on a route guard or a partly known target). In the measured graphs the unknown modality marks only edges into a sink for destinations that only the run time knows.

transition graph from `startActivity` calls [14]. GoalExplorer combines a statically derived screen transition graph with dynamic exploration [39]. UI-CTX finds the code context of a widget's behaviour with a backward data-flow analysis from the handler to the call that retrieves the widget [44].

These approaches attribute navigation to a screen through object resolution. A handler is registered on a widget that belongs to exactly one window at run time. Resolving its identifier, usually by constant propagation of `getId()`, identifies that window [78]. ProMal illustrates the limitation: imprecision in the reference analysis aliases several widgets, assigning handlers to the wrong window and producing infeasible transitions. Its authors combine static analysis with runtime traces and a learned model [45], using dynamic evidence rather than refining static attribution.

2.3 Navigation models for web applications

Web navigation models were first extracted from server-rendered sites. Ricca and Tonella modelled pages, frames and forms connected by links and form submissions. Edges carry boolean conditions on input values, derived semi-automatically using a crawler and analyst annotations [62]. WebUml combines static analysis of Hypertext Markup Language (HTML), script and server-side pages with server runs to recover a state diagram. It derives guards through control-flow analysis of server code [9]. A patent assigned to Fujitsu describes a fully static method for Struts and JSP sources [69]. SiteHopper builds abstracted navigation state machines on the fly for model checking [18].

These approaches target server-rendered frameworks, where pages correspond to files and links to anchor elements. They do not address client-side routers, component trees or dependency injection. Section 2.4 reviews work that does.

2.4 Dynamic state models for single-page applications

With Ajax and client-side frameworks, model construction moved to run time. Crawljax executes user events, compares the DOM before and after each event, and builds a state flow graph over equivalence classes of DOM states [50]. Later work changed the exploration strategy but not the model's source [51, 74]. The survey by Kertusha et al. reviews web testing research from 2014 to 2025 [36]. Every listed model-based tool whose construction method was examined for this thesis executes the application to build its model. This has historical reasons: Ajax applications lacked declarative route tables, one uniform resource locator (URL) could represent many states, and source code did not represent routes explicitly, although JavaScript static analysis existed [29].

Client-side routing frameworks declare route tables as data, navigation as attributes or method calls naming routes, and guards as named functions attached to routes. The literature search found four static analyses of such frameworks: three analyse framework-specific source code and one a compiled bundle. Kurant extracts a React component rendering graph but does not address routing [38].

The other three address routing. Oshima and Kishi extract Angular page transitions for model checking with SPIN [56]. They analyse `RouterModule` declarations and each component's navigable paths, resolving parent–child relationships among root, routed and custom child components. Paths are attributed to the containing routed component; the root component's own paths are attributed to every page. Section 5.3.3 measures this combination as the *rooted* level (levels are defined in Section 4.2). The approach excludes `children` and route parameters and does not treat data-binding changes as page transitions. The paper does not state how it handles destinations expressed as nonliteral expressions; no treatment is assumed here. Screens are identified by routed-component names, not addresses. Guards, lazy loading and named outlets are not modelled. Navigation reached through a method call or dispatch is outside the attribution mechanism. Without modalities, always-available and guarded transitions have identical representations. The evaluation uses four constructed applications and two published applications of 1824 and 6893 lines, with six and twenty-three pages. It reports whether the intended models were produced and hand-injected transition faults were detected in constructed variants of the first sample. Losses on applications the extractor cannot fully analyse are not measured; that is the subject of this thesis.

Two recent works statically analyse client-side router configuration for other purposes. Rima et al. parse Angular sources and templates into a graph of navigation, interaction sites and execution conditions, including guards. They compile workflows enumerated from the graph into browser tests and use it as a coverage denominator [64]. This account draws on the publisher's abstract, the replication package's operational specification (`softscanner-sw/softscanner_fa`, `docs/paper/approach.md`), and the anonymised manuscript in the same repository (`docs/paper/main.pdf`). The published chapter was unavailable and may differ from both documents. Both describe attribution beyond containment: active components

include those activated by the route and its ancestors, closed under composition, plus bootstrap components. This covers the attribution of the *rooted* and *outlet* levels in Section 5.3.3. Handlers are traced into same-class helpers to depth one and into the first `subscribe` callback. The specification excludes router-outlet modelling, provider resolution beyond module and root providers, and service calls. The implementation nevertheless traces calls into services whose class names match a router or navigation pattern, to depth one. Navigation inside these services is attributed to their callers' screens.

Zhang et al. parse compiled JavaScript bundles, using declared routes as vertices and navigation calls as edges. They check an access-control policy against screens reachable from an authorised screen. Two of their four subjects are multi-page applications, analysed by crawling instead. Their graph is an over-approximation and is not narrowed. The paper does not explain source-screen attribution; guards remain unmodelled runtime conditions [81]. Rima et al. compare their output with a ground truth derived from their pipeline's output and annotated by hand. Neither work classifies losses by construct or measures them across a corpus. The problem is therefore neither new nor settled; Section 2.10 identifies the remaining gap.

2.5 Static analysis of JavaScript and TypeScript source

The analysis in Chapter 3 uses the TypeScript compiler application programming interface (API) to analyse source-level abstract syntax trees rather than compiled artefacts. Baxter et al. used this approach for C clone detection [8]; SPOON provides a reusable Java library [58]. Source-level analysis preserves the connection to the developer's code, allowing results to cite source locations, but is limited to the program as written. Jensen et al. show that abstract interpretation can infer sound type information for full JavaScript [29]. Thus the framework of Section 2.6 applies to individual values, a harder problem than scope attribution (Section 3.2.2), which uses a much coarser domain.

TypeScript lets an analysis use declared types instead of inferring them. Its type system is deliberately unsound [10, 63], so a declaration is evidence, not a guarantee. Flow and TypeScript detected only about 15% of fixed bugs in JavaScript project histories [25]. Across 604 GitHub projects, TypeScript was associated with better code quality and understandability but not lower bug proneness [11]. Declarations for untyped libraries can also be wrong: TSCheck found 142 errors in those of 10 libraries [22]. Declared types narrow a call's candidate implementations, as used in the call graph of Section 2.7, but do not guarantee a unique target. An unresolved or ambiguous type must therefore widen the result. In the graph model, this distinguishes the must and may tiers.

Framework behaviour poses the practical obstacle. Madsen et al. note that JavaScript applications depend on large frameworks whose behaviour static analyses must model; hand-written models are tedious, incomplete and sometimes wrong [48]. They also model implicit Node.js control flow through an *event-based call graph* [49].

Angular poses a similar problem through its router, template event bindings and observable subscriptions. Alongside the navigation analyses of Oshima and Kishi (Section 2.4) and Rima et al. [64], Aurebesh checks consistency in AngularJS [55]. It checks identifiers and types across models, views and controllers in AngularJS 1.x, whose template language, component model and injector differ from Angular's. It does not address navigation.

2.6 Formal foundations

The analysis builds on modal transition systems, an explicit account of practical soundness limits, and dataflow analysis.

2.6.1 Modal transition systems

The *modal transition system* of Larsen and Thomsen labels each transition as either required (*must*) or merely permitted (*may*) [42]. The graph model in Chapter 3 adopts this distinction and adds *unknown* for edges whose modality could not be determined. The thesis claims soundness for the must tier only. The evaluation distinguishes the tiers because may edges have weaker supporting evidence.

With one edge kind, an unresolved transition must either be included, reducing precision, or omitted, reducing recall. Modalities record the evidence available, letting consumers select a tier. Chapter 5 therefore reports the tiers separately.

2.6.2 Soundiness

Livshits et al. argue that practical static analyses are inevitably sound only for a language subset. They recommend declaring approximated features rather than claiming general soundness [47]. Section 3.1 states this thesis's position. Sections 2.5 to 2.7 explain the limits here: TypeScript annotations are not guarantees; framework configuration determines call targets; and reactive firing conditions are not written in the program text.

2.6.3 Dataflow analysis and abstract interpretation

Kildall expressed many program analyses as fixpoint computations over lattices of abstract values. A join combines information, which propagates along graph edges until stable [37]. A finite-height lattice and monotone transfer functions ensure termination and an order-independent result. Cousot and Cousot developed the general theory of abstract interpretation [16]; Nielson et al. provide a textbook account [54]. Scope attribution in Chapter 3 uses this framework, with abstract values and join defined in Section 3.2.2.

An instance needs a bounded-height lattice and a propagation graph. Unresolved calls can make that graph incomplete, producing omissions the framework cannot detect.

2.7 Angular routing, dependency injection and reactive state

The analysis in Chapter 3 must account for Angular-specific mechanisms, described in the framework documentation [27]. Listings 2.1 to 2.3 introduce a running example.

Listing 2.1: A route table with a guarded route, nested children, a lazy route table and a constant path.

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'settings', component: SettingsComponent,
    canActivate: [requireAuth],
    children: [{ path: 'profile', component: ProfileComponent }] },
  { path: 'admin', loadChildren: () => import('./admin/admin.routes') },
  { path: paths.about, component: AboutComponent },   // constant, not a
    literal
];
```

Listing 2.2: The root template: the header sits next to the outlet.

```
<app-header />                <!-- shell: rendered on every screen -->
<router-outlet />             <!-- the active route renders here -->

<!-- header.component.html -->
<a routerLink="/settings">Settings</a>
@if (user()) { <a [routerLink]="['/profile', user().name]">Me</a> }
```

Note. Its links are present on every screen but belong to none of them.

Listing 2.3: Navigation from an injected service.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private router = inject(Router);
  private session = inject(SessionStore);
  logout(): void {
    this.session.clear();
    this.router.navigate(['/login']);
  }
}
```

Note. The call site belongs to no component, so its screens can only be found through the components that call `logout`.

2.7.1 Routes, components and navigation sites

Angular declares screens through a tree of route objects. Each has a `path` pattern and a `component`, `redirectTo` target or `children` list. A route with children but no component provides a path prefix, not a screen. Routes can load lazily through `loadChildren` or `loadComponent`. Guards include `canMatch`, `canActivate`, `canActivateChild` and `canDeactivate`; they return a boolean, redirect, or Promise

or Observable of either. When `canMatch` returns false, the router tries the next matching route rather than cancelling navigation. Resolvers fetch data that the router awaits before activation. Paths need not be string literals: resolving `path: routes.about.path`, for example, requires tracing the constant to its definition.

Each screen is rendered by a component: a class and template. Templates instantiate other components by selector and use `<router-outlet>` to mark where the router inserts the active child component. Components above the outlet form the application shell, present on every screen; those below a routed component are present only on that screen. Structural directives such as `@if` and `*ngIf` make nesting conditional. Components can also be created programmatically at run time.

Templates express navigation through `routerLink`; class code uses `Router.navigate` or `Router.navigateByUrl`. The `relativeTo` option permits targets relative to the current route. Navigation sites can occur in components, injectable services, guards, interceptors and resolvers. Their source screens are not explicitly declared and must be derived.

2.7.2 Dependency injection

Angular creates services and supplies them to components and services that declare them as dependencies. This is dependency injection, a form of inversion of control [32, 24]. Dependencies are requested by type or injection token. A token can have different implementations in different injector scopes, bound through `useClass`, `useExisting` or a factory. The pattern is common outside Angular; its use and anti-patterns have been detected statically in Java [75, 40]. Configuration is statically available but does not directly identify the binding at each call site.

For static analysis, this makes a method call a dispatch problem. Resolving `this.nav.goHome()` requires the type of `nav` and its binding in the relevant injector. This call graph construction problem is familiar from object-oriented languages. Approaches include class hierarchy analysis [17], rapid type analysis [5] and points-to analysis [2], unified by Grove and Chambers [28]. For JavaScript, Feldthaus et al. show that approximate call graphs can be precise enough for practical tools [23].

The Java literature models frameworks explicitly to resolve configuration-dependent bindings. Sridharan et al. derive framework specifications from application code and configuration [67]. Antoniadis et al. model shared enterprise-framework concepts declaratively, measuring about four times higher completeness [3]. Chen et al.'s Jasmine extends Soot to improve Spring call graph completeness [13]. Limits remain: reflection cannot be ignored in 78% of Java projects, and resolving it requires unsound assumptions [41]. In Angular, provider arrays belong to modules, routes or components. A token's implementation therefore depends on the injection site. Resolution is relative to an injector, making the provider hierarchy an input to the explicit framework model (Section 3.2.3).

2.7.3 Reactive state

Recent Angular versions include signals, values that change over time, and effects, computations that rerun when a signal they read changes [26]. This is reactive programming [21, 6]: dependencies between values are recorded at run time. Navigation inside an effect therefore depends on the reactive graph, not a control-flow condition. Static analysis can identify the signals read but cannot generally determine when the effect runs; this thesis does not attempt to bound it. Angular applications also use Reactive Extensions for JavaScript (RxJS) observables for this purpose.

2.8 Reactive programming and the limits of static dataflow

In reactive programming, the runtime controls when computations run. Cooper and Krishnamurthi embed dynamic dataflow in a call-by-value language and identify the *glitch*: a signal is recomputed before all its dependencies are up to date [15]. Drechsler et al. extend glitch freedom to distributed systems [20]. Static analysis can over-approximate an effect's signal read set. Its execution count and the application state at execution depend on which writes occur and in what order, so they cannot generally be determined exactly from source. They could still be over-approximated, but this analysis does not attempt it.

Tool support for reactive code is weak. Salvaneschi et al. found better comprehension of reactive code in an experiment with 127 subjects [66], but argue that it needs its own debugging concepts [65]. Banken et al. report that conventional debuggers lack Rx support [7]. Alabor and Stolze's small workshop study of 10 engineers suggests, without establishing, reliance on manual trace logging [1]. Tools also face broad operator use: 95.2% of the Rx operators are used in practice [82].

Angular navigation analysis inherits this limit. An effect or subscription exposes a read set in source, but that set does not exactly determine firing. Chapter 3 therefore specifies *unknown* edges with the read set as symbolic guard, following the soundiness position of Section 2.6 for unresolved constructs. The prototype emits no such edges for effects or subscriptions. Its unknown edges have destinations it cannot resolve to screens.

2.9 Mutation testing

RQ4 asks whether the graph responds to changes in the source constructs it represents. Mutation testing is applied to the application source, measuring the extracted graph's response rather than a test suite's.

Mutation testing introduces small syntactic changes and measures the share detected by a test suite. It rests on DeMillo et al.'s *coupling effect*: tests that detect simple errors are often effective against more complex errors [19]. This assumption connects mutants to developer faults. Jia and Harman provide the standard survey

[30]; a study of 357 real faults found a correlation between mutant and real-fault detection [33]. Operators exist for JavaScript and web applications [52, 53, 59]. Stryker supports JavaScript and TypeScript; its documentation lists more than 30 mutations, but no peer-reviewed description was found [68]. The procedure in Chapter 4 is therefore specified explicitly rather than delegated to tool defaults.

Following Papadakis et al. [57], Chapter 4 states the operators, mutant generation and sampling procedures, exclusions and per-operator detection rates. This use of mutation belongs to change impact analysis rather than test adequacy measurement [4, 12, 61, 43].

2.10 Summary and the gap

Static web navigation extraction has largely targeted server-rendered frameworks, where pages are files and links are anchors. Client-side routing frameworks have mostly been modelled dynamically. This literature search found three static client-side navigation extractions: two analyse framework-specific source code and one a compiled bundle, each for a different consumer. Oshima and Kishi establish the problem addressed here (Section 2.4), but exclude `children` and route parameters, and do not model data-binding changes as page transitions. Attribution uses component containment and the root component's own paths; transitions have no modality, and extraction losses are not measured. The other two works use their graphs without narrowing them: as a generated test suite's coverage denominator [64], or as an over-approximated reachability relation for access-control checks [81]. Rima et al. already analyse nested and lazy route trees, constrain edges with guards and trace router-service calls to a bounded depth. No earlier work found in this review combines cross-class and service attribution under provider bindings (generalising that bounded tracing), modalities separating unconditional from guarded or scope-uncertain transitions, and construct-level loss measurement on applications large enough to contain difficult constructs.

Android provides the closest static techniques. They resolve navigation's owning object and use runtime traces when resolution is imprecise. Web navigation sites often have no such object. In Android, a handler's owning object usually identifies its window; attribution therefore commonly reduces to object resolution. Fragments, background tasks and aliased widgets are exceptions handled with dynamic evidence. These differences prevent direct reuse of Android techniques.

Sections 2.5 to 2.9 identify established methods and remaining limits. JavaScript abstract interpretation is established [29], as is source-level analysis over typed syntax trees [8, 58]. Explicit framework models recover call edges missed by general analyses [67, 3, 13]. Aurebesh addresses AngularJS 1.x template–controller consistency rather than navigation [55]. TypeScript types narrow dispatch targets without resolving them, because the type system is deliberately unsound [10, 63]. Reactive execution depends on a runtime-built graph, limiting what source analysis determines [15, 7]. Mutation testing measures model responses to source changes [30, 57]. For static client-side navigation graphs, it has been applied only to hand-injected faults in

constructed applications [56]. This review found no operator-level mutation study of such graphs.

Table 2.1: Prior work closest to this thesis, by target, extraction mode, guard representation and evaluation basis.

Work	Target	Static	Guards	Evaluation basis
Yang et al. [79, 77]	Android windows	✓	✗	20 apps; manual on 6
Yang et al. [78]	Android callbacks	✓	context	manual vs. dynamic
StoryDroid [14]	Android activities	✓	✗	hand-built, 10 apps
GoalExplorer [39]	Android screens	hybrid	✗	manual, 36 apps
ProMal [45]	Android windows	hybrid	✗	40 apps, F1
UI-CTX [44]	Android widgets	✓	branches	2000 labelled widgets
Ricca and Tonella [62]	server-rendered pages	semi	✓	manual
WebUml [9]	server-rendered pages	hybrid	✓	case study
Tkachuk and Rajan [69]	Struts, JavaServer Pages (JSP)	✓	outcome	not stated
Crawljax [50]	Ajax DOM states	✗	✗	manual
FeedEx, FRAGGEN [51, 74]	DOM states	✗	✗	coverage
Kurant [38]	React components	✓	n/a	F1, no routing
Oshima and Kishi [56]	Angular pages	✓	✗	6 apps, faults
Rima et al. [64]	Angular workflows	✓	conditions	7 apps, tests run
Zhang et al. [81]	SPA routes, from bundles	✓	✗	4 apps (2 SPA), vs. ZAP

Note. Evaluation basis is a reference, a metric or the tool compared with, as each work states it.

Chapter 3

Static navigation analysis and scope attribution

This chapter defines the graph model and its soundness claim (Section 3.1), scope attribution for RQ3 (Section 3.2), and its partial implementation (Section 3.3).

3.1 Graph model and soundness

Definition 3.1 (UI transition graph)**.** *A UI transition graph is a labelled transition system* (S, Σ, G, M, E) *with*

$$E \subseteq S \times \Sigma \times G \times M \times S.$$

S *is the finite set of* screens*, the states of the system and the nodes of the graph.* Σ *is the finite set of* event kinds *that can be raised at a screen, such as the activation of a* `routerLink` *or a call to* `Router.navigate`*.* G *is the set of* guards*, each held as the symbolic source text of the condition under which an edge may be taken.* M *is the set of* modalities*, which has exactly three elements,*

$$M = \{must, may, unknown\},$$

and whose elements are confidence labels: a modality states how firmly the extractor asserts an edge, and never denotes a screen, a route or a navigation target. An edge $(s, \sigma, g, m, s') \in E$ *reads: at screen* s*, event* σ *under guard* g *leads to screen* s'*, asserted with confidence* m*. Both the source* s *and the target* s' *are elements of* S*, namely the first and the last component of the tuple. Constructs in the source that agree on source, event kind, guard and target yield the same edge. That edge carries the strongest modality among them, with* M *ordered* $unknown < may < must$*. So* E *holds at most one modality for each* (s, σ, g, s')*. The paragraph on witnesses below says how the constructs are traced.*

Nodes outside the model. The extractor emits two nonscreen node kinds outside the edge relation. An *unplaced origin* represents a component and navigation sites that could not be assigned to a screen. A *dynamic target* represents an unresolved destination. These preserve unplaced sites for the loss counts in Chapter 5 and unresolved destinations that may still occur at run time. Table 3.1 counts both kinds:

Table 3.1: What the published graphs are made of: nodes by kind and edges by modality.

	nodes			edges		
Application	screens	anchors	other	must	may	unknown
realworld	10	2	0	9	50	0
alfresco	27	1	26	0	36	19
ghostfolio	84	6	91	0	261	134
k8s dashboard	67	1	76	125	85	103
bitwarden	98	4	142	1	433	142
mifos	606	14	157	24	12256	152
All	892	28	492	159	13121	550

Note. An anchor stands for a whole subtree of screens; *other* counts unplaced origins and dynamic targets together. These are the attributed graphs; the node column of Table 5.2 counts the graphs of the same snapshot without attribution, which hold no anchors and fewer *other* nodes.

492 of 1412 corpus nodes (34.8 %). Of 13830 edges, 550 reach a dynamic target. Another 71 leave an unplaced origin and are labelled *may*, so Chapter 5 includes them in that tier despite their nonscreen sources. An edge to a dynamic target records outgoing navigation without identifying the destination screen. Claim 3.1 and must conversion (RQ1) concern only edges between screens.

Modalities. A *must* edge is claimed to fire for every input on which the enclosing code executes. A *may* edge is possible but conditional. *Unknown* marks an edge whose modality could not be determined, including unresolved destinations. In the measured graphs it labels only edges to the `u_` sink (Section 5.5), recording events with unresolved targets. The pinned extractor produces 550 such edges among 13830 corpus edges, across 5 of 6 applications. On every corpus edge, this modality coincides with a nonscreen target; the census rejects graphs where they differ. Unknown edges count towards totals but no rate: must conversion (RQ1) and Claim 3.1 concern the other tiers. The model also assigns *unknown* to navigation inside signal effects (Section 3.2.5), but this version emits none.

The artefact also records numeric *confidence*: 1 for must, at most 0.6 for may, and 0 for dynamic targets. This is outside the model; all figures use modalities.

The tool stores guards as source text without evaluating them. Keeping G opaque avoids representing application state.

Screens and routes. Not every configured route contributes a screen to S. Write *Routes* for the set of routes the application declares and $component(r)$ for the component that route r renders. Then

$$S \;=\; \{\, screen(r) \;\mid\; r \in \mathit{Routes} \text{ and } component(r) \text{ is defined} \,\},$$

where $screen(r)$ is the node for route r, identified by its full path: the concatenated paths of r and its ancestors. Two route kinds contribute no element of S:

- A route that declares `children` but no `component` is only a path prefix (Section 2.7). Its path is carried into the full paths of its children.

- A route whose only effect is a `redirectTo` renders nothing.

Angular rejects `redirectTo` combined with `children` or `component`, as verified in the configuration validator of the vendored Angular 18.2.14. Other routers differ: Vue Router applies a redirect only to the last matched record, so a redirecting parent renders its component for a matching child (Section 6.4).

Angular permits component-bearing routes with the same full path, distinguished at run time by `pathMatch` or `canMatch`. The model assumes no such duplicates. The extractor retains only the first route, silently for Angular and with a note for Vue and React (Section 6.4). Different paths rendering the same component produce distinct screens, requiring the attribution of Section 3.2.

Witnesses. Each static or runtime edge has a witness: its generating construct's file and line, or an observation identifier. When constructs produce the same edge, one becomes the primary witness and the others are recorded as merged witnesses (Section 4.2). The base graph requires witnesses; manual edges remain in a separate overlay. The first evaluation method checks this mechanically.

The soundness claim. Under the soundiness position of Section 2.6.2, the thesis claims only the following.

Claim 3.1 (Soundness of the must tier)**.** *Every must edge corresponds to a navigation that occurs, for every input on which the enclosing code executes, over the declared route set, under the route-order assumption of Section 6.3, and for a site whose component is not placed by a join over an ambiguous selector, within the following execution model: the event is a plain primary activation that navigates the current browsing context, no other listener the event reaches starts a navigation, no call the reader cannot resolve to a body in the application, nor any work the handler schedules for later, starts one, and all synchronous evaluation that precedes the navigation completes normally.*

The execution model is part of the claim. It excludes library calls that start a competing navigation, second listeners on the element or an ancestor, modifier-key activations or navigation into another browsing context (Section 6.3), and exceptions before navigation, whether from calls or expression evaluation. A must navigation may not complete in these executions; the claim does not cover them.

Much of the must tier depends on the route-order assumption in Claim 3.1: without it, 33 of 159 edges remain, on 22 of 25 constructs.

The must tier is therefore a lower bound only relative to four assumptions, each with a class of counterexample. *A1, route order:* a catch-all in a separately registered route table is matched after every route of the others; another registration order breaks it, and 126 must edges, on 3 constructs in 2 applications, rest on it. *A2, selectors:* no component is placed by a join over an ambiguous selector; no final must edge rests on such a join (Section 3.2.3). *A3, calls:* a call with no body in the application starts no navigation; a library call that navigates breaks it. *A4, execution model:* the plain primary activation above, in which all synchronous evaluation before the navigation completes normally; a modifier key, a second listener, another browsing context or an exception before the navigation breaks it. The edges resting on A3 and A4 are not counted, and neither is checked against a run.

What the evaluation tests. The evaluation tests only a necessary condition of the claim, which limits interpretation of Chapter 5. The witness check (Section 4.3) inspects every cited source location for agreement with the edge's destination. If the construct supporting a must label names another destination, the claim fails there. Agreement does not establish the claim. Other cited constructs support only may and are not judged against it.

Demotion conditions. Classification as must requires navigation *for every input on which the enclosing code executes*. Table 3.2 lists all conditions that demote a site to may.

Table 3.2: Conditions that demote a navigation, must → may. Every condition yields may.

Group	Condition	Example construct
Guards and resolvers	The navigation runs a guard or resolver, whatever it would decide	`canActivate` on a route it enters; a resolver, which demotes as an `await` does
	The guard list cannot be brought back to an array	a guard list built at run time
Target and source addresses	The target is the address the source already displays	navigating to the current URL, which the router ignores or follows only when a parameter or the query differs (held at may as a matter of policy, the safe side)
	The source is never displayed at its own address	a redirect answers the source's address
	Two declared routes, or a route and a redirect, can answer the address	two routes matching one path
	The base a relative target resolves against cannot be read	a relative target with an unreadable base
	The URL names an auxiliary outlet	a URL with a named-outlet segment, whose route the adapter does not read
Conditional execution	The site is under a template control-flow construct	a control-flow block, a structural directive, a container that renders lazily
	The site is in a template fragment that need not render	`<ng-template>`, an International Components for Unicode (ICU) case
	The site is under a branch, loop or fallback in code	a branch, a loop, a `catch` clause, an optional call, a default value, a callback
	The site follows an exit in its block	after an `await`, an early `return` or `throw`, a `break` out of its block
	No handler calls the enclosing function unconditionally	a module-level function, or a member of its class
	The component is mounted conditionally	a component under a template condition on that screen; a route component behind a conditional outlet

Group	Condition	Example construct
A later navigation may cancel it	Code that runs after the site, before the event is handled, may navigate	the rest of the expression holding the navigation (the body of a call it is an argument of, a right operand, a branch of a conditional whose condition it is) the statements after it in its block and in every block around it, and a `finally` block around it what runs once its function returns: what each caller in the class does after calling it, handler or not; for a constructor or lifecycle hook, the hooks Angular calls after it

A navigation runs the guards the router runs. These are the `canMatch` of every route its address is matched through. They are also the `canActivate`, `canLoad` and resolvers of every route it enters (`canLoad` is taken as run every time). Next come the `canActivateChild` of every route above those. Last come the `canDeactivate` of every route it leaves. A route the source and the target share is re-run only if its parameters can change or its `runGuardsAndResolvers` policy directs. A shell or anchor edge is judged per screen and drawn per screen where the screens disagree.

A handler is a member that the framework calls itself: a lifecycle hook, or a member that `@HostListener` registers. A handler is also a member that the template binds to an event as a bare call outside any template condition.

The router cancels a navigation if another starts before it finishes. Later code may navigate through a navigation call, including in a branch or callback, or through a call to a member of the class, a function in the file or an injected service's method whose body navigates. An awaited navigation finishes before its function continues, so subsequent code in that function does not count. Callers continue at its first await, so their subsequent code still counts. Calls without resolvable application bodies are assumed not to navigate (Section 6.3).

Exceptions before the site are not counted: almost any evaluation can throw, and Claim 3.1 assumes normal completion. An `await` is counted because request rejection is an ordinary outcome. Lazy-route fetches are not counted; chunk-load failure is a deployment fault.

How far the rules are checked. Unit tests cover each rule in both directions. A mutation sweep deletes each rule in turn and requires a test failure (Section 5.5). The tests share the extractor's interpretation; no second classifier independently derives it. The declared AI assistant checked final must-tier constructs against source with the extractor's decisions visible. This tests whether the decisions are defensible, not whether an independent reading reproduces them (Section 5.5). An earlier review of a development tier found condition-rule defects missed by the witness check (Section 4.3.2). That check establishes only source-site existence and target agreement. The stronger claim rests on the demotion rules and remains independently unverified.

Scope of the claim. This thesis claims soundness only for the must tier, relative to the declared route set. A may witness establishes that a call site exists, not that its transition ever occurs. Target agreement at the witness location is meaningful for must edges and literal-target may edges, but not for over-approximated edges. Applying that check to over-approximations produces spurious failures (Section 4.3.2, Table A.1). No soundness property is claimed for may or unknown.

The may tier as a design intent. Following modal transition systems [42], must and may together are intended to over-approximate every transition between declared screens. This intent is not an established property. Several rules under-approximate: selecting the first matching route, treating interpolation as one path segment, ordering catch-all routes last, placing dispatch and dialogs, and assuming calls without application bodies do not navigate. The graph can therefore omit intended transitions. An infeasible may edge costs precision; an omitted actual transition is a defect. No completeness theorem is claimed. The end-to-end check (Section 5.3.5) measures omissions against suite transitions; the structural references (Section 5.3.4) measure omissions only against the analysis's own relations. Section 6.3.1 sorts what is still missing by whether it lies inside the static boundary.

3.2 Scope attribution

Scope attribution uses five *input relations*. The evaluated analysis implements the following three, then adds post-fixpoint fallbacks, one nonmonotone. Termination and the least-solution property apply only to the fixpoint core:

- component nesting (N): which component's template instantiates which;
- router outlets: which components lie on the path from the root to an outlet, and below a route's component;
- the call graph with dependency injection (CG).

The derived relation *Mem* links components to their members. Dispatch (D) and guards (T), defined in Section 3.2.7, are not implemented. Thus Chapter 4 evaluates three relations, two stand-ins and a fallback post-pass.

Guards returning `UrlTree`, the injector hierarchy, and navigation inside signal effects and observable subscriptions (labelled *unknown*, Section 3.2.5) are model-only features. The extractor approximates or omits them as listed in Section 6.3. A site assigned no scope by implemented relations contributes no edge and counts as "Attribution, service" in Table 4.3.

3.2.1 The defect being addressed

Without scope attribution, only sites directly in a screen's routed component are attributed to that screen. This explains three baseline symptoms when the attribution pass is disabled (Section 3.3). Shared components contribute no edges: the shell

places the header beside the outlet, outside every screen's subtree. A component mounted at several routes contributes one edge instead of one per route. Injected-service navigation contributes none. Every baseline application nevertheless has edges.

3.2.2 Scope

The lattice. Every navigation site is anchored at a *scope*: the set of screens from which it can fire. A scope is an element of the powerset $\mathcal{P}(S)$. The abstract domain of the analysis is $\mathcal{P}(S)$ ordered by inclusion, with set union as join,

$$X \sqsubseteq Y \iff X \subseteq Y, \qquad X \sqcup Y = X \cup Y,$$

the powerset lattice over the screens. Its least element is the empty scope $\bot = \varnothing$, a site that fires from no screen at all. Its greatest element is $\top = S$, every screen.

True, computed and reference scope. The thesis distinguishes two scopes.

- $\mathit{scope}^*(x)$ is the *true scope* of a site x, the set of screens from which x really can fire. It is a property of the application and not of any analysis.
- $\mathit{scope}(x)$ is the *computed scope*, the set of screens the extractor emits edges from for x. In the model it is the fixpoint of Section 3.2.3. The prototype widens it with fallbacks that are not constraints of the model (Section 3.3).

A screen in $\mathit{scope}^*(x) \setminus \mathit{scope}(x)$ indicates a scope too small; one in $\mathit{scope}(x) \setminus \mathit{scope}^*(x)$ indicates a scope too large. This thesis establishes no true scope. For two applications, source inspection defines a third set: the *structural reference* $\mathit{scope}^R(x)$, constructed by the author for one and the AI assistant for the other. It applies the analysis's relations independently of its implementation (Section 5.3.4). Agreement with scope^R shows that the specified relations are implemented, not equality with scope^*. Elsewhere, reported scopes are explicitly identified as computed.

The empty scope. An empty computed scope can indicate two distinct situations.

- A site is *unreachable* when $\mathit{scope}^*(x) = \varnothing$: the code exists and no screen can reach it, so contributing no edge is the correct answer.
- A site is *unresolved* when $\mathit{scope}(x) = \varnothing$ because no rule of the analysis reaches it (a navigation in a service nothing is known to inject, or behind a construct the reader could not follow). The true scope is simply unknown.

Both yield $\bot$, which cannot distinguish them. To preserve unresolved sites, the extractor anchors a site in an unplaced component at an unplaced-origin node recording its file, with edges labelled *may*. A site in any other component with empty scope contributes no edge; a note records its file and reason. These traces make unresolved sites countable in Chapter 5, distinguishing them from unreachable code.

Table 3.3: Scope of each construct that can contain a navigation site.

Construct	Scope
component above the router outlet (the shell)	$\top$
component of route r	$\mathit{subtree}(r)$
component mounted at routes $r_1, \ldots, r_k$	$\mathit{subtree}(r_1) \sqcup \cdots \sqcup \mathit{subtree}(r_k)$
component rendered in the template of another component	join of the scopes of the components that render it. The prototype applies this row only to components that no route mounts (Section 3.3)
member (method, property initialiser, lifecycle hook) of a component	the scope of the component
method of an injectable service	join of the scopes of its callers
`canDeactivate` on route r	$\mathit{subtree}(r)$
HTTP interceptor	$\top$

Named elements. Three elements of the domain are named, because attribution anchors edges at them. Write $r' \succeq r$ for the reflexive ancestor relation on routes: r' is r itself or an ancestor of r in the route tree. The three elements are

$$\{\mathit{screen}(r)\}, \qquad \mathit{subtree}(r) \;=\; \{\, \mathit{screen}(r'') \in S \;\mid\; r \succeq r'' \,\}, \qquad \top \;=\; S,$$

and they form the chain

$$\bot \;\sqsubseteq\; \{\mathit{screen}(r)\} \;\sqsubseteq\; \mathit{subtree}(r) \;\sqsubseteq\; \top.$$

The chain is stated for a route with a component; for a prefix route only $\mathit{subtree}(r)$ is defined. These names anchor edges without restricting the domain. A component mounted at unrelated leaf routes has scope $\{\mathit{screen}(r_1), \mathit{screen}(r_2)\}$, none of the three named elements. Anchoring at $\top$ or $\mathit{subtree}(r)$ stores an edge once rather than per screen. Table 3.3 assigns scopes to Angular constructs.

Computing the scopes. Services require more than component-tree traversal: their scopes join their callers' scopes, including calls from other services. The model computes a fixpoint over the relations in Section 3.2.3. Outlet-derived initial scopes propagate along N, Mem and CG. This core is a monotone dataflow problem [37, 54], with two properties that do not extend to the fallbacks in Section 3.3.

- Finite height gives termination. The height of $\mathcal{P}(S)$ is $|S|$, so the scope of one node can rise at most $|S|$ times before it reaches $\top$.
- Monotonicity gives the result. The iteration reaches the least solution of the constraint system, independent of the order in which nodes leave the worklist. The transfer functions are monotone because the only operation they apply to a scope is a join, and union is monotone in both arguments.

The worklist handles cycles, including mutually recursive templates. The restrictions, fallbacks and stand-ins of Section 3.3 operate outside it.

3.2.3 Input relations

Three relations, in order of increasing difficulty, provide the input to the fixpoint.

- *Component nesting.* $A \to B$ when A's template instantiates B's selector, resolved within the correct import scope. Matching selector text globally is unsound, since two components may share a selector string in different scopes.
- *Router outlets.* Components on the path from the root to the outlet form the shell and take $\top$. Components below the component of route r take its scope *subtree*(r). A named outlet carries a second mount, which makes auxiliary routes representable without separate treatment.
- *Call graph with dependency injection.* A call links to a method only after resolving the receiver's type and its binding in the relevant injector. Resolution follows class hierarchy analysis [17]. Angular's provider declarations are modelled explicitly as the source of bindings, as the framework-modelling literature recommends [67, 3]. Where a token admits several implementations the analysis joins over all of them, which is one source of *may* modality.

Algorithm 1 adds screens only along N, *Mem* or CG, and scopes only grow. It therefore computes the least solution of Table 3.3' constraints. A component mounted by a route and rendered in another component's template receives the join of both scopes. The prototype instead blocks propagation into routed components and adds fallbacks outside these constraints (Section 3.3). Its result is not the least solution of this system or any restriction of it.

3.2.4 Guards as program points

A guard returning `UrlTree` initiates navigation, typically to login. A `canActivate` or `canMatch` guard on r redirects from the navigation's source screen, not the guarded route. Only `canDeactivate` redirects from a screen in *subtree*(r) itself. The prototype does not implement this edge class.

The model's edge guard conjoins structural conditions from screen to call site with the router guards run during navigation (Section 3.1). The measured graph omits this conjunction: structural conditions and enclosing `if` statements demote edges to may but are not included in the guard. Screen-to-screen edges record only router guards, so they identify conditional navigation without its full condition. An optional mode retains enclosing conditions, but no Chapter 5 table uses it.

3.2.5 Boundary

Navigation inside a signal effect depends on a reactive graph whose read set alone is statically visible (Section 2.7). The model labels these edges *unknown* and records the signals read as a symbolic guard. This version emits none and does not attempt to determine when they fire.

3.2.6 Anchored edges

Explicit attribution would create one edge per screen for a shell link. For the largest application, the bound is its screen count (Table 5.2) times its 1343 constructs. Anchoring removes this source-scope factor: edges are stored once at their scopes, and outgoing edges are computed on demand:

$$\mathit{edgesFrom}(r) \;=\; E_{\top} \;\cup\; \bigcup_{r' \succeq r} E_{\mathit{subtree}(r')} \;\cup\; E_{\{\mathit{screen}(r)\}},$$

where E_X contains edges anchored at scope X. The $r' = r$ term includes $E_{\mathit{subtree}(r)}$. Since the query is for a screen, $\mathit{component}(r)$ is defined; r' ranges over all ancestors, including prefixes. Other screen sets are stored per screen and included through $E_{\{\mathit{screen}(r)\}}$ in $\mathit{edgesFrom}(r)$.

Claim 3.2 (Expansion)**.** *The expansion of the anchored edge set equals the materialised edge set.*

Section 5.3 tests, rather than proves, this claim by comparing the sets on the corpus and fixtures.

Relative targets. Attribution to several screens represents possible firing from each, but relative targets do not resolve against each source screen. Angular resolves a `routerLink` without a leading slash, or a `Router.navigate` call relative to the injected route, against the route rendering the component containing the site. Its template also renders on descendant screens, where links still resolve against that component's route. Write $\mathit{holder}(s, c)$ for the nearest route at or above screen s whose component is c. A relative target at a site in component c that fires on s resolves against the path of $\mathit{holder}(s, c)$. If no route on that chain renders c, as with template-nested components, the full target's base is unresolved and the edge is may (Table 3.2). A target prefix falls back to s's address; all prefix edges are may.

3.2.7 Extensions not evaluated

Two model-only input relations were not evaluated. *Dispatch* (D) treats dispatch as call-graph edges from each dispatcher to each handler, covering NgRx effects (NgRx is Angular's state-management library) and router event subscriptions. An effect for action a receives the join of all dispatchers' scopes for a.

Guards (T) links each callable containing navigation to the entry guards it runs. A `canActivate`, `canActivateChild` or `canMatch` guard on r receives the join of scopes of sites with known targets in $\mathit{subtree}(r)$, approximating the source screens of navigation into r. With these relations, propagation along N, Mem, CG, D and T remains monotone. The prototype uses stand-ins instead (Section 3.3).

3.3 Implementation

The accompanying extractor was built for this thesis and evaluated at commit `a9d29df` (Chapter 4). A framework-independent core defines the graph model and

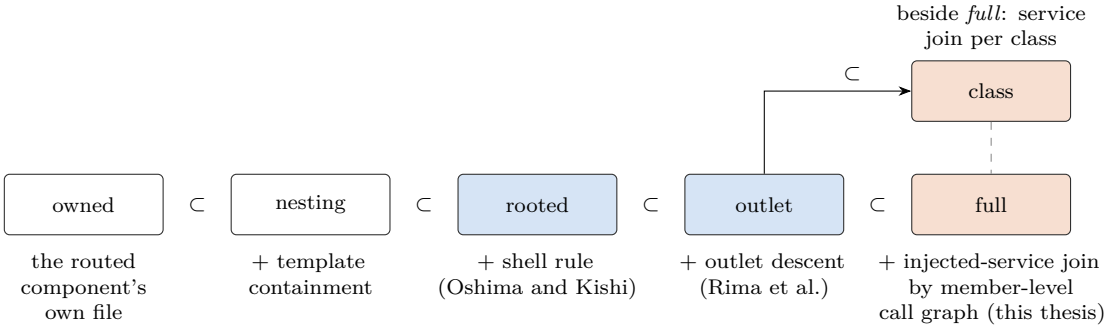


Figure 3.1: Attribution levels: ownership and containment baselines $\subset \cdots \subset$ full analysis.

Note. Blue: the levels that reimplement the two earlier Angular extractions compared in Section 5.3.3. White: the ownership and containment baselines, the first being the direct ownership of the Android literature. Orange: what this thesis adds; *class* is the per-class variant of the service join. Both orange levels also carry the fallback join and the stand-ins for guards and effects. The levels nest by construction: each withholds every mechanism above it (Section 5.3.3).

operations; framework adapters map source constructs to that model. The Angular adapter uses the TypeScript compiler API to analyse routes, templates and class code. It emits screens and edges for `routerLink` attributes and `Router.navigate` calls, with the generating rule identifier and source location as witnesses.

The Angular adapter's separate attribution pass is enabled with `--attribution`. Disabling it reproduces the baseline edge set on every corpus application. The pass implements component nesting, router outlets and the call graph, with stand-ins for D and T below.

The evaluation enables these mechanisms cumulatively (Section 5.3.3). Figure 3.1 distinguishes levels representing earlier work from this thesis's additions.

Component nesting. The pass analyses each component's selectors and template, excluding test hosts and stories. It adds $A \to B$ when A's template contains an element matching B's selector. Ambiguous matches are restricted to components imported by A's file. If none is imported, the pass joins all candidates and records a note. Such edges are not capped at *may*, so sites may be attributed to screens rendering another candidate. Claim 3.1 excludes these edges (Section 6.3). Such joins occur in k8s dashboard and bitwarden, but no final must edge cites a construct placed through one. Attribute selectors are matched similarly, assigning scopes to directives as well as components.

Router outlets. Route extraction records the route tree. A route component's template renders on its own and descendant screens, giving its sites scope $\mathit{subtree}(r)$. An unrouted component that contains the primary `<router-outlet>` and is unplaced by nesting is treated as the shell; everything it renders receives $\top$. An unrouted, unreferenced component whose subtree contains the outlet is also treated as shell. It is likely mounted through an unresolved route declaration, such as a route-table helper. Its edges are capped at *may*, protecting Claim 3.1 from an excessive scope.

Departures from Algorithm 1. The pass has two departures.

First, propagation into routed components is blocked. They retain $\mathit{subtree}(r)$

without receiving the scopes of components whose templates render them. Traversal still passes through them, so their nested components receive scopes from above. This restriction alone can only shrink a routed component's scope. Two other rules can enlarge scopes beyond the model's:

- components that nothing references and no route renders, but whose subtree holds the primary outlet, are treated as shell and given $\top$, with their edges capped at may;
- an ambiguous selector that no candidate's import resolves joins over all candidates, which Claim 3.1 excludes.

A smaller scope loses edges without inventing them, so this restriction affects recall, not Claim 3.1. Chapter 5 counts the constructs it leaves unplaced.

Second, the call graph differs from Section 3.2.3. The pass operates on class members, assigning component members their component's screens. A call `this.f.g()` is resolved through `f`'s declared type (a constructor parameter, typed field, `inject(T)` field or `@Inject(TOKEN)` parameter). The call resolves to:

- the class that type names;
- the classes a provider binds it to with `useClass` or `useExisting`; or
- for an abstract class no provider binds, its subclasses in the application.

This is class hierarchy analysis [17] restricted to the declared type. Bindings are collected application-wide rather than per injector. Different bindings for one token are joined, which can only widen scopes. Factory and value providers are not analysed; calls on types declared outside the application contribute no edges.

The seeded call-graph iteration reaches its constraints' least solution. Service scopes then receive two fallbacks:

- a member no caller reaches falls back to the join over every component that injects its class, the per-class approximation that the *class* level of the ablation applies to every service (Section 5.3.3);
- a member reached by the iteration receives additional screens from callers not represented in the graph: that caller's class join is added, or this class's own join if the caller is unplaced.

Neither fallback belongs to Table 3.3. The first is nonmonotone in the call graph: adding a call edge can narrow scope. Emitted service scopes are therefore the least call-graph solution widened by fallbacks, not the least solution of Table 3.3. Service sites remain *may* because conditions along their call paths are not analysed.

Anchored edges. Edges with scope $\top$ are stored once at a `scope` node. Edges scoped to exactly $\mathit{subtree}(r)$, where r is a screen with a descendant, are stored once at a node listing its screens. Prefix routes that are not screens are not anchored;

other sets are stored per screen. The core provides $edgesFrom(r)$ and materialisation of all anchored edges.

Stand-ins for D and T. Two approximations replace unimplemented relations. They are withheld below Figure 3.1's *full* level in Section 5.3.3, so lower levels exclude their sites.

- *Guards (for T).* A guard receives the screens of routes naming it as a bare identifier in `canActivate`, `canActivateChild`, `canMatch` or `canDeactivate`. It receives each route's own screen, not $subtree(r)$, and none for componentless prefixes. This is narrower than the model for `canDeactivate`. For entry guards it differs from the model, since redirects leave the navigation's source screen. Scope may therefore be too small when navigation starts elsewhere, and too large when it does not start at the guarded route.
- *Effects (for D).* An effect, being registered for the lifetime of the application, is given $\top$.

Both produce only *may* edges. Classes not reached by calls or injection remain without scope and count as "Attribution, service" in Table 4.3. One baseline change moved inline-template witnesses from the template's first line to the construct's own line. Baseline graphs are identical with and without this change on the corpus. Unit tests cover each case above.

3.4 Summary

The graph model uses three modality labels. Its must-tier soundness claim is relative to declared routes, route order and an execution model; evaluation tests only a necessary condition. Scope attribution computes screen sets as a least fixpoint over five relations. The prototype implements three, substitutes stand-ins for dispatch and guards, and adds fallbacks outside the model. Chapter 4 describes the measurement.

Algorithm 1 Scope attribution as a worklist fixpoint over $\mathcal{P}(S)$.

Require: routes R with route tree; for each route r its entry guards $\mathit{entry}(r)$, named in `canActivate`, `canActivateChild` or `canMatch`, and its `canDeactivate` guards $\mathit{leave}(r)$; components C; nesting $N \subseteq C \times C$; HTTP interceptors I; call graph CG; dispatch relation D; guard relation T, linking a callable x to $g \in \mathit{entry}(r)$ when x holds a navigation site whose target lies in $\mathit{subtree}(r)$

Ensure: $\mathit{scope}(x) \in \mathcal{P}(S)$ for every component and callable x; the callables include the members of components and of services

for all x **do** $\mathit{scope}(x) \leftarrow \varnothing$
end for
for all $r \in R$ and c the component of r or a layout wrapping it **do**
    $\mathit{scope}(c) \leftarrow \mathit{scope}(c) \cup \mathit{subtree}(r)$ ▷ own screen and outlet descendants
end for
for all c on the nesting path from the root component to the holder of the root-level primary outlet **do**
    $\mathit{scope}(c) \leftarrow S$ ▷ application shell, $\top$
end for
for all $r \in R$ and $g \in \mathit{leave}(r)$ **do**
    $\mathit{scope}(g) \leftarrow \mathit{scope}(g) \cup \mathit{subtree}(r)$ ▷ *model only*: runs as the user leaves r
end for
for all HTTP interceptor $i \in I$ **do** ▷ *model only*: registered for the application
    $\mathit{scope}(i) \leftarrow S$
end for
$W \leftarrow$ every x with $\mathit{scope}(x) \neq \varnothing$
while $W \neq \emptyset$ **do**
    take x from W
    for all y with $(x, y) \in N$ **do** ▷ x's template renders y
        Propagate(x, y)
    end for
    for all members m of x, if x is a component **do** ▷ a member runs where its component is shown
        Propagate(x, m)
    end for
    for all y with $(x, y) \in CG \cup D \cup T$ **do** ▷ CG: member level, bindings per injector; D, T: *model only*
        Propagate(x, y)
    end for
end while
procedure Propagate(x, y)
    if $\mathit{scope}(x) \not\subseteq \mathit{scope}(y)$ **then**
        $\mathit{scope}(y) \leftarrow \mathit{scope}(y) \cup \mathit{scope}(x)$; add y to W
    end if
end procedure

Note. The relations marked *model only* belong to the full analysis and are not implemented in the prototype evaluated in Chapter 5; the prototype's other departures, among them the stand-ins for guards and effects and the rule for routed components, are listed in Section 3.3.

Chapter 4

Evaluation setup and research material

This chapter defines the corpus (Section 4.1), metrics (Section 4.2) and evaluation methods (Section 4.3) used to evaluate the analysis of Chapter 3. Chapter 5 reports the results.

4.1 Corpus selection and validity

4.1.1 Selection

Ten candidate applications were screened against six criteria: a real application rather than a library or template; a nested, lazily loaded route tree; guards present; end-to-end test configuration for a browser driver; retrievable history in which tests and source change together; and an active repository. Six candidates were retained and pinned to commits for repeatable measurement (Table 4.1). The criteria were fixed before any results were obtained.

Criterion (4) required configuration, not a usable suite. At the pinned commits, only alfresco has specifications the test-source cross-check can parse; ghostfolio and bitwarden have no browser-test configuration. The criterion was applied inconsistently: `gravitee` was rejected for a sparse UI suite while two applications with none were retained. Route-tree size and end-to-end test density were inversely related. Methods requiring test suites therefore cover only applications with suites; the other methods cover all six.

Four candidates were rejected; the counts were read from the repositories and are not generated. `angular/components` is a component library with no product router. `vendure` has 169 end-to-end specifications, all of which exercise the GraphQL server and none the user interface. `gravitee` declares 618 routes but has 14 UI specifications. `SAP/spartacus` has a large Cypress suite, but the extractor does not support its configuration-driven routing. Tool support was not one of the six criteria; this ground was added after screening began.

4.1.2 Validity of an open-source corpus

The selection follows Kalliamvakou et al.'s recommendation to use and report explicit criteria [34, 35], with the departures stated above. External validity remains limited: open-source applications may differ from proprietary ones. Selection bias is also possible because I selected the corpus and wrote the extractor. Selection was not blind to tool support, which became a rejection criterion after screening began. Pooled percentages are dominated by mifos, so per-application figures are the primary summary where available. The corpus spans nearly two orders of magnitude in route counts; the pinned commits and released harness permit repeated measurement.

The extractor was also developed on this corpus, so its measurements are fitted to it (Section 6.2). The sole out-of-sample measurement compares the frozen extractor with Rima et al.'s ground truth on their seven subjects, preregistered before extraction (Section 5.3.7). A proprietary Angular case study reporting aggregate figures is planned to address external validity (Chapter 6). The literature reviewed in Chapter 2 provides no standard benchmark corpus of single-page applications; the corpus and manifest are therefore an output of this thesis.

4.2 Metrics

Table 4.2 defines the measures for each research question. RQ1 has two levels: the share of declared screens with a node and the share of navigation constructs yielding at least one must transition. Both are needed because a transition requires its endpoint screens. For RQ2, defect classes require two lines of evidence, at least one from an evaluation method (Section 4.3). The guard class, accepted on source review alone, is the exception (Table 5.4). Every unrecovered construct is assigned to one loss category. RQ3 compares the same extractor with the analysis disabled and enabled, using the references below. RQ4 uses mutation testing as a witness-mapping sanity check, not an independent test. For RQ5, source analysis assigns each defect class of Table 5.4 to one of three causes: resolved by the final version (a tool defect), still unresolved but statically resolvable, or beyond a static route-table reader. This produces a cause list, not a ratio. Every site without a transition must carry a label (Section 5.5).

Navigation construct. A navigation construct is a `routerLink` binding in a template, or a call to `Router.navigate` or `Router.navigateByUrl` in class code, in the application source excluding tests. A separately written parser finds constructs and shares no code with the extractor. Its independence is qualified by Table A.1: both tools use the same resolution rules for route-path forms added during development, and three parser corrections followed disagreements with the extractor.

Must conversion (RQ1). Must conversion is the fraction of in-scope constructs that are witnessed by at least one must edge. Three rules make it precise.

- A construct is matched to a witness on its own line or on a line either side, to absorb offsets in the extractor's locations. Two constructs on adjacent lines

can therefore both be credited by one witness, so the ratio is an upper bound at line granularity.

- Query-only navigations and the two exclusions of Table 4.3 are left out of the denominator as well as the numerator.
- When several constructs produce the same edge, the extractor keeps the most certain as the witness and records the others as merged witnesses, each with the modality it earns on its own. A construct counts as converted only where it earns must itself.

The other ratios. Several measures have been called coverage in prior work, but here they differ by more than an order of magnitude: 99.2 % of sites are accounted for and 1.4 % convert to must. Node coverage rises when the parser recognises less of an application, so it is reported with a blind-spot count. Accounted share measures whether losses are declared, not how much is recovered. The loss tables measure the unrecovered constructs, or recovery gap.

Agreement between a scope and the rendered interface (RQ3). The measures above do not test whether a site's computed scope matches the screens from which it can fire. The *rendered-interface check* tests one direction using the runtime route oracle's harness (Section 4.3). It navigates to each screen in the graph, extracts in-app links from the DOM and compares them with the graph's per-screen edges.

A rendered link without a corresponding edge shows that a site was under-attributed or not extracted. The converse is weaker: links may render conditionally, and the harness does not sign in. An unresolved target also yields reduced-confidence edges to every address matching its static prefix; these do not assert that a link exists. The converse is therefore assessed only for exact destination predictions, whose count is reported. Three restrictions apply. Only template sites are judged because event-handler calls to `Router.navigate` leave no DOM link. Guard-redirected screens display another screen's links and cannot be judged; they are counted separately and constitute most screens here (Section 5.3.1). Ghostfolio's per-locale build is incompatible with this harness, so this check excludes it; every other method includes it.

The ablation and the structural reference (RQ3). These measurements test whether attribution improves agreement with a structural reference or merely produces larger screen sets.

The ablation withholds the mechanisms of Section 3.2 one at a time and extracts the whole corpus at each level:

- *Owned*: a site belongs to the screen whose routed component's own file holds it, the direct ownership of the Android literature.
- *Nesting*: adds component containment through template element selectors.
- *Rooted*: adds the shell rule, so a component the root renders is attributed to every screen. Oshima and Kishi assemble their page information from the root

component as well (Section 2.4), so this level measures how far the prior work reaches here.

- *Outlet*: adds router-outlet descent.
- *Full*: adds the member-level call graph, with its fallback join and the stand-ins for guards and effects.

Each level adds to its predecessor; *full* adds several mechanisms. A sixth level, *class*, replaces the call graph with the per-class rule, joining a navigating service to every component that injects its class. The harness verifies nesting on the application graphs and refuses to produce a table if any (site, screen) pair disappears at the next level.

The structural reference supports scoring in both directions. For one application, source inspection identified each site's screens with a derivation in `route-oracle/ground-truth/realworld.json`. This is the structural reference of Section 3.2.2, termed ground truth below. A missing reference pair indicates under-attribution; an extra pair indicates over-attribution, both relative to the reference. Four limits apply.

- On realworld it is one application, small enough to read exhaustively. A second, alfresco, was built under the same rules by the AI assistant from the sources alone, without the extractor's output (`route-oracle/ground-truth/ALFRESCO-PROTOCOL.txt`); it leaves 17 sites undetermined (Section 5.3.4).
- The realworld reference is the author's reading, so it is not independent of the tool's author. It is published with its reasoning so a reader can check it.
- Its rules are the analysis's own relations, and a template condition or guard that admits a screen only in one sign-in state does not narrow a scope under them. The ground truth therefore tests whether the implementation applies the relations correctly, not whether they match what the running application renders.
- It is checked in one direction only: the scorer reports every site the graph places that the ground truth omits.

The end-to-end navigation check (RQ3). The structural reference applies the analysis's own relations, so it cannot show that they miss a navigation the application performs. This check takes its reference from execution. Two applications ship an end-to-end suite that can be run here. The first, realworld, runs against the public demonstration backend, which is not pinned. The second, mifos, runs against a local Fineract server with the image pinned by digest. A small module added to each application logs every navigation the router completes while the suite runs, with the route patterns of the screen left and the screen reached.

An observed pair consists of source and target route patterns for an application-triggered navigation. Back-button navigations and the first screen after a full page load are excluded. A pair is *contained* if any must or may edge connects its screens

after scope expansion, ignoring parameter names. Unknown targets never count. A hit need not come from the site the test used, so it does not validate an individual site's scope. Redirects are compared on the requested address, as preregistered. The scorer initially departed from this rule (Table A.1); that figure is not used.

A missed pair is classified automatically from the graph by the first matching rule:

- *redirect*: the navigation was redirected and the graph holds the pair on the address it landed on (the preregistered redirect class);
- *base*: the target is reached from a screen above the source but not from the source, as a relative link resolved against the wrong route appears;
- *dynamic*: the source screen's routed component, or one above it, holds a site whose target the extractor records as unresolved;
- *other*: none of these.

The base and dynamic classes were defined after inspecting the misses. Dynamic counts only misses not assigned to base. Every mifos miss met the dynamic test regardless of order, so that count alone was uninformative. Section 5.3.5 identifies the sites repaired after the check.

Except for that departure, the counting rules, application-specific predictions and three validity checks were recorded before inspecting logs. The checks require matching test outcomes with and without instrumentation, at least ten distinct pairs, and the same pair set in a second instrumented run. The experiment measures containment only for exercised transitions. Its interpretation depends on graph density: realworld contains 71/100 possible screen pairs, with 19/19 hits; mifos contains 162567/367236, with 19/22 hits. Uniformly sampled pairs at these densities would yield 13.5 and 9.7 hits. This is a reference, not the expected success of a suite that chooses its paths. Untested edges are not errors, so the check cannot measure precision. One run cannot refute a must edge.

Loss breakdown (RQ2). Each construct is matched to the edges whose witness cites it. A construct that is in scope, cited by no must or may edge and not accounted for is *lost*. It is assigned to exactly one loss category by the first rule in Table 4.3 that applies. The categories partition the lost constructs. Named outlets are excluded before assignment, and guards are assessed separately. Constant target, Node missing and Other were not assessed under the defect-class admission rule (Chapter 5).

Guard defects mislabel rather than remove edges, so they are absent from this table. No method directly validates guard labels. Guard handling is assessed only by guard-removal mutation and source review (Section 5.5); red-team review of the graph and source found the defect.

The nested branch classifies the *owning* route component. A node-category construct can still convert if attribution places it through a shell or subtree (Section 5.3). *Other* counts constructs requiring manual inspection, not a single cause.

Change in recovery and must-edge target consistency (RQ3). Must conversion and the loss breakdown are measured at one extractor commit on identical

inputs, with attribution disabled and enabled. Disabling attribution reproduces every application's baseline edge set exactly (Section 3.3).

Must-edge target consistency requires two conditions: the witness line contains a construct of the claimed kind with the claimed target, and the independent route set declares the destination node's address. Without the latter, a node at an address the application never mounts could pass.

Unread destinations are neither passes nor failures. Consistency is bounded by the share of must edges that pass and the share that do not fail. Recovery and consistency alone cannot assess RQ3: attributing every site to every screen would raise recovery without reducing must conversion or target consistency. Three measures are therefore considered together: recovered share and must conversion over the same denominator; scope width against that trivial assignment (Section 5.3.2); and agreement with the two structural references, the application suites and Rima et al.'s tool (Section 5.3.6). These references do not derive solely from attribution output. Consistency bounds are reported for both runs but cover different must-edge sets, so they are not compared as rates.

Detection rate (RQ4). This sanity check tests whether an injected source change produces the predicted graph change, reported per operator with its n. It does not establish change-impact accuracy. Detection is nearly guaranteed because mutants are sampled from the graph, and the harness was tuned on its misses (Section 4.3.2). Changes to unrepresented constructs are excluded rather than counted as misses: they repeat recovery gaps measured by the loss tables. The rate is conditional on recovery and reported with the exclusion count.

4.3 Evaluation methods

Figure 4.1 locates the five methods in the extraction pipeline. Table 4.4 states their measures and blind spots; none establishes correctness alone. RQ3 adds four checks not listed there: the rendered-interface check, ablation, structural reference and end-to-end check. Figure 4.2 identifies their graph targets and reference sources.

Four of the five read the application's sources, and two readers of the same text can be wrong in the same way. The fifth, the runtime route oracle, was added for that reason.

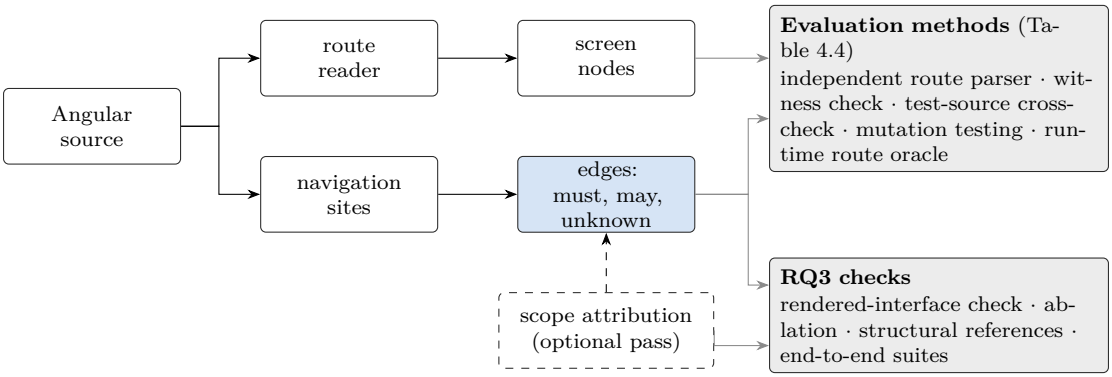


Figure 4.1: What is evaluated: the extractor, scope attribution and the evaluation methods that check the resulting graph.

Note. The extractor reads route tables into screen nodes and navigation sites into edges, tiered must, may or unknown by the demotion rules; scope attribution, an optional pass, places sites that belong to no single screen. The five methods of Table 4.4 check nodes and edges against independent readings; the RQ3 checks examine the attribution.

Table 4.4: Evaluation methods, the quantity each measures, and the blind spot each has.

Method	Quantity	Blind spot
Witness check	Must-edge target consistency. For every must edge, the witness file is re-parsed at the claimed line, the construct kind and target are compared, and the destination node's address is tested against the independent route set.	Cannot see missing edges. Target agreement is invalid for over-approximated may edges, whose witness proves only that the call site exists. Where both tools read a table at the wrong prefix, they agree on a wrong address (Section 5.5.3). Target agreement is literal, so an edge to the screen a redirect lands on is reported as a failure.
Differential test	Disagreements with a separately written route-configuration parser.	Diagnostic, not a metric. The independent route parser is independent, not stronger, and for the path forms added during development (Table A.1) it is no longer independent of the extractor at all.

continued on the next page

Table 4.4 continued from the previous page

Method	Quantity	Blind spot
Test-source cross-check	Node-level recall against a fourth reader of the source. The URL literals in the application's end-to-end specifications are collected; a URL its own test suite names is an address its authors expect to exist, and the extractor is asked whether it minted a node there.	Reads test source text and runs nothing, so a collected string is an address a test *names*, not one a test was observed to reach. Tested subset only. Textual extraction under-samples click-driven navigation. A direct `goto` is not a UI transition, so visit pairs are not counted as edges.
Mutation test	Detection rate over four operators: rename a path, remove a guard, delete a `routerLink`, retarget a `navigate`. Ground truth is the authored delta. Detection is judged on the node and edge sets and on the witness set, since several constructs can produce one edge.	A mutant whose construct is absent from the baseline graph re-detects a recovery gap and is excluded, as is a guard mutant on a screen that no edge enters. A component several routes render maps to every screen those routes declare, which errs towards scoring a mutant rather than dropping it. Detection is conditional on recovery, and the attributed and unattributed graphs do not exclude the same mutants, so they are also compared over the mutants both can score, paired by operator, file, line and subject. The sample is drawn from the graph, so it is not independent of the artefact under test.

continued on the next page

Table 4.4 continued from the previous page

Method	Quantity	Blind spot
Runtime route oracle	Node-level soundness against the framework itself. Each application is built in its development configuration, loaded in a headless browser, and asked for the route configuration the Angular router holds, with every lazily loaded subtree expanded through the application's own `loadChildren`. The addresses the extractor mints are compared with the addresses the framework mounts. Nothing re-reads the route files. Which addresses count as mounted is itself a reading, so the harness navigates the running application to a list of addresses and records what the recogniser made of each, before any `canActivate` guard runs and after the `canLoad` and `canMatch` guards that take part in matching.	Requires the application to build and boot without its server. It reads the configuration and not the rendered screen, so a route that a guard always refuses still counts as mounted. An address claimed through a `UrlMatcher` cannot be decided from the configuration. A workspace application that composes the measured library mounts the shell's own routes as well, so part of any difference is scope and not reading. Its result files record no extractor commit or runtime version (Section 4.3.3).

4.3.1 Preconditions

Two properties of the extractor were checked before any measurement. *Determinism*: two extractions from the same unchanged source give identical nodes and edges. *No accumulated state*: extracting into an existing database gives the same graph as extracting after deleting it. Without them, differences seen in mutation testing could not be told from noise.

4.3.2 Corrections made to the protocol

The thesis distinguishes three extractor versions: *early development* (commit f17a302), before evaluation-driven corrections; *intermediate* (extractor eb9dab80, with 20 must edges, tables printed at 7d109c7); and *final*, whose results are presented throughout. Only the intermediate must-tier figures are cited as printed: the underlying runs were lost, and an archived rerun produced a different loss ledger. Superseded instrument figures cannot be regenerated with the released harness and are not cited as results. Unexpected figures are checked against their inputs before reporting, because several

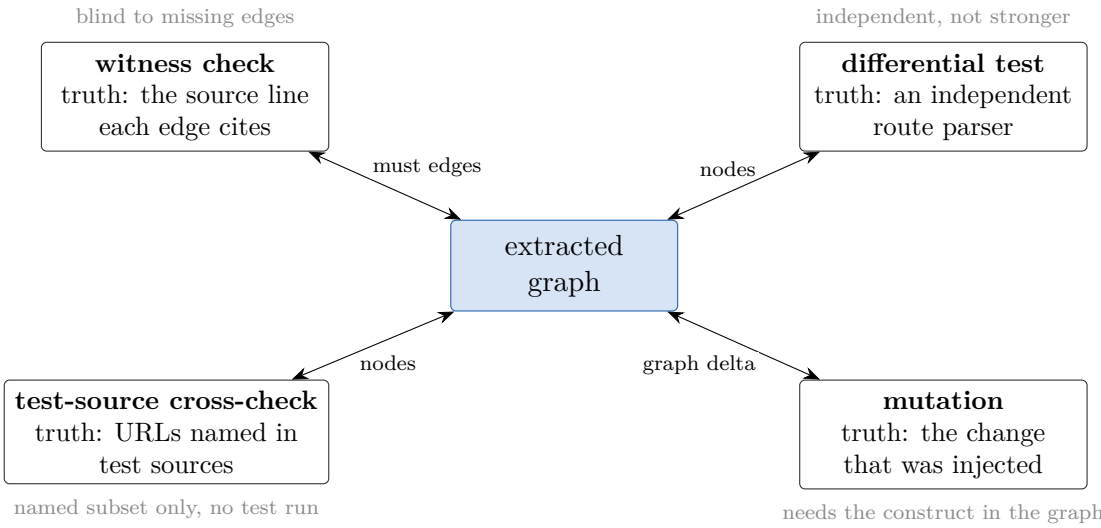


Figure 4.2: The four evaluation methods that read the applications' sources, the graph part each checks, and its ground truth.

Note. The fifth method, the runtime route probe, is not drawn because its ground truth is the running application rather than any part of the graph. No method sees everything; the grey notes give the blind spot of each, as listed in Table 4.4.

arose from measurement defects rather than extractor behaviour.

The mutation harness was corrected after inspecting its misses, so the final run is not independent. It uses the same seed but a partly different sample: 46 of 73 mutants share an operator, file and line with one of the earlier 70, although destinations may differ. 6 of the 9 inspected misses are absent from the final sample's locations.

Appendix A lists every correction, one row each, in Table A.1.

Availability. The extractor and harness are in the accompanying repository under the MIT licence. Rima et al.'s tool was run from its public repository at commit `f33003f`; it carried no licence when checked on 5 November 2026, so it is not redistributed.

4.3.3 Reproducibility

Scripts generate numerical macros; `REPRODUCE.md` lists each table's script and commit. The final results use extractor `a9d29df`, after repairs prompted by Rima et al.'s subjects (Table A.1), with harness 1.10.0 and mutation harness mutate 1.9.1. The demotion-rule sweep and may-reason census use the same commit. The pre-repair end-to-end figure (Table A.1) uses the archived graph from `c42f220`. The first Rima comparison uses `2aac284`, frozen before its evaluation-driven repairs as preregistered (Section 5.3.7). Two measurements remain historical: the rendered-interface check (Table 5.11) and the runtime route configurations recorded on 29–30 October without an extractor commit. The latter are compared with the final graph (Section 5.5.3).

Table 4.1: The corpus: commit, route count and role in the study of each application.

Application	Commit	Licence	Routes	Role in the study
gothinkster/angular-realworld-example-app	`dd99ed2c`	MIT	14	size floor; all literal paths
alfresco/alfresco-content-app	`b88d47f4`	LGPL-3.0	≥57	87 Playwright specs including a navigation suite
ghostfolio/ghostfolio	`9884fa0b`	AGPL-3.0	≥135	constant-valued route paths; self-hostable
kubernetes/dashboard	`0ba796d9`	Apache-2.0	140	mid-size control
bitwarden/clients (web)	`e40ce6f0`	GPL-3.0‡	≥168	many guards, guard factories among them
openMF/web-app	`7f781796`	MPL-2.0	≥909	size ceiling

Note. Licences are read from the local checkouts at the pinned commits; ‡the `bitwarden_license/` directory is under the Bitwarden License. Routes are the route entries reached by the independent route parser of Section 4.3, as it stands in the final version (Section 5.5). A count marked ≥ is a lower bound: the parser records a blind spot for that application, so the entries behind it are not in the figure; Table 5.3 gives how many blind spots each application has. The marks are on alfresco, ghostfolio, bitwarden, mifos, on these grounds: children not resolvable to a route array; matcher route: path not statically knowable; spread route source: not followed.

Table 4.2: The ratios and measures of the thesis, with numerator, denominator and research question.

Name	Numerator	Denominator	RQ	Reported in
Node coverage	declared screens that have a node	screens the independent route parser declares	RQ1	5.1
Must conversion	in-scope constructs witnessed by a must edge	in-scope constructs	RQ1	5.1
Recovered share	in-scope constructs witnessed by a must or may edge from a placed screen	in-scope constructs	RQ1, RQ3	5.3, 5.5
Accounted share	recovered constructs plus in-scope constructs recorded as dynamic or unplaced	in-scope constructs	RQ2	5.2
Must-edge target consistency	must edges whose witness passes (lower bound) or does not fail (upper bound)	all must edges	RQ3	5.5.3
Node-level recall	distinct URLs named in an application's end-to-end test sources that have a node	distinct URLs so named	RQ1	5.1
Structural-reference recall and precision	(site, screen) pairs the reference and a level share	pairs in the reference (recall) or pairs the level places (precision)	RQ3	5.3.4
End-to-end containment	observed screen pairs joined by an edge of the graph	observed screen pairs	RQ3	5.3.5
Scope width	mean screens a site is attributed to	the same when every site has scope $\top$	RQ3	5.3.2
Detection rate	injected changes that produce the predicted change in the graph	injected changes the graph can score (excluded mutants reported)	RQ4	5.4
Cause trace	defect classes traced to a cause in the source, by kind (read by the final version, still not read, beyond a static reader); no ratio	defect classes	RQ5	5.5

Note. Each measure has one name, used throughout. A construct is *in scope* when it is neither a query-only navigation nor excluded by the first rows of Table 4.3. Node coverage is the only measure called coverage in this thesis.

Table 4.3: Rules that classify a construct, in the order the harness tests them.

Category	The construct ...
Query only	changes only the query string of the current screen (`navigate([], {queryParams})`), so it is not a transition and is not counted as a loss
Named outlet	targets an auxiliary outlet (`{outlets: ...}`)
No commands	names no destination at all
Converted, must or may	is cited as witness by an edge of that tier from a placed screen
Accounted, dynamic	is cited by an edge into a sink: the extractor records its destination as unresolved
Accounted, unplaced	is recorded by the extractor as a site it read but could not place
Attribution, service	has no owning component, or its owner is a class that is not a component: a service, guard, interceptor, resolver or effect. A site in a template whose owner cannot be resolved falls to *other* instead
Attribution, shared	is in a component that no route renders directly
Owner is a route component, but that component has no node:	
Component unlinked	a node for the owner's route exists, but the extractor did not link it to the component
Constant path	every declaration of the owner's route gives its path as a constant reference rather than a literal
Other	some declaration of the owner's route does have a node
Lazy boundary	some declaration of the owner's route is in a lazily loaded routes file
Node missing	a declaration exists but none of the above applies, or neither parser found a route for the owner
Constant target	owner has a node, but the construct's own target is not a literal
Other	matches none of the above; inspected by hand

Note. The first three rows take a construct out of the denominator, the next counts it as recovered in the tier that cites it, and the two after that as accounted for. Only a construct that reaches the rows below is lost. The indented rules are the nested decision taken when the construct's owning route component has no node.

Chapter 5

Results

This chapter reports measurements from the harness of Section 4.3 on the corpus of Section 4.1. Sections 5.1–5.4 answer RQ1–RQ4: losses, defect classes, scope attribution and mutation sensitivity, respectively. Section 5.5 traces loss causes (RQ5), and Section 5.6 reports dead-link candidates.

The numerical tables are generated from the harness result files; summaries compiled by hand are marked as such. The development versions and the corrections between them are listed in Section 4.3.2.

All extractions use source trees without installed dependencies. The corpus was selected with tool support in view, so results are likely optimistic for Angular applications generally. Section 6.2 discusses both conditions.

5.1 The two levels of loss (RQ1)

RQ1 measures declared screens represented as nodes and navigation constructs yielding deterministic transitions (Section 4.2). Nodes are a precondition for transitions, but the first ratio does not numerically bound the second: their units are screens and constructs, respectively.

Table 5.2 gives the second measure on the final snapshot. The constructs that earn must number 21 in the whole corpus without attribution and 25 with it. They lie in 4 applications with attribution, and the other 2 have none. Edge counts must be read per tier and not summed as distinct constructs, because one unresolved site can produce many over-approximated may edges. An optional measurement switch records every condition that demotes a may witness. Disabling it leaves output unchanged on all six applications (Section 4.3.3). Table 5.1 groups reasons for 1657 may-only witness locations. 1631 involve an application decision identified by the extractor: a guard or resolver, branch, callback or template condition. Only 26 remain at may solely through extractor uncertainty. Under the present rules, better resolution alone could promote at most those to must; proving that a guard always admits or a branch always executes could promote more. The small must tier thus follows largely from its definition. A guard that always admits still causes demotion: the census identifies decision constructs, not runtime conditionality. Mifos dominates these counts, with 1289 of 1657 locations.

Table 5.1: Why the may-only witness locations are may, by family of reason.

			Any, by application					
Family	Any	Sole	realworld	alfresco	ghostfolio	kdash	bitwarden	mifos
async	333	10	4	0	15	5	61	248
branch	231	9	4	4	17	1	85	120
template	790	31	30	3	37	7	30	683
guard	1518	174	16	9	99	11	202	1181
reader	127	1	1	1	0	1	3	121
attribution	194	0	1	4	8	3	94	84
target	690	4	13	3	29	5	97	543
same screen	203	0	8	3	14	1	6	171

Note. Any counts the witness locations with at least one reason of the family, *Sole* those whose every may witness has that one reason only. The application's own decisions are the async, branch, template and guard families; reader, attribution and target are the extractor's uncertainty; same screen is neither.

Table 5.2: Must conversion per application, baseline.

Application	Constructs	In scope	Nodes	Must	May	Must conversion
realworld	40	39	10	1	23	2.6%
alfresco	34	27	37	0	7	0.0%
ghostfolio	198	186	112	0	61	0.0%
k8s dashboard	72	68	72	0	7	0.0%
bitwarden	131	112	113	1	89	0.9%
mifos	1343	1343	642	15	4429	1.4%

Note. Must conversion is the fraction of in-scope constructs whose own witness earns must; the edge columns count edges. *Nodes* counts every node of the baseline graph: the screens of Table 5.3 together with its unplaced origins and dynamic targets (Table 3.1 counts the attributed graphs, which add anchors and more of the latter). The baseline and attributed graphs are extracted from the same final snapshot on identical corpus inputs.

Table 5.3 compares extracted nodes with the independent route parser's declared screens. Every declared screen has a node on all six applications, relative to that denominator.

The two readers still disagree about a handful of addresses. Two qualifications apply.

- The denominator is the screens a *static reader* can enumerate. Against the addresses the running framework mounts, the same node set is 892/1146 (Section 5.5.3). Almost all of the difference is one construct in one application that the extractor declares unread.
- The two static readers were partly corrected against each other, so their agreement is weaker evidence (Section 6.2). The runtime route oracle is the external check, and it finds no node at an address where the framework mounts

Table 5.3: Node extraction against the independent route parser, in both directions.

Application	Declared	Nodes	Matched	Coverage	Unmatched	Blind spots
realworld	10	10	10	100.0%	0	0
alfresco	26	27	26	100.0%	1	10
ghostfolio	84	84	84	100.0%	0	2
k8s dashboard	67	67	67	100.0%	0	0
bitwarden	98	98	98	100.0%	0	1
mifos	605	606	605	100.0%	1	1
All		892			2	

Note. Declared screens that have a matching node, and nodes for which the independent route parser holds no declared screen. The blind-spot column counts route entries the parser could not follow (an unfound lazy route array, an unresolvable path expression, a custom matcher, or a route source spread from a call it does not follow), which marks where either direction is incomplete. The node set is the same with scope attribution off and on, since attribution mints no nodes.

nothing (Section 5.5.3).

This completeness concerns only resolvable screens. Blind-spot counts identify but do not bound omissions: one unresolved route entry can conceal many screens (Section 5.5.3).

5.2 Which failure is worth an analysis (RQ2)

Table 5.4 separates evidence from named evaluation methods and other observations. The guard class has no named method. Attribution, addressed by Chapter 3, relies most heavily on observations among the other classes; these are not independent instruments.

Five classes were identified during development, each with at least two confirmations. A sixth, nodes unlinked to their rendering component, has only the parser–graph comparison as evidence. It remains a candidate and has no instances in the final version.

Table 5.5 and Figure 5.1 classify 1818 constructs under Table 4.3, recording their tier or loss reason. A construct counts as must only through its own must witness. Rows are loss categories, distinct from the defect classes of Table 5.4; guards are assessed separately (Section 4.2).

Under the loss rules' precedence, attribution is the largest loss category in every application. The two attribution categories contain 454 of 1818 constructs; no other loss category contains a tenth as many. Unresolved destinations are counted separately and number about a third as many.

Ownership is tested before the target. Among attribution losses, 101 targets fail the literal test; reversing this order would make target losses larger than attribution on alfresco and k8s dashboard.

Constant paths, lazy boundaries and unlinked components have no instances in

Table 5.4: Defect classes and the evidence for each.

Class	Evidence	Methods	All
Attribution: navigation outside route components is invisible	mutants excluded on the baseline graph; multi-mount edge absent; manual audit; an application with nodes and no edges on a development version (k8s dashboard; Section 4.3.2)	1	4
Lazy module boundary not followed	independent route parser; test-source cross-check; mutation	3	3
Guards unreliable in both directions	red-team panels and a source survey of the guard factory and the call-form `canActivate` (Section 5.5.1)	0	2
Named router outlets not implemented	test-source cross-check; source survey	1	2
Constant-valued route paths not resolved	independent route parser; node-count collapse	1	2
Candidate: node not linked to its component	independent route parser against graph	1	1

Note. A class is admitted on two lines of evidence, at least one from a named evaluation method; the methods column counts only those. The guard class is the one exception: both its lines of evidence are source review, and the mutation run neither confirms nor refutes it (Section 5.4). This summary is compiled by hand; it is not harness output.

Table 5.5 because the final extractor resolves their causes (Section 5.5.1). Under the stated precedence, the remaining unplaced navigation mostly lacks a single owning screen, motivating scope attribution.

Category interactions mean that the 454 attribution constructs neither lower-bound nor upper-bound attribution's recoveries (Section 5.3).

Table 5.6 reports the witness check on the final attributed tier and the test-source cross-check. Section 5.5 treats the must tier separately. 623 edges in 3 applications have their only witness outside the application directory; both tools read those files, so their agreement there is not independent evidence.

5.3 Scope attribution (RQ3)

The evaluation covers the scopes, attribution rules and anchored representation defined in Sections 3.2.2, 3.2 and 3.2.6 of Chapter 3.

Table 5.7 and Figure 5.2 compare identical inputs with and without attribution, construct by construct. A baseline attribution loss is *recovered* if a must or may edge from a placed screen witnesses it after attribution. Dynamic sinks and unplaced origins do not count as recovery.

Attribution recovers 303/454 assigned constructs: a per-application median of 40.2 % (range 14.3 %–100.0 %) and a pooled share of 66.7 %. Most recoveries occur on mifos. Excluding it reduces the share to 38.6 %; 4 of six applications recover under half. Edge counts rise from 4729 to 13830. Most recovery uses mechanisms

Table 5.5: Outcome of every navigation construct, baseline.

Outcome	rw	alf	gf	k8s	bw	mif	Total
Converted, must	1	0	0	0	1	19	21
Converted, may	16	3	55	4	53	1025	1156
Attribution, shared component	21	10	63	51	37	224	406
Attribution, service	1	8	9	5	15	10	48
Lazy boundary	0	0	0	0	0	0	0
Constant path	0	0	0	0	0	0	0
Component unlinked	0	0	0	0	0	0	0
Constant target	0	0	0	0	0	0	0
Node missing, other	0	2	3	1	0	34	40
Other	0	0	6	0	2	0	8
Dynamic destination	0	4	50	7	4	31	96
Source the analysis cannot place	0	0	0	0	0	0	0
Query only (not a transition)	1	0	11	4	19	0	35
Excluded: names no destination	0	0	1	0	0	0	1
Excluded: navigates a named outlet	0	7	0	0	0	0	7
All constructs	40	34	198	72	131	1343	1818

Note. Columns: realworld, alfresco, ghostfolio, k8s dashboard, bitwarden, mifos. All rows are measured on the final snapshot with attribution disabled.

from earlier work. The *full* step adds the member-level service join, fallback join and guard/effect stand-ins, placing 36 sites (2.0 %) beyond *outlet*. The call graph alone adds 7 (0.4 %) beyond the per-class rule it replaced (Section 5.3.3).

Measured by RQ1's must conversion, the gain is far smaller. Of the recovered constructs, 2 earn must and 301 earn may, and 2 of the six applications gain any must conversion at all. The must constructs rise from 21 to 25. Two of the rise are those recoveries, both on realworld. The other two are k8s dashboard constructs that the baseline already converted at may and attribution raises to must. The recovery count does not include them.

Counted in edges, the must tier grows from 17 to 159. The edges added are the links of two application shells, drawn once for each screen the shell renders (Section 5.5).

Tracking construct identifiers across runs finds 0 conversion regressions and 0 must-to-may demotions. No baseline must conversion was lost. This measures recovery, not target consistency; Section 5.5.3 assesses the must tier and remaining edges.

Claim 3.2 requires anchored-edge expansion to equal the materialised edge set. A separate implementation expands each scope-anchored edge over its member screens and deduplicates the result. Both sets agree on every application; otherwise

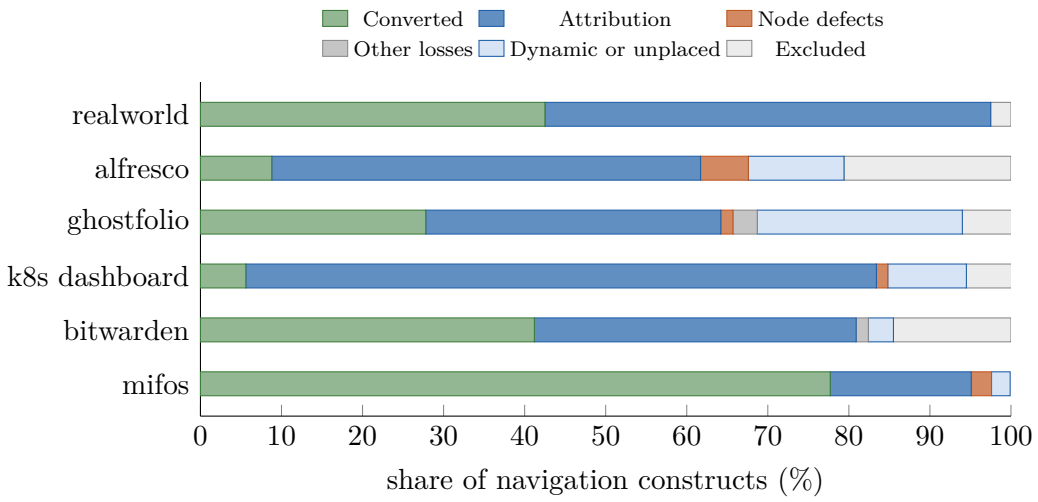


Figure 5.1: What happens to the navigation constructs of each application without the attribution pass.

Note. Attribution losses are the largest category in every application; the lazy-boundary, constant-path and component-unlinked node classes are empty on this instrument, and the remaining one holds few constructs, most of them in mifos. Exact counts per class are in Table 5.5. *Converted* counts constructs witnessed at must or may, as that table does; the must conversion of Figure 5.2 counts must only.

Table 5.8 is not produced. This is the sole measurement of storage savings: on mifos, materialisation yields 13.7 times as many edges as stored.

The check tests member traversal and deduplication, but both paths share the scope-membership definition. It supports rather than proves the claim on 6 applications and fixtures. The library's materialisation function is not used: on 5 applications it returns more entries than either path, reflecting iteration rather than distinct-edge differences.

Table 5.9 gives the recovery by the loss category each construct held in the final baseline and by the modality its own witness earns.

Table 5.10 classifies each construct that stays lost.

The 454 assigned constructs comprise 303 recovered and 151 remaining. All but 2 remaining constructs have a recorded reason. Those 2 have a scope and a destination node but no diagnosed cause. The reasons combine implementation gaps and limits of the source population.

- Some components have no recovered route or template path to a screen. This is an empty computed scope, not proof that the application never renders them.
- Some sites lie beyond the member-level call graph's supported receiver and provider forms.
- Other sites have a scope but a dynamic destination, or a target for which the graph has no node.

Table 5.6: Results of the witness check on the final attributed tier, and of the test-source cross-check.

Method	Result	n
Witness check, final attributed must tier	0 detected failures; 62 edges have a destination the checker cannot read. Unread edges are not passes, and target agreement does not establish Claim 3.1.	159
Test-source cross-check, alfresco	10 of 14 URLs the specifications name have a node; one negative test excluded.	14

Note. Mutation is reported separately in Section 5.4, on the attributed graph. This summary is written by hand from the harness output rather than generated.

Table 5.7: Before → after scope attribution, on the final evaluated snapshot.

	Attribution losses, recovered			Must conversion (RQ1)			Must, read and passed	
Application	Lost	must	may	Before	After	pp	Before	After
realworld	22	2	20	2.6%	7.7%	+5.1	1/1	9/9
alfresco	18	0	4	0.0%	0.0%	+0.0	–	–
ghostfolio	72	0	33	0.0%	0.0%	+0.0	–	–
k8s dashboard	56	0	8	0.0%	2.9%	+2.9	–	63/125
bitwarden	52	0	18	0.9%	0.9%	+0.0	1/1	1/1
mifos	234	0	218	1.4%	1.4%	+0.0	15/15	24/24
All	454	2	301					

Note. First, the constructs assigned to attribution in the baseline and how many recover onto must or may, the measure of RQ3. Then must conversion (RQ1) in per cent and its change in percentage points, and must-tier edges whose destination the witness check read and passed; a construct's tier is the modality its own witness earns. The remaining must edges are unread destinations, all 62 of them on k8s dashboard, which are neither passes nor failures; no edge failed. Constructs converted from other baseline categories are in Table 5.9.

5.3.1 Checking the scopes against the running interface

Recovery does not establish whether a site's computed scope matches its actual screens. The rendered-interface check (Section 4.2) tests one direction. Table 5.11 reports an earlier graph's observations: neither the browser experiment nor its predictions were recomputed for the final call graph. These findings concern that earlier graph.

Scopes too small. Of the 33 (screen, target pattern) pairs that the running applications render on the judged screens, 29 are predicted by the computed scope (87.9 %). Behind these predictions stand 67 raw anchors. On realworld, which renders most of its screens without a backend, every rendered pair on every judged screen is predicted. The pairs that are not predicted are all bitwarden's.

The unit is a (screen, destination) pair, not a site. Confirmation does not validate every underlying site's scope, but the 4 unpredicted pairs still demonstrate under-

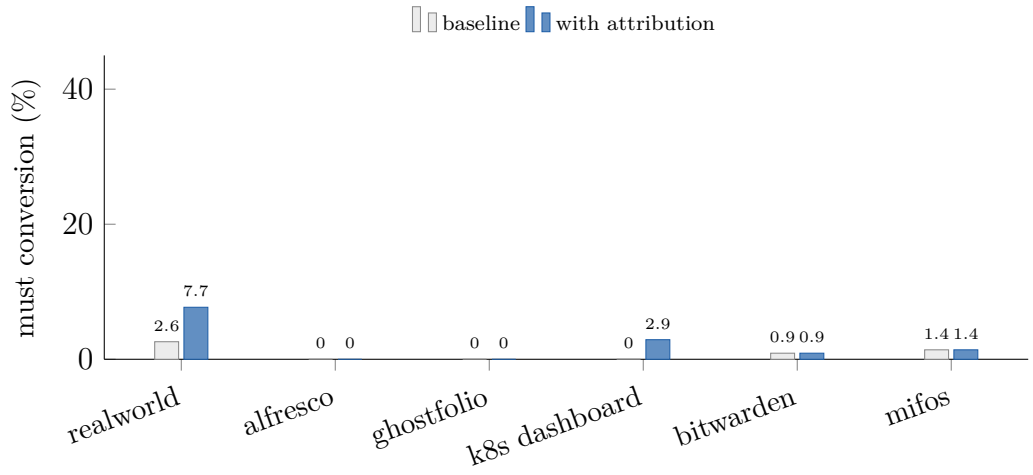


Figure 5.2: Must conversion, before → after scope attribution, on identical inputs.

Table 5.8: The anchored edge set against its expansion.

Application	Screens	Anchored	Expanded	Independent
realworld	10	59	88	88
alfresco	27	55	100	100
ghostfolio	84	395	1467	1467
k8s dashboard	67	313	698	698
bitwarden	98	576	764	764
mifos	606	12432	170011	170011

Note. Anchored is what the graph stores, *expanded* what the per-screen query yields, and *independent* a second materialisation written for this check. Claim 3.2 requires the last two to be equal; the check refuses to emit this table if they differ on any application.

attribution. The check covers 30/806 screens; 776 redirect to sign-in without a backend. It includes 5 of six applications, excluding ghostfolio because its per-locale bundles do not fit the harness's single served directory. These restrictions leave only one direction tested (Section 6.2).

The 4 unpredicted links share one cause, and all are in bitwarden: the graph holds the edges, with the right witness, but anchors them at an origin node rather than at a screen, because the component that renders them lives in a workspace library outside the application directory and attribution computed the empty scope for it. These sites are the *unplaced* class of Table 5.20.

Scopes too large. The 80 predicted but unrendered links combine three causes. Two are not defects: realworld's header hides links from unsigned users, and a tag page visited with a placeholder segment lists no articles. The third is target over-approximation, which concerns the destination rather than the scope. The combined count cannot measure over-attribution.

Only predictions that name a destination exactly can be refuted by a page not

Table 5.9: Where recovered constructs came from on the final snapshot.

Baseline class of the construct	must	may	total
Attribution, shared component	2	281	283
Attribution, service	0	20	20
Other, no class rule applied	0	2	2
Out of the attribution classes	2	301	303
Out of the node classes	0	0	0
Out of the remaining classes	0	2	2
All conversions gained	2	303	305

Note. Rows are their baseline loss categories; columns give the modality each construct's own witness earns after attribution. Constructs are followed by identity, not matched by source location.

Table 5.10: Why attribution losses remain after attribution.

Reason	rw	alf	gf	k8s	bw	mif	Total
Owner component not reached from any screen	0	8	9	5	17	1	40
In a service or guard, not placed	0	6	4	4	7	7	28
Scope found, target screen has no node	0	0	24	2	6	3	35
Scope found, target fully dynamic	0	0	2	37	2	5	46
Scope found, not explained	0	0	0	0	2	0	2

linking there. There are 7 of them across the judged screens, against 102 over-approximated. 0 of the 7 were not rendered. A denominator of 7 supports no rate. The over-attribution direction was therefore not refuted on this corpus, because the graph almost never claims an exact destination at a screen this check can observe.

5.3.2 What the attribution discriminates

Since the rendered-interface check cannot establish over-attribution, Table 5.12 compares computed scopes with assigning every represented site to every screen. This widest assignment places sites but neither resolves unknown targets nor adds missing destinations; it does not recover every construct. The ratio measures how much the analysis narrows source scopes.

Computed scopes are 1.7 to 18.7 times narrower than the baseline. The lower bound limits RQ3: on k8s dashboard, the average site is attributed to 39.5/67 screens (59 %), excluding only about two screens in five.

This has two implications.

1. The trivial baseline is not a competitor for the must tier. An unguarded site in a shared component attributed to every screen would yield a must edge from screens that never render it, so Claim 3.1 would fail by construction. That

Table 5.11: Recorded scope predictions of an earlier graph against the rendered interface, on the 5 applications the harness could serve.

Application	Screens	Judged	Pairs rendered	Predicted by the scope	Not predicted	Raw anchors
realworld	10	7	25	25	0	59
alfresco	26	3	0	0	0	0
k8s dashboard	67	1	0	0	0	0
bitwarden	98	18	8	4	4	8
mifos	605	1	0	0	0	0
All	806	30	33	29	4	67

Note. These predictions were not recomputed for the final call graph. *Judged* counts screens that rendered themselves rather than redirecting to a sign-in page; the rest cannot be judged from what rendered, because a redirected screen shows another screen's links. *Not predicted* is the only column that counts against the analysis; the other direction is reported in the text, because it can be judged only where the graph names a destination exactly.

Table 5.12: The computed attribution against scope $\top$ for every site.

Application	Screens	Sites	Attributed pairs	Mean scope	All-screens pairs
realworld	10	39	188	4.82	390
alfresco	26	19	117	6.16	494
ghostfolio	84	193	4827	25.01	16212
k8s dashboard	67	62	2450	39.52	4154
bitwarden	98	170	1790	10.53	16660
mifos	605	1319	42750	32.41	797995

Note. A site is a source construct a witness cites; an edge anchored at a scope is counted over the screens that scope stands for, since counting it at its anchor would understate the analysis's breadth. *Mean scope* is the attributed pairs per site.

none of the readable must edges attribution added fails the witness check is meaningful only where the attribution is narrow. Of the 142 must edges it added, 62 are unread, all on k8s dashboard, where the scopes are widest.

2. Some recovery relies on broad scopes. Attributing a site to three fifths of an application provides little discrimination. The evaluation neither validates nor refutes these scopes because the rendered-interface check cannot test over-attribution.

5.3.3 What each mechanism contributes

Table 5.13 extracts the corpus at 5 nested levels, each withholding every mechanism above it, including the stand-ins of Section 3.3; otherwise the lowest level would credit direct ownership with sites that only the stand-ins place.

Table 5.13: The corpus extracted at each attribution level.

		sites / (site, screen) pairs				
Application	Screens	owned	nesting	rooted	outlet	full
realworld	10	17 / 26	25 / 48	38 / 178	38 / 187	39 / 188
alfresco	26	11 / 33	14 / 37	14 / 37	14 / 37	19 / 117
ghostfolio	84	112 / 122	124 / 147	174 / 4347	186 / 4435	193 / 4827
k8s dashboard	67	11 / 12	59 / 275	59 / 275	59 / 2250	62 / 2450
bitwarden	98	124 / 134	152 / 394	157 / 884	159 / 1453	170 / 1790
mifos	605	1088 / 1445	1251 / 1773	1310 / 37468	1310 / 41223	1319 / 42750
All	890	1363 / 1772	1625 / 2674	1752 / 43189	1766 / 49585	1802 / 52122

Note. Sites counts the source locations a witness cites that the level places on at least one screen; one construct can have more than one, and a location can lie outside the application directory, so site counts are not comparable with the construct counts of Table 5.5. *Pairs* are the (site, screen) attributions it asserts. The levels are nested, and the table is produced only if each one's pairs are a subset of the next one's. *Screens* counts the screens at an address a link can name, so the catch-all screens of alfresco and mifos are left out of it and of the pairs, and the column falls short of the census's total by those screens.

Containment adds the most sites. The pre-existing shell rule adds the most (site, screen) pairs and reference recall. Outlet descent mostly widens scopes; counts alone cannot validate this width (the next subsection evaluates it). Full adds the member-level service call graph, fallback join and guard/effect stand-ins, which only full and its class variant include. Relative to the class rule it replaced, the call graph adds 7 sites (0.4 % of the corpus) and 331 pairs. The class rule remains the fallback for members without identified callers.

Comparison with earlier work uses two levels. The shell rule is pre-existing; Rima et al.'s specification also includes ancestor-route components, as *outlet* does. These levels reimplement earlier mechanisms inside this extractor; they do not run the earlier tools or show that those tools would place fewer sites. Against the closest level, *outlet*, full adds 36 sites (2.0 % of the corpus), all in services, guards or effects rather than components.

5.3.4 Agreement with a structural reference

Source inspection produced 39 sites and 188 (site, screen) pairs for realworld (Section 4.2). Realworld has only one service navigation and barely exercises the new step. The alfresco reference includes service navigation: 16 sites and 174 pairs. Table 5.14 scores every level against both references in both directions.

On realworld, the table requires nested pair sets and no full-level pair outside the reference; levels therefore differ only in recall. Most recovery comes from the pre-existing shell (*rooted*) and enclosing-route (*outlet*) rules; full adds the remaining 1 pair. On alfresco, full adds service, guard and effect sites beyond earlier levels, but is less precise than both those levels and the class join.

- *The reference is not independent of the graph.* The realworld reference was

Table 5.14: Each attribution level against the structural reference of each application.

	realworld			alfresco		
Level	Precision	Recall	Off shell	Precision	Recall	Off shell
owned	100.0 %	13.8 %	44.8 %	100.0 %	9.2 %	36.4 %
nesting	100.0 %	25.5 %	82.8 %	100.0 %	11.5 %	43.2 %
rooted	100.0 %	94.7 %	82.8 %	100.0 %	11.5 %	43.2 %
outlet	100.0 %	99.5 %	98.3 %	100.0 %	11.5 %	43.2 %
full	100.0 %	100.0 %	100.0 %	82.0 %	23.6 %	90.9 %
class	95.4 %	100.0 %	100.0 %	92.3 %	13.8 %	52.3 %

Note. Precision and *recall* are agreement with that reference, not with what the running application renders. *Off shell* is recall over the pairs outside the shell, where attribution decides. *class* is the class-level join that the call graph of *full* refines.

completed in its first version with nine header sites the scorer reported from the graph. Its precision is therefore weakest as evidence where the shell dominates.

- *The shell dominates.* On realworld, 13/39 sites are placed at $\top$, supplying 130/188 pairs that every level from *rooted* upward places trivially. On alfresco, 5 sites, 130/174 pairs, sit in a shell taken from a library outside its sources, which no level reads. The off-shell column is the one where attribution decides.

- *Two repairs were guided by the reference:* the call graph, written after the realworld reference exposed the class-level defect, and the rule that places an empty-path child on the screen above it (Section 6.3). The 100.0 % precision and the recall of *full* are scored on the same reference and test neither repair independently (Section 6.2).

- *The class join's excess on realworld* is 9 pairs, all at the `navigate` inside `UserService.logout`: the class join attributes the service to every screen whose components inject it, while its only caller is a button on the settings screen.

- *An AI assistant built the alfresco reference without the graph,* using realworld's rules under a protocol committed before scoring (`ground-truth/ALFRESCO-PROTOCOL.txt`). 17 undetermined sites are not scored. Afterwards, four lines reported by the scorer as placed but unlisted were inspected using only line numbers, without graph placements. None was a navigation site. They were added as excluded, a protocol deviation: only adding missed sites was permitted. This changes no figure because unlisted lines are not scored.

- *One configuration-driven site accounts for all full-level excess on alfresco.* The 9 excess pairs arise at 1 site, `navigateToSearch`. Its only caller is an effect triggered by a toolbar button in extension configuration that the analysis does not parse. It assigns the site to $\top$ at may, producing most pairs added over *outlet*. The class join's 2 excess pairs arise at the same site. Both levels also place an undetermined site at $\top$. It is unscored; treating it as firing nowhere would increase both excess counts.

Table 5.15: The graph against the pairs of screens the applications' own end-to-end suites navigate between.

	realworld	mifos
Navigations completed	141	62
Distinct pairs	19	22
Pairs in the graph (hits)	19	19
through the source screen or a scope that names it	16	17
only through the shell scope	3	2
Hits without attribution	13	8
Hits at the rooted level	19	16
Pairs redirected (redirect bucket)	0	5
of which hits	0	2
Pairs the expanded graph contains	71	162567
Possible ordered pairs of screens, self-pairs included	100	367236
Misses	0	3
redirect	0	3
base	0	0
dynamic	0	0
other	0	0
Misses that meet the dynamic test in any order	0	2

Note. A redirected navigation is compared on the address it asked for (the preregistered rule). *Rooted* is the level that stands for earlier work's mechanism. The miss classes after *redirect* were defined after the misses had been read, and are tested in the order redirect, base, dynamic, first match winning. All cells are generated figures.

- *The protocol's predictions.* The first (A1), that the full level recalls more than *outlet*: ✓. The second (A2), that its precision is ≥ the class join's: ✗.

What the two references establish is narrower than soundness. Both apply the analysis's own relations, and neither licenses the corpus-wide widths of Table 5.12 on an application of six hundred screens.

5.3.5 Agreement with the applications' own suites

The end-to-end reference comes from execution, without applying the analysis's relations (Section 4.2). Both applications pass the second and third validity checks. Mifos also passes the first: its uninstrumented run fails the same tests. Realworld has only instrumented run records, all without failures.

Table 5.15 gives the figures for both applications.

- *realworld.* The result does not bear on what the levels above *rooted* add, because the *rooted* level already contains every pair.
- *mifos, hits.* Table A.1 records the scorer's departure from the preregistered rule. As predicted, no matched pair has a must edge. One of 2 redirect-bucket hits depends solely on the notifications tray, a shell site whose target retains

two leading parameters and connects every screen to every account screen. This establishes graph containment, not site resolution. The same site widens the graph from 28185 to 162567/367236 possible pairs after the repairs below, reducing discrimination. Uniform pairs at that density would yield 9.7/22 hits (realworld: 13.5/19). This reference is not the suite's expected success rate; the observed hits demonstrate containment more than discrimination. No other preregistered prediction is reported here.

- *mifos, misses.* Only the redirect class was preregistered, and its misses are not classified further. The other classes were defined after the misses had been read, before the repairs below. Every remaining miss is in the redirect class (3/3): the graph holds the pair on the screen the redirect lands on. For 1 of them a better reader would also hold the address asked for (Section 6.3.1). 2 of them also meet the dynamic test, which fires on any unresolved site in the source component or above it, so it does not show that a missed navigation is that site. The preregistered five-class scheme and its prediction that misses would be mostly a scope too narrow or a site not read are not scored.

Before repair, the graph contained 12/22 pairs. Post-hoc classification identified unsupported source forms rather than runtime-only destinations: commands-array template literals were treated as variables, `relativeTo` through an options variable was not evaluated, and client-view relative links used child-tab addresses instead of the holding component's route (Section 3.2.6). Each gap was repaired with a test. Extractor-dependent figures were remeasured, except the historical groups in Section 4.3.3 (Table A.1). The repairs implement general Angular rules without naming sites, but were identified and evaluated by this check. Only the earlier 12 figure uses an extractor frozen before log inspection (Section 6.2); no other runnable suite remains for independent testing.

5.3.6 What scope attribution adds

Table 5.16 compares no attribution, earlier work's mechanism (*rooted*) and full analysis. Rooted reimplements the mechanism using this extractor's nodes (Section 5.3.3). Only Rima et al.'s subjects are out-of-sample (Section 5.3.7).

Three things follow, with the figures in Table 5.16.

- *Attribution puts more of the suites' transitions into the graph, in a wider graph.*

- *The gain is reached by the mechanisms of earlier work.* On realworld the rooted level holds every suite pair. On mifos it holds 16; the other 3 hits have a child tab as source and a site in the parent view, which the *outlet* level, the enclosing routes of Rima et al.'s specification (Section 5.3.3), places on the child screens. That level holds 19, as many as the full analysis. These pairs were missed before the repairs, so the difference is in-sample (Section 5.3.5). Oshima and Kishi name two of their restrictions, no `children` attribute and no route parameters; these alone admit none of the six applications, which declare both.

Table 5.16: The graph without attribution, with the mechanism of earlier work (*rooted*), and with the full analysis.

	no attribution	rooted	full
Suite transitions in the graph, realworld	13/19	19/19	19/19
Suite transitions in the graph, mifos	8/22	16/22	19/22
Graph pairs, realworld	24	69	71
Graph pairs, mifos	4275	158873	162567
Sites placed on at least one screen, corpus	–	1752	1802
Navigation inside services, constructs lost at baseline and recovered	–	–	20/48
Rima et al.'s reference pairs, their seven subjects	42/73	60/73	68/73
Sites placed in services, guards and effects	–	–	36

Note. Suite transitions are the distinct pairs of screens the application's own end-to-end suite navigates between (Section 5.3.5); *graph pairs* are those the graph holds. Only the full column of the suite rows was preregistered. The no-attribution graphs and realworld's rooted graph come from existing measurements; the mifos rooted graph was mapped for this comparison after the suites had been scored. The row for Rima et al.'s subjects is out-of-sample (Section 5.3.7). A dash marks a figure not measured. All cells are generated figures.

Rima et al. state no such restriction, and their specification covers this level (Section 2.4).

- *The service join generalises the bounded router-service tracing of Rima et al.'s tool, which follows a call into a service whose class name matches a router or navigation pattern to a bounded depth.* A navigation call inside an injected service belongs to no component, so the levels up to *outlet* cannot place it on any screen, on every application of the corpus (Table 5.5). The member-level call graph is more precise than a class-level join on realworld, but on alfresco the precision order reverses (Table 5.14). The levels above *rooted* find the reference pairs rooted misses, and on the two applications with suites the step above *outlet* adds no transition the suites take.

Shell links produce wide scopes; computed scopes are 1.7 to 18.7 times narrower than universal attribution (Section 5.3.2). Width alone does not establish over-attribution. Full produces no excess reference pair on realworld and 9 at one alfresco site, plus a universally attributed undetermined site. Both references apply the analysis's relations and cover only two applications. Over-attribution can pass witness checks, while suites test only omissions (Section 6.2).

Oshima and Kishi's restrictions admit 0 of the 6 corpus applications.

5.3.7 On the applications of Rima et al.

Rima et al. publish seven Angular subjects and ground truths comprising enumerated workflows with manually annotated start and terminal routes [64]. None was used in extractor development. Two preregistered comparisons use these subjects. The first is this thesis's only out-of-sample measurement: preregistration preceded the

Table 5.17: Rima et al.'s reference pairs held at each attribution level, and the graph's other pairs.

		Reference pairs held				Other graph pairs	
Subject	Pairs	none	rooted	outlet	full	all	real/read
posts-users	4	3	4	4	4	35	35/35
heroes	0	0	0	0	0	9	9/9
airbus	3	0	0	0	3	20	17/20
petclinic	35	25	32	32	32	152	35/40
traduora	11	9	9	9	9	104	37/40
jumpstart	11	3	11	11	11	29	29/29
event-booking	9	2	4	9	9	8	8/8
All, preregistered	73	57.5 %	82.2 %	89.0 %	93.2 %	357	170/181
Without 3 read as absent†	70	60.0 %	85.7 %	92.9 %	97.1 %		
Per start route†	292	21.2 %	64.0 %	87.3 %	88.4 %		
Repaired extractor†	73	58.9 %	83.6 %	90.4 %	94.5 %	341	

Note. none is the graph without attribution; *rooted* and *outlet* are the rules of Oshima and Kishi and of Rima et al.'s specification (Section 5.3.3). heroes has no pair under the preregistered rule: its entries are external links or list their terminal among their start routes. *Read*: pairs judged from the source, all of them where a subject has at most 40 other pairs and a seeded sample of 40 otherwise. The reference's wildcard route matches no screen, so its one pair counts as missed, while the graph's own wildcard screen is counted among its other pairs. †Post hoc, not preregistered: the first drops the reference pairs the source shows to be absent, and only missed pairs were read, so it can only rise; the second keeps the entries the preregistered rule drops, and its misses were not read; the third maps the subjects again after the repairs this comparison prompted. All cells are generated figures.

scorer and results, and the extractor was frozen before extraction. The second uses the extractor repaired after the first (Table A.1) and is therefore in-sample. Because their registry pins no commits, each subject uses its last commit before its ground-truth date. The AI assistant assessed both comparisons; a stronger model reassessed a gated subset specified below. The preregistered unit is a reference pair: when an entry's terminal differs from every start route, each start is paired with the terminal. Their tool contains every reference pair by construction because its output generated the reference. Their published measure is test-execution success, not graph recall. This comparison measures containment of their pipeline's pairs and examines additional graph pairs; it does not rank the tools.

Table 5.17 gives the result. The full level holds 68 of the 73 reference pairs and the *outlet* level 65. Two of the four preregistered predictions held: the full level holds at least 90 %, and the *outlet* level is within five points of it. Two failed: no miss was classed as a route the graph lacks or a navigation outside the router (the wildcard pair, whose route matches no screen, was classed as absent from the source), and only 63 of the 357 other pairs are reached only from the shell scope.

Of the 5 misses, the source holds 3 as no transition at all (dropdown toggles, a form with no router, a form that returns to the owner's page). The other 2 come from one relative link whose only segment is an expression, which the extractor leaves

to an unknown target although the route it reaches can be read; the second of them also starts on a route that only redirects. The step from *outlet* to *full* is 3 pairs, all on airbus, where the navigation runs inside an injected router service, which the specification's rule does not follow but their implementation traces to a bounded depth (Section 2.4). The preregistered rule drops every entry whose component is also shown on its target, which removes most links of a shell; the per-start-route row of Table 5.17 counts them.

Besides the counted pairs the graph holds 357 others; 190 of them are pairs of the entries the preregistered rule drops. Of the 181 read from the source, 170 are transitions the application has. 11 are not: 6 come from a navigation whose target is matched to longer routes than its command can produce, and 5 from a navigation placed on a sign-in or password-reset screen where the component holding it is not shown. Read per start route, Rima et al.'s reference lists all five of these pairs too. All 11 are at the may tier; 0 has a must edge.

All assessments use one AI model family and establish consistency, not correctness. A stronger model reassessed every miss, every pair not judged real and every sixth real pair: 45 pairs, with 3 changes. One batch was repeated because its first response omitted all 20 additional pairs. Adversarial review later found that the second response incorrectly classified a shell bar as rendered on sign-in and password-reset screens where a guard hides it (Section 6.3). 2 verdicts were corrected and 1 citations revised manually. A script verifies cited navigation sites, not their rendering on start screens. Four defects were repaired after recording these figures (Table A.1), and the corpus was remeasured. Re-extraction contains 69/73 reference pairs and 341 additional pairs. It gains the locale link and removes three invalid non-reference airbus pairs from sign-in, one traduora pair and petclinic's target over-approximations. These are post-hoc figures; none of their pairs was reassessed.

5.3.8 The tool of Rima et al. on the same applications

This comparison runs their tool at `f33003f`. Its static extraction and workflow enumeration stages ran on their seven subjects and the six corpus applications; backend-dependent test stages did not run. Preregistration preceded inspection of its corpus output. Workflow counts match their reference on every subject; start and terminal routes also match on the six with the same reference format. Only triggers resolved to navigation claim transitions. Other workflows record the start as terminal and are excluded. Each claimed transition is paired with every start route; this extractor uses scope-expanded edges. Relative recall concerns real pairs in the tools' union, not all real transitions. Proposing more pairs enlarges that reference union. Within each application, pairs are stratified as shared, only-theirs or only-ours. An only-ours stratum exceeding 1000 pairs is split into shell-only and other pairs. Every pair in each stratum, or a seeded sample of 12 when larger, was assessed from source without disclosing the proposing tool. Precision estimates the real share of a tool's pairs; relative recall estimates its share of real pairs in the union.

Their tool records constant-valued paths as expression text and omits lazy-module parent prefixes. None of its 88 ghostfolio paths names a declared route. To avoid

Table 5.18: Rima et al.'s tool and this extractor on the same thirteen applications.

	Rima et al.'s tool			This extractor		
Application	pairs	precision	recall	pairs	precision	recall
posts-users	39	100.0 %	100.0 %	39	100.0 %	100.0 %
heroes	6	100.0 %	100.0 %	6	100.0 %	100.0 %
airbus	23	87.0 %	100.0 %	20	100.0 %	100.0 %
petclinic	169	99.4 %	100.0 %	165	98.8 %	97.0 %
traduora	113	99.1 %	99.1 %	113	100.0 %	100.0 %
jumpstart	30	100.0 %	75.0 %	40	100.0 %	100.0 %
event-booking	20	73.3 %	93.6 %	17	92.2 %	100.0 %
realworld	66	95.5 %	100.0 %	63	100.0 %	100.0 %
alfresco	23	84.8 %	41.9 %	29	100.0 %	62.4 %
ghostfolio	10	30.0 %	0.4 %	1039	66.8 %	100.0 %
k8s dashboard	5	40.0 %	0.8 %	393	66.2 %	99.2 %
bitwarden	113	67.0 %	14.0 %	560	83.4 %	86.2 %
mifos	11095	80.0 %	77.4 %	161363	6.5 %	92.2 %
Their seven subjects	400	97.4 %	97.0 %	400	99.2 %	98.8 %
Corpus	11312	79.9 %	69.1 %	163447	7.4 %	92.4 %
Corpus without mifos	217	75.2 %	10.2 %	2084	72.6 %	94.1 %

Note. Recall is relative: the share of the real pairs either tool finds. Real pairs are estimated per stratum from a reading of at most 12 pairs, so a figure over a large stratum is an estimate. Routes are identified by declaration site, and the row without mifos was added after reading (both post hoc). All cells are generated figures.

losing pairs through naming differences, routes were matched by declaration site: first to the recorded path, then mounts ending in that path, otherwise all mounts. Moves between multiply mounted routes remain within one mount. This rule was chosen after inspecting their output.

Table 5.18 reports point estimates. On their seven subjects, the tools are close: theirs recalls 97.0 % at 97.4 % precision; this extractor recalls 98.8 % at 99.2 %. Excluding mifos, a post-hoc grouping, this extractor recalls 94.1 % of real union pairs versus 10.2 %, at 72.6 % versus 75.2 % precision. They propose 2084 and 217 pairs, with estimated real counts of 1512 and 163. Setting each stratum's rate to its Wilson interval endpoints preserves separate recall ranges: 88.4 %–97.1 % versus 5.6 %–17.9 %. On mifos, this extractor proposes 161363 pairs, theirs 11095, and both share 9308. Most of this extractor's pairs are shell-only (Section 5.3.5); precision is 6.5 % versus 80.0 %. Mifos dominates pooled corpus recall ranges, 79.6 %–99.4 % versus 10.3 %–85.6 %, which do not rank the tools. Intervals assume independent sampled pairs despite site clustering and cover sampling only. Systematic error from the single model family would shift both endpoints. Of the three preregistered predictions, the first (this extractor's recall on the corpus is the higher) holds at the point estimate, and its bounds separate only without mifos, a group formed after reading; the second (their precision on their subjects is at least this extractor's) and the third (their tool fails on some application) did not hold.

Table 5.19: Mutants sampled, excluded, scored and detected, with scope attribution on and off.

	attributed graph	without attribution
Candidate mutations found	272	–
Sampled under the quota	73	73
Excluded	6	22
guard: graph enters no screen the route yields	5	–
guard on a route with no component and no children	1	–
Scored	67	51
Detected	67	50
rename a route path	22/22	–
delete a `routerLink`	14/14	7/7
retarget a `navigate`	20/20	–
remove a guard	11/11	–
Paired mutants, detected	51/51	50/51

Note. An excluded mutant carries no information about sensitivity. All exclusions are guard removals: for the first kind no edge of the unmutated graph enters any screen the guarded route yields, and the second guards a route that declares neither a component nor children. *Paired* counts the mutants both graphs can score. A dash means the figure is not reported for that graph. Operator rows are detected over scored.

Assessment follows the first comparison's rules: 263 pairs, with 114 reassessed by a stronger model and 10 changes. I checked 41 sampled assessments underlying the five-application figures, three per stratum, against source. The model verdict was visible, so this was not blind and measures agreement rather than correctness. I judged 41 cases and agreed on 41 (Wilson 95 % interval 91.4 %–100.0 %). An initial pass incorrectly paired start routes of non-navigating workflows. It was withdrawn; valid remaining assessments were reused and 108 new sample pairs assessed. Mifos estimates reuse those samples after excluding cross-mount moves. Preregistration deviations are recorded with the results.

5.4 Change sensitivity (RQ4)

Mutation testing re-extracts each application per mutant, so it covers three of six applications: realworld and alfresco, used in earlier mutation runs, and guard-heavy bitwarden. Results use the attributed graph, harness mutate 1.9.1, a fixed seed and a quota of eight mutants per operator. Table A.1 records harness corrections.

Table 5.19 gives the sample, the exclusions and the detections on the attributed graph and without attribution; the detection rate per operator and application, and the outcomes summed over the applications, are in the accompanying repository.

Rename detection is nearly guaranteed: mutants require an existing node for the renamed segment, and full paths identify screens (Section 3.1). The other operators are similarly restricted. Deletion samples only `routerLink` lines cited by existing edges. Retargeting requires a cited `navigate` call beginning with a plain absolute

string. Guard removal is scored only if an edge carries that guard into the guarded screen; detection is conditional on that edge existing. Exclusions are few with attribution but more numerous without it. Samples are small (Section 6.2), favouring interpretation by operator pattern rather than rate. RQ4 checks witness-to-source mapping, not change-impact accuracy: only cited lines can fail, and the harness was corrected after inspecting misses (Table A.1). The headline uses scored mutants, 67/67; the paired row includes only mutants scored by both graphs.

Guard removal succeeds if an edge loses a guard from the removed list; all 6 exclusions fail the eligibility condition above. Without attribution, class matching by imported component file cannot distinguish realworld's empty-path and `:slug` editor children, which load the same file. No edge enters `/editor`, but one enters `/editor/:slug`. The empty-path guard is consequently scored and missed rather than excluded: the sole paired disagreement reflects the harness, not the graph.

The mutation run neither confirms nor contradicts the defect class *guards unreliable in both directions*; Chapter 6 gives the reading. In this sample the graph responded to every scored change (67/67) in the constructs it represents, a result the construction makes close to certain.

5.5 What the losses were (RQ5)

RQ5 traces the defect classes of Section 5.2 to causes in the final version, using the three outcomes defined in Section 4.2.

5.5.1 What the losses were

A *class* is a row of Table 5.4; a *cause* is its explanation from source review.

The three-part answer attributes repaired classes to tool defects, without extending that conclusion to all constructs. The 1631/1657 may-only witness locations involving application decisions (Table 5.1) remain may under rules that do not test whether the decision can vary. They therefore do not establish inherent application uncertainty. The lists count causes; Tables 5.10 and 5.20 count constructs, including 2/151 residual attribution constructs without a recorded reason.

Read by the final version. The information was in the source, so the loss was the tool's.

- *Lazy module boundary* and *constant-valued route paths*: causes in the extractor, read by the final version, and empty in Table 5.5.
- *Guards*: the two unread forms (a guard on a route a wrapper returns, a `canActivate` element written as a call) are read, and the guard rule is the router's own semantics (Section 3.1).
- The candidate class *node not linked to its component*: empty on the final version.

- Three forms that the end-to-end check exposed (Section 5.3.5, Table A.1).

Still not read (open). The final version does not read these causes.

- *Attribution*: 66.7 % of its constructs are recovered. The remainder is in Table 5.10, including sites that no relation the prototype implements reaches. It mixes gaps of the tool with components that no screen may render, and no measurement separates them.
- *Named router outlets*: a construct the model represents and the extractor does not implement. Its navigations are excluded from the denominator (7 sites) and its screens left out of the runtime route oracle's mounted set (64 route entries).
- ghostfolio's spread of computed routes is a construct the extractor could read and does not. 242 of the 251 unminted leaf screens sit behind it.

Beyond a static reader of a route table. No reader of a route table can resolve these.

- ghostfolio's custom `UrlMatcher` computes a route's path in a function. No reader of a route table can resolve it, and both tools report it as a blind spot.
- mifos chooses between two route tables by a value injected at deployment, which the runtime route oracle of Section 5.5.3 finds and the sources leave undecided.

Resolved causes form three groups. *Unreached route tables:* a `loadChildren` module keeps routes in a separate routing module. Stopping early places screens at the application root. Both static readers made this error; only the runtime route oracle exposed it. The final extractor follows up to three hops (Table A.1). *Referenced paths:* one application declares every path through a constant table; failing to resolve it collapses the graph to one node. *Referenced destinations:* targets may use fields, route-table chains, commands-array elements, builder returns or lookup values. Literal-initialised fields remain unresolved because they are mutable; resolving them as constants would turn runtime values into must edges.

5.5.2 Construct accounting

Table 5.20 separates four outcomes. A site is *recovered* when a must or may edge from a placed screen witnesses it with a destination. It is *dynamic* when the destination is unresolved: backend values, history entries, fields treated as mutable despite route-table initialisation, or commands-array template literals (Section 6.3). These yield unknown-modality edges to a screen's `u_` sink. A site is *unplaced* when its target is resolvable but its source screen is not; its edge uses an `o_` origin. Remaining

Table 5.20: Construct accounting after the loss list was worked.

Application	Sites	Excl.	In scope	Recovered	Dynamic	Unplaced
realworld	40	1	39	100.0 %	0.0 %	0.0 %
alfresco	34	7	27	25.9 %	25.9 %	44.4 %
ghostfolio	198	12	186	48.4 %	44.1 %	6.5 %
k8s dashboard	72	4	68	17.6 %	70.6 %	8.8 %
bitwarden	131	19	112	64.3 %	14.3 %	17.9 %
mifos	1343	0	1343	94.0 %	3.1 %	2.5 %
All	1818	43	1775	83.5 %	11.0 %	4.7 %

Note. Percentages are of the in-scope sites.

sites have a recorded reason but no transition. The check enforces reason presence, not correctness.

43 sites are excluded using construct text before consulting the graph: 35 pass empty commands arrays and change only query parameters; 1 names no destination. Neither connects screens. 7 navigate a secondary outlet, represented by the model as a second mount (Section 3.2.3) but unimplemented. This recall limitation raises alfresco's ratios when excluded. Both columns of Table 5.7 use the same 1775 sites; the comparison rejects any changed exclusion reason.

k8s dashboard combines low recovery with many runtime-dependent destinations. Its dynamic sites span 42 files, including 37 with one site each, indicating a distributed coding pattern rather than a single defect. Mifos contributes 75.7 % of in-scope sites and dominates pooled results. Without it, recovery is 50.9 % and accounted share 97.9 %. Section 4.3.2 records route-denominator changes.

The named-outlet restriction reaches a named outlet's *subtree* too: alfresco's viewer routes were minted at the primary address, which Angular reaches only as `(viewer:...)`. Both tools read the subtree the same way, so only the runtime route oracle of Section 5.5.3 exposed it.

5.5.3 The final tier and the node set

The final tier. The attributed tier has 0 witness-check failures among 159 must edges and 62 unread destinations, neither passes nor failures. All unread destinations are toolbar edges in one application, relying on chrome component scope and assumed route order. The check compares construct and edge destinations; it does not test unconditional execution or router admission, which the demotion rules address. Neither source inspection (Section 6.2) nor this mechanical check independently proves Claim 3.1.

The whole graph. The 13830 edges carry 15638 witness citations, including merged witnesses and repeated locations. Sites producing more edges therefore receive more weight. Only 7.0 % are clean; 87.2 % are over-approximations without

a target-agreement claim. One `navigateByUrl` call to an empty-prefix template literal, `view-transaction.component.ts:135` in mifos, produces 39.4 % of edges (5454). Mifos contributes 89.9 % of all edges and dominates pooled figures. Among 1437 judged citations, 76.2 % are clean; no remaining citation belongs to a must edge. Must-tier consistency does not describe the whole graph.

The node set. The compared graph declares 0 screens at unmounted addresses. Every claimed address appears in the framework configuration. This is external evidence for the six applications: static readers can agree on an incorrect prefix. Node validity is a precondition of the must-tier property, not part of it. The address-based census excludes named outlets and `UrlMatcher` entries (65 route entries) from both columns.

The runtime route oracle. The oracle loads each application, built in its development configuration, in a headless browser, reads the live router's route configuration after expanding every lazily loaded subtree, and counts an address as mounted when the configuration places a component there on the primary outlet. No route file is parsed. Of 1146 mounted addresses, the graph represents 892/1146. The 251 missing addresses are leaf screens; 3 more are prefixes, not missing screens: 2 redirect into their subtrees and 1 displays only a layout. The applications were probed once, on 29–30 October, against an earlier graph already corrected using oracle findings. Against the final graph, the archived configuration gives identical counts and address sets on 6/6 applications: 892 nodes, 0 unmounted addresses and 251 missing leaf screens. Navigation checks agree with the oracle on 41/41 addresses; the 251 missing addresses were not checked individually.

5.6 Dead links, found from the source

A link to an unmounted target is a dead link. Static candidates are checked against the recorded runtime route configuration before reporting. None was navigated to in a running application; the list therefore establishes neither runtime occurrence nor confirmed defects.

The clearest case is mifos, where a transactions tab links to `/journal-entry/view/:id` although the application declares that screen at `/accounting/journal-entry/view/:id`. The link omits the parent prefix, and mifos's catch-all route renders the not-found screen, which a user can mistake for missing data rather than a broken link.

The list is not one defect class, and the split matters for what each part is worth.

- 18 of the candidates are an absolute link written against a path the application does not mount, usually the name the server uses rather than the one the router does. Those are the application's.
- 5 come from a *relative* `routerLink`, such as `[routerLink]="'edit'"` or `[routerLink]="['../../transactions']"`. Their target depends on the

route the link resolves against, that of the component holding it (Section 3.2.6), so a wrong base is the analysis's question before it is the application's defect.

- 6 cannot be classified because the result files preserve insufficient construct text. They are counted separately.

The last two groups provide weaker evidence. Counting source locations rather than targets gives 18, 3 and 6 respectively. One shared-template relative link resolves against three owning screens, explaining the difference.

A further 69 sites in ../../libs, ../aca-shared, ../browser, across 2 repositories, fall outside the measured application directories and are excluded from the census. Most belong to sibling frontends whose routers declare addresses that would otherwise be labelled dead.

Chapter 6

Discussion

The results in Chapter 5 locate extraction losses at nodes and edges; each level limits the other.

6.1 Key findings

Two places where extraction fails. Node completeness has two definitions. Against static readers of the route configuration, extraction is complete on all six applications (Table 5.3); the loss classification uses this weaker definition. The runtime route oracle finds no node at an unmounted address, an absence of observed errors rather than a proof. It finds 251 mounted leaf screens without nodes, including 242 in ghostfolio (Section 5.5.3). Only 2 applications therefore have a node for every mounted leaf screen. The final version resolves the constant-path and lazy-boundary defects traced in Section 5.2 (Section 5.5); those loss categories are empty.

What remains is lost at the edge. Most remaining losses concern sites and screens already represented in the source and graph: the extractor cannot determine which screens contain a site. Exceptions are labelled as target screens without nodes, a category that also includes targets stored in fields; most occur in ghostfolio (Table 5.10). Scope attribution addresses the largest loss category in every application (Table 5.5) under the chosen rule order, which is part of the measure (Section 6.2). This ranking describes the implementation, not a limit of the approach. The two levels depend on each other: attribution needs destination nodes, while complete nodes yield few transitions if sites outside routed components remain unattributed. Chapter 5 therefore reports node coverage beside must conversion.

RQ3 and what the loss rules can delimit. A lost construct enters the first applicable category (Table 4.3). A shared-component construct rendered only on screens without nodes is thus an attribution loss, although attribution alone cannot recover it. The categories diagnose losses; they do not bound attribution recovery.

The demotion rules classify whether the toolbar edges attribution adds on k8s dashboard (Section 5.5.3) occur on every input; 125 remain at must. These rules were developed partly in response to red-team panels run with the AI assistant. The witness

check finds 0 failures among 159 must edges, but cannot resolve 62 destinations, all on k8s dashboard's toolbar edges. Only about half of that application's must edges are therefore checked.

RQ4 is a sanity check, and it is conditional on RQ1. The test checks whether witness mapping reacts to injected edits, not change-impact accuracy. Detection of renames (22/22) is almost guaranteed by construction. The other three operators have a denominator of 45, restricted to represented constructs in the simplest supported form. The attributed graph detects 45 of 45 scored mutants (Section 5.4): every deleted `routerLink`, 20/20 retargeted calls and 11/11 removed guards. The harness was corrected after its misses were examined (Section 4.3.2), so this result is not independent and need not generalise to a larger or differently drawn sample.

Guard mutants are observable only if an edge enters a screen produced by the guarded route (Section 5.4). The class "guards unreliable in both directions" in Chapter 5 instead rests on two forms examined in Section 5.5: a guard on a route returned by a wrapper function, and a `canActivate` element written as a call. Red-team source review found these defects, which the witness check missed; the mutation run neither confirms nor refutes the class.

Every scored change to a represented construct was visible. RQ1 measures how often constructs are represented. Change impact queries are likewise limited to the represented part of an application.

Two uses of the graph. The first is change impact analysis, supported only by the witness-mapping sanity check. Each edge records its construct's file and line, linking changed lines to edges and their source and destination screens. This is the screen-level counterpart of Chianti [61], in the impact analysis tradition of Arnold and Bohner [4, 12, 43]. RQ4 tests this necessary mapping property on represented constructs under a design that almost guarantees detection. It measures neither missed affected screens nor excess predictions.

Neither use was tested with consumers. The may tier generates candidates; only the must tier (25 constructs) is intended to describe transitions guaranteed under Claim 3.1's assumptions.

The second, unmeasured use is continuous integration without application execution. The graph can be recomputed deterministically on each commit without installing dependencies, serving or crawling the application (Section 4.3). Properties such as "a screen still has an inbound deterministic transition" can then be asserted like a type check. Guard assertions remain unsupported, and a screen without incoming edges supplies no evidence. A dynamically built model needs a running application, test data and an exploration budget [50, 51, 36].

Where both uses stop. Both uses face dynamic targets, route tables assembled from startup configuration, and overlays without routes. The graph omits these transitions or approximates them at may, which has no soundness claim. The must tier is a lower bound only under Claim 3.1's assumptions about route order,

ambiguous selector joins, unresolved calls and execution; each admits counterexamples. Section 3.1 counts edges relying on the first two. The graph alone therefore cannot answer queries that require every transition. Authentication also limits must: guards depend on unresolved state, so Claim 3.1 requires demotion. Guards, branches, callbacks and template conditions explain most demotions (Table 5.1). The small must tier largely follows from these rules; proving a guard always admits could enlarge it. Ignoring all guards while retaining other demotions raises the tier from 159 to 203 edges in a separate corpus extraction (`route-oracle/guard-only.mjs`).

6.2 Limitations and threats to validity

The tool's author evaluated one tool on one corpus using five methods and end-to-end checks on two applications. These choices limit the conclusions. Table 6.1 records each threat, its effect, mitigation and remaining limits.

Table 6.1: Threats to validity: effect on the results, mitigation applied, and what remains open.

Threat	Effect on the results	What was done	What remains open
Internal validity			
Tool developed with the evaluation, on the same corpus	The extractor was corrected during evaluation (Section 4.3.2). All figures on the six applications are in-sample: the extractor, independent parser, mutation harness and structural reference were corrected against their evaluation inputs. The reference guided two extractor repairs; the end-to-end check guided three.	The end-to-end check came closest to an out-of-sample corpus test: before its logs were examined, the frozen extractor contained 12/22 mifos pairs. Subsequent repairs made the final result in-sample. The comparison on Rima et al.'s seven subjects is out-of-sample: the extractor was frozen before mapping them against their tool-generated reference (Section 5.3.7). All corrections are reported.	The end-to-end preregistration and logs were committed together, so the archive cannot establish their order. For Rima et al.'s subjects, commits place preregistration before the scorer and results. Corpus figures do not estimate performance on other applications.
Dependencies not installed	Without installed dependencies, TypeScript cannot resolve framework or third-party types, weakening steps such as narrowing an injected receiver.	The configuration is reported.	Results apply only to this configuration.

Threat	Effect on the results	What was done	What remains open
Mutation experiment	Several operator–application samples have single-digit sizes. Graph-based exclusions couple the sample to the tool's recovery.	Sampling uses a seeded quota. Since operator choice affects mutation results [57], detection patterns are more informative than rates. Exclusions are reported.	The sample is not independent of the artefact under test.
Test-source cross-check	Measures node existence on one application.	Extends the syntactic proxy below with URLs the application's authors name in end-to-end specifications.	These URLs are expected addresses, not observed visits: the specifications are parsed, not run.
Scope attribution			
Scope compared in both directions on two applications only, against the analysis's own relations	Other methods in Chapter 4 check nodes, edges or witnesses, not whether a site's computed screen set equals the set from which it fires.	Three partial methods, below.	Attribution to too *many* screens can still pass every witness check: the construct exists and names the target. The rendered-interface check cannot establish over-attribution, leaving this direction unrefuted on the other four applications.
Structural references (Section 5.3.4)	Measures agreement with the analysis's relations in both directions on two of six applications. On alfresco, full recalls 90.9 % off shell (23.6 % overall) at 82.0 % precision over 16 determined sites (17 undetermined). Precision is below the class join's 92.3 %, a trade for recall. On realworld, recall is 100.0 % at 100.0 % precision. The replaced per-class rule, *class*, adds 9 excess pairs at one site.	The call graph was written after that finding.	Scoring on the same reference shows consistency with the repair, not independent validation.
End-to-end check (Section 5.3.5)	Checks containment of navigations observed in two applications' own suites. On realworld, the earlier-work shell rule already contains all 19 observed pairs (19). On mifos, it contains 16/22; the enclosing-route level contains 19, matching full (Section 5.3.6), in a graph containing 162567/367236 possible pairs.	Hits and misses are reported in Section 5.3.5.	Two applications only; the mifos figure after the repairs is in-sample.

Threat	Effect on the results	What was done	What remains open
Rendered-interface check (Section 5.3.1)	Checks one direction on 5 of six applications and 30/806 screens.	Uses a graph predating the final call graph.	A predicted link absent from the DOM may require sign-in, so absence cannot establish over-attribution.
Construct validity			
Denominator of must conversion (Section 4.2)	Includes constructs in unmounted code but omits navigation outside the parser's syntactic forms: `routerLink` attribute or binding and a call to `navigate` or `navigateByUrl` on a receiver whose name contains *router*. Helper-wrapped navigation, `location.href` assignments, server-driven redirects and third-party navigation count as neither losses nor successes. The figure is neither an upper nor a lower bound.	Interpreted as recall against an enumerable syntactic proxy; only the test-source cross-check above extends it.	Proxy completeness is unmeasured.
Denominator of node coverage	Screens whose paths neither parser resolves are excluded. Parsing less can therefore raise reported coverage.	The runtime route oracle supplies a second reading (Section 5.5.3).	The stronger threat, in the opposite direction to the one above.
Witness check	Must conversion uses must witnesses; target consistency is limited to what the checker resolves by re-parsing the cited source.	Checks every must witness, including merged witnesses: compares the construct and target with the edge, tests the destination against the independent static route set, and reports 62 unresolved destinations separately.	Neither runtime mounting, checked by the runtime route oracle, nor navigation execution is established.
Precedence of the loss rules	Testing targets first would make that category larger than attribution on 2 applications (alfresco and k8s dashboard).	Chapter 5 compares category sizes only within an application under a fixed order.	Cross-order comparison is not supported.
External validity			

Threat	Effect on the results	What was done	What remains open
Corpus and selection	Six open-source Angular applications at single commits may differ from proprietary applications in size, routing and navigation mechanisms [34, 35]. Selection favours the tool: after screening began, `SAP/spartacus` was rejected for unsupported configuration-driven routing (Section 4.1). Results are likely optimistic for Angular generally. Since mifos dominates pooled percentages, the per-application median, 40.2 %, is primary.	Six criteria were fixed before results; rejections and reasons are reported.	Tool support was outside those criteria. Two retained applications fail the test-suite criterion (Section 4.1).
Uneven outside evidence	Parsed test URLs come only from alfresco. Runtime configurations cover all 6 applications; partial rendered-link observations cover 5 against an earlier graph. Executed suites cover realworld and mifos, with 22 distinct transitions against $\geq$909 route entries in mifos.	–	Scale findings rest mainly on runtime configurations and the extractor's own parsing.
Angular only	Results concern Angular; the graph model and lattice are framework-independent.	Section 6.4 discusses other routers.	Transfer is unmeasured.
Oracles and experimenter			
Oracle independence	The independent parser shares no extractor code, but may still be wrong. Both apply the same rules to forms added during development: frozen tables, constant-built paths and workspace aliases (Table A.1). Agreement on these forms is weaker evidence.	Disagreements were resolved against the source.	The author of both implementations resolved them; differential testing detects disagreement without establishing correctness.

Threat	Effect on the results	What was done	What remains open
Experimenter bias	I wrote the extractor, designed the evaluation, selected the corpus and defined the defect classes.	Commits and harness version are pinned. Tables are generated from results (Section 4.3), except the manually compiled defect-class and witness/test-source summaries. All protocol corrections, including those worsening results, are reported (Section 4.3.2).	The experimenter was unblinded, loss classification had no second annotator, and residual constructs had no independent manual audit. Defect classes were defined after initial observations.
AI-assistant reading of the final must tier	The declared artificial intelligence (AI) assistant reviewed the final must tier's 25 constructs with extractor decisions visible. It also helped write the extractor and independent parser.	–	No human second reader checked it; agreement establishes rule consistency, not independent correctness. Both Rima comparisons used one model family; second passes changed 3 of 45 and 10 of 114 verdicts. In the corpus comparison, tool identity was hidden from the model but the experimenter was unblinded. My 41 re-reads agreed in 41 of 41 cases (Wilson 95 % interval 91.4 %–100.0 %), with model verdicts visible and tool identity hidden.
Two lines of evidence per defect class	This control on fitting leaves one class as a candidate. Yet three of five accepted classes rely on one named method plus observations; the guard class relies only on source review (Table 5.4).	Requirement of two lines of evidence per class.	Those three combine one method with my judgements; guards have source review alone.
The runtime route oracle (Section 5.5.3)			
Oracle selection effect	Runtime observation can expose wrong addresses shared by both static parsers, but requires applications that build and boot. Selecting partly for buildability biases the corpus.	–	The selection is not random.

Threat	Effect on the results	What was done	What remains open
Configuration, not rendered page; workspace scope; development build	Configuration counts routes as mounted even if guards always refuse them or rendering throws. A workspace application may mount routes outside the measured library; two of alfresco's three differences reflect this scope mismatch. All builds use development mode because production removes the required debug interface.	–	The first two enlarge the framework's set beyond reachable screens, making containment easier. Production configuration may add or remove routes in ways absent from the table, with no fixed direction of effect.

These oracle threats enlarge the mounted set rather than marking mounted addresses unmounted. Containment tests whether every extracted address is mounted under Section 5.5.3's abstraction and the recorded development configuration. The graph contains 0 unmounted addresses: no failure was observed in that configuration, without proving failures impossible.

6.3 Declared approximations

Appendix B, Table B.1, identifies unsupported constructs and their consequences for graph queries. Failures on exactly supported constructs are defects counted in Chapter 5; approximated constructs carry no completeness claim. They remain in Table 4.2' construct denominator as dynamic, unplaced or attribution losses. Before the repairs in Table A.1, mifos's end-to-end check exposed a relative-link base error in a template shared with child tabs; no such miss remains (Section 5.3.5). The first two rows describe omissions discovered by the runtime oracle (Section 5.5.3), rather than by inspecting the analysis.

6.3.1 Why the may tier is incomplete

The may tier is intended to over-approximate (Section 3.1), but is not a proved upper bound: the first-route, one-segment, catch-all-order, dispatch and dialog rules can omit transitions. Missing application transitions are therefore defects; excess may edges are not. Table 6.2 classifies every remaining miss in the two suites (Section 5.3.5) and two structural references (Section 5.3.4) by where its destination is fixed. *Inside* the static boundary, frontend code, installed libraries or committed configuration fix it, so recovery requires a better parser. These are tool limitations, not limits of static extraction. *Outside*, targets or routes depend on backend data, fetched configuration or user input. *Held* pairs exist at the landed screen but are scored at the requested address under the preregistered redirect rule; improved parsing cannot make them hits, so they belong to neither side. All alfresco reference misses

Table 6.2: Why the graph still misses transitions: every remaining miss, grouped by gap.

Gap	Example	Boundary	Fix	Pairs
Shell from a library (alfresco)	`src/lib/components/sidenav/components/sidenav-header.component.html:20`	inside	read the shell route that @alfresco/adf-core/shell provides under node_modules and the extension JSON in the repository that fills it	129
Routed component nested in another screen's template (alfresco)	`src/lib/components/files/files.component.ts:143`	inside	attribute its sites to the screens of the component that nests it as well as to its own routes	4
Redirected navigation (mifos)	`src/app/login/login.component.ts:121`	held	none in the reader: the graph holds the pair on the screen the redirect lands on, and the rule scores the address asked for	2
Default commands of a service navigation (mifos)	`src/app/clients/clients-view/client-actions/client-action-notifier.service.ts:34`	inside	read a parameter's default value and the route each caller passes as relativeTo	1

Note. Covers the two suites and the two structural references. *Pairs* counts missing suite pairs and missing (site, screen) pairs of a reference.

lie inside (133 of 133). Of mifos's 3 suite misses, 1 lie inside and 2 are held; realworld has no miss in either measure. Recovering all inside misses would yield 100.0 % of alfresco's reference pairs (now 23.6 %) and 90.9 % of mifos's suite pairs (now 86.4 %). This ceiling is weak evidence on the references: both apply the analysis's relations (Section 5.3.4), placing every pair inside by construction, and realworld's reference was partly completed from graph output. Only the suites can reveal outside misses; neither does. The ceiling excludes unexercised causes, including the two cases beyond a static route-table reader in Section 5.5.1. It also excludes excess may edges: mifos's graph contains 162567 of 367236 possible pairs (Section 5.3.5). The author classified each miss; the joining script rejects missing and surplus classifications.

6.4 Transfer to other frameworks

All measurements concern Angular. The following account of React Router and Vue Router is qualitative, based on documented behaviour [60, 80]. The graph model, scope lattice, fixpoint, anchored edges and expansion query would transfer unchanged. So would outlets: all three routers let a parent route render a placeholder for its matched child. The other four input relations in Section 3.2 require framework-specific derivation. Hooks and composables simplify the call graph by giving navigation sites

owning components. React nesting is harder: recovering component relationships from JSX is a dataflow problem resembling points-to analysis. A React Router loader redirect plays the role of `canActivate`; Vue's global `beforeEach` most closely resembles `canActivateChild` on a root route anchored at $\top$. Redirect semantics differ. Vue redirects a record only when it is the last matched record, so a redirecting parent can render its component for a matched child, which Angular's validator forbids. Applying Angular's rule in the Vue extractor dropped those components. The Vue extractor retains them and records a residual limitation at the parent: visiting its own address redirects without mounting its component. Mixing Vue 2's catch-all `'*'` with Vue 3's `'/:pathMatch(.*)*'` maps both to one address. The extractor retains the first declaration, potentially the wrong one, and reports the collision. These expectations require evaluation on real applications (Section 6.5).

6.5 Future work

Priorities, ordered by their potential effect on the claims, are:

- *An oracle for screen sets on a large application.* The references in Section 5.3.4 cover the two smallest applications using the analysis's relations, but most attributions concern applications with hundreds of screens (Table 5.12). Observe rendered links after sign-in, or annotate a stratified sample of sites.
- *Unimplemented model features.* These include dispatch relations for NgRx effects and router events, the injector hierarchy (provider bindings are currently application-wide; Section 6.3), and guards returning a `UrlTree`. The broader guard model in Chapter 3 requires substantial work; guard-removal tests currently reach only recorded guards (Section 5.4).
- *A runtime tier.* Observation identifiers are already valid witnesses. Comparison with dynamic approaches [50, 51, 74] would measure transitions found only by a crawler or only statically. Observation confirms a may edge in one run without promoting it to must; an unobserved candidate remains may, preserving Claim 3.1. This tier is neither built nor evaluated here.
- *A broader corpus.* Automated selection from a larger repository pool, retaining poorly handled applications [34, 35], would reduce tool-driven bias (Section 6.2). A proprietary industrial case study, reporting aggregates, would address external validity.
- *Consumer evaluation.* Controlled experiments are needed to test benefits for mobile testing agents, coding agents (Chapter 1) and automated test generation; none was attempted here.

6.6 Summary

Framework transfer remains unmeasured. The priorities are a large-application screen-set oracle and a runtime tier extending the graph to signed-in use.

Chapter 7

Conclusion

This thesis measured static navigation extraction on six Angular applications, classified its losses and evaluated scope attribution. Table 7.1 summarises the answers to the five research questions.

Table 7.1: Answers to the research questions, supporting figures and main limitations.

RQ	Answer	Key figures	Main limit
RQ1: how much is recovered	Every declared screen identified by the static readers has a node; few constructs yield must transitions.	Nodes complete against the mounted routes on 2 of six applications. 25 constructs yield 159 must edges, 125 from one toolbar; 33 without an assumed route order.	Most must edges depend on the unvalidated route-order assumption (126 edges); the toolbar's scope is untested.
RQ2: where extraction fails	Five defect classes and one candidate class (Table 5.4); attribution is the largest loss category in every application under the chosen rule order.	Table 5.5; testing the target first makes the target category larger than attribution on alfresco and k8s dashboard.	The ranking depends on the order in which the loss rules are applied.
RQ3: what scope attribution adds	It recovers constructs without losing any recovered by the baseline, mostly through earlier mechanisms and almost all as possible transitions. On the alfresco reference, the added step trades precision for recall.	66.7 % of the attribution constructs (38.6 % without mifos), with screen sets 1.7 to 18.7 times narrower than every represented site on every screen. The step adds 36 sites (2.0 %) over *outlet*. Alfresco reference: recall 23.6 % against 13.8 % for a per-class join, precision 82.0 % against 92.3 %.	Both structural references apply the analysis's own relations. The added step recovers no extra end-to-end transition over *outlet* on either application with a suite (mifos: 19/22 with and without it, against 9.7 for uniformly drawn pairs at the graph's density). The comparison with Rima et al.'s tool gives no overall ranking (Section 5.3.8).

RQ	Answer	Key figures	Main limit
RQ4: whether the graph responds to change	A conditional sanity check of witness mapping on three applications; change-impact accuracy remains unmeasured.	67/67 scored mutants detected.	Operators act only at constructs already represented, making detection close to guaranteed; the harness was corrected after inspecting its misses.
RQ5: whose losses they are	Three groups emerge: repaired tool defects; unresolved cases (remaining attribution losses, named outlets and one application's computed routes); and cases beyond static route-table analysis (a custom `UrlMatcher` and deployment-selected routes). Application decisions trigger may classification under the present demotion rules; this does not establish an inherent limit of static analysis.	1631/1657 may-only witness locations sit behind an application decision, 26 behind the extractor's uncertainty alone; 2 of the 151 remaining attribution constructs are labelled with no reason found.	These causes describe the classes found and repaired. The harness enforces the presence of a label, not its correctness.

Contributions. The primary contribution is the measurement: six pinned applications, five evaluation methods with different blind spots, four further scope checks and a classification of every lost construct. The extractor supports nested and parameterised route trees, which the stated restrictions of an earlier Angular extraction exclude. Those restrictions admit 0 of the 6 corpus applications; this extractor processes all of them (Section 5.3.6).

The evaluation apparatus makes the measurements inspectable. Tables are generated from harness results, and source-based methods re-run from pinned commits. Runtime reproduction is partial: the route oracle re-runs only in part, the realworld end-to-end check uses an unpinned public backend, and superseded figures cannot be regenerated.

Scope attribution is specified as a dataflow analysis over screen sets and implemented over three of its five input relations (Section 3.2). Most recovery comes from earlier mechanisms included as analysis levels. The added step places navigation in services, guards and effects. Its type-directed, transitive service join generalises the bounded, one-level, name-matched router-service tracing in Rima et al.'s implementation. With a fallback join and stand-ins for guards and effects, it adds 36 sites (2.0 %) over outlet descent. The call graph alone adds 7 (0.4 %) over the per-class rule it replaced (Section 5.3.3).

What remains open. Dispatch is specified but unimplemented, guards use a stand-in, and the call graph ignores the injector hierarchy. Screen sets were checked in both directions on only two applications, against references that apply the analysis's own relations.

The main corpus is in-sample. On Rima et al.'s seven applications, excluded from initial development, the extractor contains 69 of the 73 reference pairs after four repairs (68 before them). These results are therefore partly fitted too (Section 5.3.7). The comparison with their tool uses small AI-read samples; I confirmed 41 of the 41 cases I re-read (Section 5.3.8). The mifos end-to-end check exposed unsupported target forms and a relative link resolved against a child tab's address instead of its containing route. Repairs after that check make the final mifos result in-sample (Section 5.3.5).

The must-tier judgments were made by the AI assistant; two readings by one model measure consistency rather than correctness. Rendered-interface checks covered only screens reachable without signing in. Signed-in execution is needed to test the rest (Section 6.5). Consumer benefit, including support for UI agents and test generation, remains unmeasured.

A route table alone is insufficient for static navigation extraction. May edges are candidates that require further checks. Must transitions (25 constructs) hold only under the assumptions of Claim 3.1.

Bibliography

[1] Manuel Alabor and Markus Stolze. Debugging of RxJS-based applications. In *Proceedings of the 7th ACM SIGPLAN International Workshop on Reactive and Event-Based Languages and Systems (REBLS)*, pages 15–24. ACM, 2020.

[2] Lars Ole Andersen. *Program Analysis and Specialization for the C Programming Language*. PhD thesis, DIKU, University of Copenhagen, 1994. DIKU report 94/19.

[3] Anastasios Antoniadis, Nikos Filippakis, Paddy Krishnan, Raghavendra Ramesh, Nicholas Allen, and Yannis Smaragdakis. Static analysis of Java enterprise applications: Frameworks and caches, the elephants in the room. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 794–807, 2020.

[4] Robert S. Arnold and Shawn A. Bohner. Impact analysis – towards a framework for comparison. In *Proceedings of the Conference on Software Maintenance (CSM)*, pages 292–301. IEEE Computer Society Press, 1993.

[5] David F. Bacon and Peter F. Sweeney. Fast static analysis of C++ virtual function calls. In *Proceedings of the 11th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 324–341, 1996.

[6] Engineer Bainomugisha, Andoni Lombide Carreton, Tom Van Cutsem, Stijn Mostinckx, and Wolfgang De Meuter. A survey on reactive programming. *ACM Computing Surveys*, 45(4):52:1–52:34, 2013.

[7] Herman Banken, Erik Meijer, and Georgios Gousios. Debugging data flows in reactive programs. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pages 752–763. ACM, 2018.

[8] Ira D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier. Clone detection using abstract syntax trees. In *Proceedings of the International Conference on Software Maintenance (ICSM)*, pages 368–377. IEEE Computer Society, 1998.

[9] Carlo Bellettini, Alessandro Marchetto, and Andrea Trentini. WebUml: Reverse engineering of web applications. In *Proceedings of the 2004 ACM Symposium on Applied Computing (SAC)*, pages 1662–1669, 2004.

[10] Gavin Bierman, Martín Abadi, and Mads Torgersen. Understanding TypeScript. In Richard Jones, editor, *ECOOP 2014 – Object-Oriented Programming*, volume 8586 of *Lecture Notes in Computer Science*, pages 257–281, Berlin, Heidelberg, 2014. Springer.

[11] Justus Bogner and Manuel Merkel. To type or not to type? A systematic comparison of the software quality of JavaScript and TypeScript applications on GitHub. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 658–669. ACM, 2022.

[12] Shawn A. Bohner and Robert S. Arnold, editors. *Software Change Impact Analysis*. IEEE Computer Society Press, Los Alamitos, CA, USA, 1996.

[13] Miao Chen, Tengfei Tu, Hua Zhang, Qiaoyan Wen, and Weihang Wang. Jasmine: A static analysis framework for Spring core technologies. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1–13. ACM, 2022.

[14] Sen Chen, Lingling Fan, Chunyang Chen, Ting Su, Wenhe Li, Yang Liu, and Lihua Xu. StoryDroid: Automated generation of storyboard for Android apps. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 596–607, 2019.

[15] Gregory H. Cooper and Shriram Krishnamurthi. Embedding dynamic dataflow in a call-by-value language. In Peter Sestoft, editor, *Programming Languages and Systems, 15th European Symposium on Programming (ESOP 2006)*, volume 3924 of *Lecture Notes in Computer Science*, pages 294–308, Berlin, Heidelberg, 2006. Springer.

[16] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

[17] Jeffrey Dean, David Grove, and Craig Chambers. Optimization of object-oriented programs using static class hierarchy analysis. In Mario Tokoro and Remo Pareschi, editors, *ECOOP'95 – Object-Oriented Programming*, volume 952 of *Lecture Notes in Computer Science*, pages 77–101. Springer, 1995.

[18] Guillaume Demarty, Fabien Maronnaud, Gabriel Le Breton, and Sylvain Hallé. SiteHopper: Abstracting navigation state machines for the efficient verification of web applications. In *Web Services and Formal Methods (WS-FM 2012)*, volume 7843 of *Lecture Notes in Computer Science*, pages 103–117. Springer, 2013.

[19] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[20] Joscha Drechsler, Guido Salvaneschi, Ragnar Mogk, and Mira Mezini. Distributed REScala: An update algorithm for distributed reactive programming. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 361–376. ACM, 2014.

[21] Conal Elliott and Paul Hudak. Functional reactive animation. In *Proceedings of the 2nd ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 263–273, 1997.

[22] Asger Feldthaus and Anders Møller. Checking correctness of TypeScript interfaces for JavaScript libraries. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 1–16. ACM, 2014.

[23] Asger Feldthaus, Max Schäfer, Manu Sridharan, Julian Dolby, and Frank Tip. Efficient construction of approximate call graphs for JavaScript IDE services. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 752–761, 2013.

[24] Martin Fowler. Inversion of control containers and the dependency injection pattern. `https://martinfowler.com/articles/injection.html`, January 2004.

[25] Zheng Gao, Christian Bird, and Earl T. Barr. To type or not to type: Quantifying detectable bugs in JavaScript. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 758–769. IEEE, 2017.

[26] Google. Angular documentation: Effects. `https://angular.dev/guide/signals/effect`, 2026. Accessed November 2026.

[27] Google. Angular documentation: Routing. `https://angular.dev/guide/routing`, 2026. Accessed September 2026.

[28] David Grove and Craig Chambers. A framework for call graph construction algorithms. *ACM Transactions on Programming Languages and Systems*, 23(6):685–746, 2001.

[29] Simon Holm Jensen, Anders Møller, and Peter Thiemann. Type analysis for JavaScript. In Jens Palsberg and Zhendong Su, editors, *Static Analysis, 16th International Symposium, SAS 2009*, volume 5673 of *Lecture Notes in Computer Science*, pages 238–255, Berlin, Heidelberg, 2009. Springer.

[30] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[31] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[32] Ralph E. Johnson and Brian Foote. Designing reusable classes. *Journal of Object-Oriented Programming*, 1(2):22–35, June/July 1988.

[33] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 654–665. ACM, 2014.

[34] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, pages 92–101, 2014.

[35] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071, 2016.

[36] Iva Kertusha, Gebremariam Assres, Onur Duman, and Andrea Arcuri. A survey on web testing: On the rise of AI and applications in industry. *Science of Computer Programming*, 253:103473, 2026.

[37] Gary A. Kildall. A unified approach to global program optimization. In *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 194–206, 1973.

[38] Łukasz Kurant. Static components dependency graph detection with evaluation metrics in React.js projects. In *Communication Papers of the 20th Conference on Computer Science and Intelligence Systems (FedCSIS)*, volume 45 of *Annals of Computer Science and Information Systems*, pages 127–134, 2025.

[39] Duling Lai and Julia Rubin. Goal-driven exploration for Android applications. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 115–127, 2019.

[40] Rodrigo Laigner, Diogo Mendonça, Alessandro Garcia, and Marcos Kalinowski. Cataloging dependency injection anti-patterns in software systems. *Journal of Systems and Software*, 184:111125, 2022.

[41] Davy Landman, Alexander Serebrenik, and Jurgen J. Vinju. Challenges for static analysis of Java reflection – literature review and empirical study. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 507–518. IEEE, 2017.

[42] Kim G. Larsen and Bent Thomsen. A modal process logic. In *Proceedings of the Third Annual Symposium on Logic in Computer Science (LICS)*, pages 203–210, 1988.

[43] Steffen Lehnert. A taxonomy for software change impact analysis. In *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution (IWPSE-EVOL)*, pages 41–50. ACM, 2011.

[44] Jiawei Li, Jiahao Liu, Jian Mao, Jun Zeng, and Zhenkai Liang. UI-CTX: Understanding UI behaviors with code contexts for mobile applications. In *Network and Distributed System Security Symposium (NDSS)*, 2025.

[45] Changlin Liu, Hanlin Wang, Tianming Liu, Diandian Gu, Yun Ma, Haoyu Wang, and Xusheng Xiao. ProMal: Precise window transition graphs for Android via synergy of program analysis and machine learning. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pages 1755–1767, 2022.

[46] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 1–13. ACM, 2024.

[47] Benjamin Livshits, Manu Sridharan, Yannis Smaragdakis, Ondřej Lhoták, J. Nelson Amaral, Bor-Yuh Evan Chang, Samuel Z. Guyer, Uday P. Khedker, Anders Møller, and Dimitrios Vardoulakis. In defense of soundiness: A manifesto. *Communications of the ACM*, 58(2), 2015.

[48] Magnus Madsen, Benjamin Livshits, and Michael Fanning. Practical static analysis of JavaScript applications in the presence of frameworks and libraries. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 499–509. ACM, 2013.

[49] Magnus Madsen, Frank Tip, and Ondřej Lhoták. Static analysis of event-driven Node.js JavaScript applications. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 505–519. ACM, 2015.

[50] Ali Mesbah, Arie van Deursen, and Stefan Lenselink. Crawling Ajax-based web applications through dynamic analysis of user interface state changes. *ACM Transactions on the Web*, 6(1):1–30, 2012.

[51] Amin Milani Fard and Ali Mesbah. Feedback-directed exploration of web applications to derive test models. In *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, pages 278–287, 2013.

[52] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. Efficient JavaScript mutation testing. In *Proceedings of the 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation (ICST)*, pages 74–83. IEEE, 2013.

[53] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. Guided mutation testing for JavaScript web applications. *IEEE Transactions on Software Engineering*, 41(5):429–444, 2015.

[54] Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. *Principles of Program Analysis*. Springer, Berlin, Heidelberg, 1999.

[55] Frolin S. Ocariza, Karthik Pattabiraman, and Ali Mesbah. Detecting inconsistencies in JavaScript MVC applications. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 325–335. IEEE, 2015.

[56] Naito Oshima and Tomoji Kishi. Model checking method for SPA page transition based on component-based framework. In *Proceedings of the International Conference on Software Engineering and Knowledge Engineering (SEKE 2018)*. KSI Research, 2018.

[57] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. In *Advances in Computers*, volume 112, pages 275–378. Elsevier, 2019.

[58] Renaud Pawlak, Martin Monperrus, Nicolas Petitprez, Carlos Noguera, and Lionel Seinturier. SPOON: A library for implementing analyses and transformations of Java source code. *Software: Practice and Experience*, 46(9):1155–1179, 2016.

[59] Upsorn Praphamontripong and Jeff Offutt. Applying mutation testing to web applications. In *Proceedings of the 2010 Third International Conference on Software Testing, Verification, and Validation Workshops (ICSTW)*, pages 132–141. IEEE, 2010.

[60] Remix Run. React router documentation, version 8. `https://reactrouter.com/`, 2026. Version 8.4.0. Accessed 5 October 2026.

[61] Xiaoxia Ren, Fenil Shah, Frank Tip, Barbara G. Ryder, and Ophelia Chesley. Chianti: A tool for change impact analysis of Java programs. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 432–448. ACM, 2004.

[62] Filippo Ricca and Paolo Tonella. Analysis and testing of web applications. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 25–34, 2001.

[63] Gregor Richards, Francesco Zappa Nardelli, and Jan Vitek. Concrete types for TypeScript. In John Tang Boyland, editor, *29th European Conference on Object-Oriented Programming (ECOOP 2015)*, volume 37 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 76–100, Dagstuhl, Germany, 2015. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[64] Bachar Rima, Anas Shatnawi, and Abdelhak-Djamel Seriai. E2E test generation over a source-code-derived workflow space: An Angular case study. In *Software Engineering and Advanced Applications*, Lecture Notes in Computer Science, pages 334–351, Cham, 2026. Springer Nature Switzerland. Published online 27 August 2026; print volume dated 2027. The method is described in this thesis from the publisher abstract and the replication package; the full text was not available.

[65] Guido Salvaneschi and Mira Mezini. Debugging for reactive programming. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, pages 796–807. ACM, 2016.

[66] Guido Salvaneschi, Sebastian Proksch, Sven Amann, Sarah Nadi, and Mira Mezini. On the positive effect of reactive programming on software comprehension: An empirical study. *IEEE Transactions on Software Engineering*, 43(12):1125–1143, 2017.

[67] Manu Sridharan, Shay Artzi, Marco Pistoia, Salvatore Guarnieri, Omer Tripp, and Ryan Berg. F4F: Taint analysis of framework-based web applications. In *Proceedings of the 2011 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 1053–1068. ACM, 2011.

[68] Stryker Mutator contributors. Stryker mutator: Mutation testing for JavaScript, TypeScript, C# and Scala. `https://stryker-mutator.io/`, 2026. Accessed 2026-10-21.

[69] Oksana Tkachuk and Sreeranga P. Rajan. Method and system to extract a navigation model for analysis of a web application, 2013. US Patent 8,402,319 B2, Fujitsu Limited.

[70] Chenxu Wang, Tianming Liu, Yanjie Zhao, Minghui Yang, and Haoyu Wang. LLMDroid: Enhancing automated mobile app GUI testing coverage with large language model guidance. *Proceedings of the ACM on Software Engineering*, 2(FSE):1001–1022, 2025.

[71] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 50(4):911–936, 2024.

[72] Siyi Wang, Sinan Wang, Yujia Fan, Xiaolei Li, and Yepang Liu. Leveraging large vision-language model for better automatic web GUI testing. In *Proceedings of the 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 125–137. IEEE, 2024.

[73] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. AutoDroid: LLM-powered task automation in Android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*, pages 543–557. ACM, 2024.

[74] Rahulkrishna Yandrapally and Ali Mesbah. Fragment-based test generation for web apps. *IEEE Transactions on Software Engineering*, 49(3):1086–1101, 2023.

[75] Hong Yul Yang, Ewan Tempero, and Hayden Melton. An empirical study into use of dependency injection in Java. In *Proceedings of the 19th Australian Conference on Software Engineering (ASWEC)*, pages 239–247. IEEE, 2008.

[76] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 50528–50652, 2024.

[77] Shengqian Yang, Haowei Wu, Hailong Zhang, Yan Wang, Chandrasekar Swaminathan, Dacong Yan, and Atanas Rountev. Static window transition graphs for Android. *Automated Software Engineering*, 25(4):833–873, 2018.

[78] Shengqian Yang, Dacong Yan, Haowei Wu, Yan Wang, and Atanas Rountev. Static control-flow analysis of user-driven callbacks in Android applications. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, pages 89–99, 2015.

[79] Shengqian Yang, Hailong Zhang, Haowei Wu, Yan Wang, Dacong Yan, and Atanas Rountev. Static window transition graphs for Android. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 658–668, 2015.

[80] Evan You and Eduardo San Martin Morote. Vue router documentation, version 5. `https://router.vuejs.org/`, 2026. Version 5.3.1. Accessed 5 October 2026.

[81] Tao Zhang, Fuzhong Hao, Yunfan Wang, Bo Zhang, and Guangwei Xie. VeriFlow: A framework for the static verification of web application access control via policy-graph consistency. *Electronics*, 14(18):3742, 2025.

[82] Carlos Zimmerle, Kiev Gama, Fernando Castor, and José Murilo Mota Filho. Mining the usage of reactive programming APIs: A study on GitHub and Stack Overflow. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 203–214. ACM, 2022.

Appendix A

Corrections made to the protocol

Table A.1 lists corrections in chapter order; the repository records their commit hashes. *Figure* marks an implausible or unwelcome result, *panel* a red-team panel, *review* an assistant-supported source review, *harness* a mutation-harness change, and *end-to-end check* an extractor gap exposed by the suites in Section 5.3.5, then repaired with a regression test. Reviews and panels used the declared AI assistant. Every reported defect was reproduced before repair.

Table A.1: The corrections made to the instruments.

Found by	What was wrong	What changed	Effect on the figures
Protocol and harness			
One application's recall fell when the rule was tightened.	Test-source cross-check: a URL parameter in a test could match a literal route segment.	A parameter no longer matches a literal segment; a negative test is excluded.	The earlier recall cannot be regenerated; end-to-end recall there is now 71.4 %.
Figure: inspection of the flagged edges.	A low witness-check pass rate came entirely from may edges at one over-approximated call site.	Over-approximated sites are reported separately; the extractor is unchanged.	The low rate reflected the measurement; no must edge failed.
Harness: not stated beyond one application's misses.	Mutants with targets absent from the baseline graph repeat recovery gaps counted elsewhere.	These mutants are classified separately.	Most of that application's misses left the detection rate.
Figure: a construct count that tripled.	Substring counting included `routerLinkActive` as `routerLink`.	The independent parser counts constructs, giving the ratio and loss breakdown one definition.	alfresco: the independent route parser finds 34; the substring count was ≈three times larger.
Figure: coverage reached 100 per cent (mifos).	The independent parser skipped wrappers, stopped before lazy modules' routing modules, and lost lazy-table prefixes.	It follows wrappers and routing modules, retaining prefixes.	mifos has 610 declared screens, 58 with a node; the extractor was the same in both runs.

Found by	What was wrong	What changed	Effect on the figures
Figure: a ratio that rose implausibly after attribution (mifos).	Must conversion originally divided must edges by constructs.	Conversion is counted per construct; subtree-scoped edges are stored once (Section 3.2).	Edge storage inflated the rate without adding evidence. Definition: Section 4.2.
Harness, *review*, *figure*: harness review and a detection-rate shift exceeding twenty points.	The mutation harness patched the first textual match, judged detection on node and edge sets alone, and retargeted only the head of a commands array.	Patches use construct offsets; detection includes witnesses; retargeting selects another declared route; guards without incoming edges are excluded; renames require the renamed route to appear.	realworld's detection rose sharply. The seed stayed fixed, but the sample changed: 70 → 73 mutants (earlier → final), 46 sharing a location (operator, file, line; destinations may differ). The final sample omits 6/9 earlier misses. Of six earlier paired disagreements favouring attribution, five used retargeting that replaced only the commands-array head; correcting it also lets the baseline detect them.
Harness: reading the graph at the sites the rule let through.	Locating a guard's route by the nearest preceding path literal fails for children, empty paths, constants and template literals.	Brace matching after blanking comments and strings locates the route; routes without screens are excluded.	Four unobservable mutants, previously counted as graph defects, are now excluded. The superseded run sampled 70 mutants.
Harness: a change in the extractor, not a figure.	Guard removal was misjudged: a guarded parent's child lost the parent guard, and set comparison hid changes among three identical guard texts.	Guard multiplicities are counted; an edge losing a guard is detection, while absence from all baseline edges means exclusion.	Scoring changes; sampled mutants do not.
A table script stopped.	The construct denominator took constructs from commented-out links and code comments.	Comments are blanked before parsing, with the compiler's scanner for TypeScript.	One construct leaves the denominator in each of bitwarden and mifos; nothing else changes.
The extractor and the deterministic tier			
Panel: red-team panels reading graph and source after commit f17a302; the witness check missed it.	A guard on a wrapper-returned route was omitted, making all descendants unconditional.	Screen guard sets include function-returned routes.	At f17a302: 1024 must edges with 26 witness failures, read from that commit's results. Reading the guard cut mifos to 1 on the intermediate version.

Found by	What was wrong	What changed	Effect on the figures
Panel, as above.	Calls in `canActivate`, including guard factories, were skipped.	Unresolved elements still gate edges.	bitwarden's deterministic edges into its root are no longer unconditional.
Correcting the witness check itself.	It checked only the first construct producing an edge; later bitwarden citations targeted unresolved `/vault`.	It checks every cited site. Edges initially took the weakest cited modality (replaced below).	Later citations are 11.6 % of all witness citations, and up to 55.1 % in one application. The intermediate tier held 20 must edges, 20/20 passing, 0 failing.
Review, then *panel*: all intermediate-tier constructs (20 edges, 30 distinct constructs, 36 citations). Witness checking compares targets, not unconditional execution.	The demotion rules were incomplete: a link under `*ngIf`, in `@if`, a callback, a `catch`, or after `await` or an early return was must.	Demotion covers Section 3.1's conditions. Panels added members without unconditional handler calls, conditional component placement, ICU cases and ambiguous routes; later review added potentially superseding navigation.	32/36 citations ran under such a condition.
Review, as above.	A guard on an empty-path child without a component was not read, and `canMatch`, `canActivateChild` and `canLoad` not at all.	Such a guard gates the screen above; the other three kinds are read.	10 of the intermediate tier's edges, all of bitwarden's, are guarded.
Panel: a red-team panel.	Gating on every destination guard also demoted navigations the router permits.	The extractor computes the guards the router runs for each source and destination: levels left, matched, entered, kept.	Navigations between screens under a shared guarded parent pass; mifos holds 24 must edges again.
Review, as above.	Navigation to the current address was treated as occurring on every execution.	Such an edge is drawn per screen: must from the others, a may self-loop at the destination.	1 of the intermediate tier's edges is such a self-loop.
Reading the edges that survived.	Weakest-modality merging discarded certainty, as for realworld's unconditional Global Feed link.	Edges take their strongest construct's modality. Other witnesses retain theirs; must checking uses only witnesses that earn must.	The final tier has 159 edges on 25 constructs, a different set rather than a subset of the intermediate tier. Current conversion counts require each construct to earn must; superseded tables credited every cited construct.

Found by	What was wrong	What changed	Effect on the figures
The independent route parser and the denominators			
First node-audit disagreements: the extractor was scored against paths the independent parser could not resolve.	The parser missed constant, enum and `$localize` paths; first-party `baseUrl` imports; and named route arrays.	It resolves all three forms.	Without the last repair, it missed almost all kubernetes/dashboard screens and labelled nearly every extracted node unmatched.
The dead-link corrections.	It included Storybook fixtures and a sibling application's aliased tables, but failed to resolve a path in an `Object.freeze` table.	It excludes fixtures and sibling tables, and resolves the frozen table. Excluding frozen routes as unreadable was rejected: they were one call away.	Declared sets changed in 5/6 applications, with counts unchanged in 1.
Fixing the provenance of the comparison.	An interim parser's output was misidentified as earlier output; the script also treated missing files as unchanged denominators.	It uses the earlier parser's committed output.	The resulting bitwarden decomposition is withdrawn.
Resolving the first node audit's disagreement.	The first-audit extractor lost parent prefixes and created a `*` node per catch-all.	The loss-list repairs: a table not reached, a path or destination named and not written (Section 5.2).	Of 143 nodes with no declared route, 138 were the tail of a declared route and 5 catch-alls. The final version has 2 nodes matching no declared screen, 2 at `*`.
Not stated; classification followed graph inspection.	Denominator exclusions were tested after consulting the graph.	Exclusions are decided from source text first; comparison rejects changes between runs.	Attribution had enlarged the denominator; both columns now use the same 1775 sites.
Findings of the runtime route oracle			
The oracle found two alfresco addresses served by the catch-all. Both static parsers made the same error, hidden by 100 per cent coverage.	A named-outlet route could inherit a sibling's primary address; descendants also received primary-address nodes.	Traversal preserves outlet context and creates no screen beneath a named outlet.	Extracted screens at unmounted addresses: 0. Remaining gaps limit recall.
The oracle against both static route sets; each parser had independently acquired the defect.	Lazy modules reached through barrels were followed only one hop; their tables were scanned at the root.	Both follow bounded chains, rejecting at each hop the route's own file, files loaded by other routes, and files with root-registered tables.	10 addresses in alfresco, bitwarden, including the two alfresco addresses, were wrong. Static node coverage had been 100 per cent against the same wrong addresses.

Found by	What was wrong	What changed	Effect on the figures
Instrumenting the predicate (commit `d40c8dc`).	The root-table exclusion predicate recognised `RouterModule.forRoot` and never fired in the corpus.	Both tools mark root tables per array, from the argument of either registration API.	The inactive predicate had been credited with an untested property.
The dead-link census			
Refutation attempts on the dead-link finding.	An earlier correction followed only the redirecting kind of catch-all route.	Both kinds are followed; edges remain may and record the catch-all destination.	The original correction worked (✓) on its development application but failed (✗) on two using the other kind.
Refutation attempts.	Target resolution was conflated with dead-link detection. An unresolved enum sent all bitwarden authentication paths through the wildcard.	The enum and wrapper are resolved; lower-bound route sets are noted; caveated candidates are checked against runtime configuration.	Wildcard matching had inflated recovery. Withdrawn: 23 mounted screens, 19 screenless prefixes and 0 redirects; 0 remain caveated.
Refutation attempts.	Unresolved routes were treated as undeclared: a spread-built array, ghostfolio's `matcher` and alfresco's tsconfig `paths` alias.	Both tools record unresolved regions and exclude undeclared claims within them.	ghostfolio's `/admin/users` and two live alfresco links had been called dead; all three constructs are now resolved.
Refutation attempts.	The route-set test treated a commands array's first quoted segment as its complete target.	Whole-array patterns, with computed segments as holes, are matched against route patterns.	alfresco's two candidates were cleared and more mifos sites found; the census contains 29 candidates.
Mutation sweep and end-to-end check			
Mutation sweep of the demotion rules, at commit c8eec9b.	The final revision's first sweep found untested rules added by router repairs.	Tests cover most survivors; observationally equivalent mutants were retired.	Of 20/164 survivors, 18 gained tests and 2 were retired. At a9d29df, all 174 deletions fail a test. Testing also exposed relative links climbing above the root.
Panel: a review panel checking the scorer against the preregistration.	Redirects were scored at landed addresses, contrary to preregistration.	The scorer applies the preregistered rule.	mifos then scored 12/22 pairs, 5 of them redirected; the landed-target figure had been 15/22.
End-to-end check: mifos's suite missed transitions to action screens.	A commands-array template literal (`navigate([`actions/${name}`])`) was treated as an unresolved variable.	Static text is retained; interpolated segments become parameters.	The client, group and savings views reach their action screens.

Found by	What was wrong	What changed	Effect on the figures
Figure: after that repair the re-measured mifos graph joined every pair of screens.	Truncation at the first parameter emptied the notifications tray's two-parameter prefix, connecting its shell site to every route.	Only trailing parameters are dropped from a target prefix; a parameter inside it matches any one segment.	The tray reaches its named account screens. Graph pairs before → after repair: 28185 → 162567/367236.
End-to-end check: the same.	Unresolved `relativeTo` in an options variable made loan-view actions root-relative.	A `const` initialised with an object literal is read as that literal.	The loan view reaches its action screens.
End-to-end check: mifos's suite missed tab transitions.	A parent-template link rendered with child tabs was resolved against each child, producing unmounted targets.	Resolution uses the route rendering the component (Section 3.2.6), also in witness checking (harness 1.10.0).	With the preceding two repairs, mifos suite hits rose from 12 → 19/22. All extractor-dependent figures were re-measured.
Comparison on Rima et al.'s subjects (Section 5.3.7), after its frozen figures			
Comparison: petclinic's `navigate(['/owners', id])` reached the owner's edit and pet screens.	A command array ending in a bound value was read as a prefix, though Angular splits only the first command on slashes.	A command after the first is one segment, so the array is a pattern; a matrix-parameter object adds no segment.	The target pairs read as spurious are gone.
Comparison: traduora's locale link produced no transition.	A single bound command (`[routerLink]="[l.code]"`) produced an unknown target.	It becomes one segment below the declaring route, as already assumed for first-command interpolation. Some corpus bindings violate this assumption (mifos's `[cancelRoute]` can be `../../`), though pair counts were unchanged.	The locale list reaches the translation screen.
Comparison: airbus's navigation bar was placed on its sign-in screen.	An unrendered outlet holder and its navigation bar were mistaken for the shell.	If the bootstrapped root renders an outlet holder, unrendered holders are excluded from the shell.	The bar is placed on the pages that include it.
Comparison: airbus's router-service call from its dialogs was placed on the sign-in screen.	A class-opened component (`dialog.open(X)`) had no screens. A service member called only there therefore inherited every injecting component's screens.	Class-named opening (`open`, `createComponent`, `ComponentPortal`) creates a nesting relation to the opener.	With the preceding three repairs, Rima reference hits rose from 68 → 69/73, and other pairs changed from 357 → 341. All extractor-dependent figures were re-measured.

Note. *Found by* names the method as the main text does; *figure*, *panel*, *review*, *harness* and *end-to-end check* are explained in the text above.

Results marked *figure* exposed measurement errors rather than extractor defects.

Appendix B

Declared approximations

Table B.1: Declared approximations and exact-read constructs, and what the graph loses for each.

Construct	How it is treated	Consequence (tier, missing edges, note)
Found by the runtime route oracle		
Route declared outside the language	alfresco registers its about page through a JSON path and identifier, bound to a component by the runtime extension registry. TypeScript parsing cannot recover it; supporting the plugin format as a second source language is outside scope.	The route is absent without a note: no TypeScript construct is unresolved.
Table chosen outside the sources	mifos selects between two users-module route tables, one per identity provider, using a constant derived from deployment-injected `window.env`. Source-fixed conditions, such as environment-file flags, select an arm; unresolved conditions, as here, mark the subtree unread. Extraction uses checked-out environment values; production values may select differently. The oracle likewise uses a development build to measure the same configuration.	Declared unread subtree, 5 screens wide.
Read exactly (the analysis's responsibility)		

Construct	How it is treated	Consequence (tier, missing edges, note)
Literal routes, links, calls, nesting, outlet, guards	Supported exactly: string-literal route paths and their components; `routerLink` bindings whose value is a literal or an array of literals; `Router.navigate` and `Router.navigateByUrl` calls with a literal target; the nesting relation from element selectors resolved within the importing file's scope; the primary `<router-outlet>` with the shell rule that follows from it; guards named on a route, as labels on the edges entering that route's subtree.	Failures are defects counted in Chapter 5: constant-path, lazy-boundary and unlinked-node bugs (Section 5.2), resolved in the final version (Section 5.5), rather than approximations.
Approximated (no completeness claimed)		
Components created programmatically (dialogs, overlays)	Unrouted overlays have no screen nodes. Components or directives opening a class in their own file (`open(X)`, `createComponent(X)`, `new ComponentPortal(X)`) supply their screens through nesting (Table A.1). Service-opened or unresolved constructions have empty scope. Attribution gives these navigating components file-labelled unplaced origins, with *may* edges from their sites (Section 3.1).	The graph omits overlay states, entry/exit transitions and, for empty scopes, the screens from which exits fire; exit destinations remain represented. Navigation in a service injected only by such a component produces only a note.
Named outlets and `{outlets: ...}` targets	An auxiliary route adds a second active route and mount point, encoded as `(name:path)` [27]. Chapter 3 models a second mount, but the adapter omits it. Non-primary-outlet routes, their descendants and routes with unresolved outlet strings create no nodes. `navigateByUrl` drops auxiliary groups and marks primary-path edges *may*, since an unmatched auxiliary route would prevent navigation.	Unresolved outlet strings contribute to the incomplete-route-set note. Commands containing `{outlets: ...}` produce neither edges nor notes, even if the group includes the primary outlet. Later string commands are single segments; auxiliary URLs there match only as single segments. The test-source cross-check misses such URLs.
Navigation relative to the active route (`relativeTo`)	Template `routerLink` and `Router.navigate` with a `relativeTo` key in an options object (inline or in a `const`) resolve against the route rendering their component on the source screen's ancestor chain (Section 3.2.6), applying leading `.` and `..` segments. Calls without `relativeTo` and `navigateByUrl` resolve against the root.	Edges remain *may* when the chain has no rendering route (template-nested components), `relativeTo` is shorthand, uses other variables or names another route, a link binds `[relativeTo]`, or the source is the shell. They may name wrong screens, which must forbids.

Construct	How it is treated	Consequence (tier, missing edges, note)
Templates passed as values (`TemplateRef`)	Template values are not traced to instantiation. Links in `<ng-template>` are attributed to the declaring component's screens.	Links remain *may*: neither instantiation nor its location is established. Passing templates to other screens does not move their edges. Recovering instantiation requires dataflow analysis, as for JavaScript XML (JSX; Section 6.4).
Dispatch through NgRx effects and router event subscriptions	Chapter 3's dispatch relation is unimplemented, leaving dispatching screens unresolved. At *full* and *class*, navigation calls in application files containing `createEffect` receive $\top$: *may* edges from the shell, or edges into its `u_` sink for fully dynamic targets. Lower levels omit this stand-in.	Sound and coarse, since an effect can fire on any screen. An effect outside the application directory, or declared with the older `@Effect` decorator, is treated as any service call.
Navigation inside signal effects and observable subscriptions	Firing depends on a runtime dependency graph rather than textual control flow (Sections 2.7.3 and 2.8). The model assigns *unknown* with a symbolic read-set guard; neither is implemented. Navigation in an `effect`, `subscribe` callback or other called function receives its class's scope (Section 5.5).	Conditional invocation keeps edges at *may*; fully dynamic targets enter the `u_` sink. The graph identifies possible exits without their firing conditions.
Computed targets, no static part	Runtime-built targets cannot generally be enumerated statically. Targets without static parts, including interpolated `routerLink` values, produce edges into their source node's `u_` sink.	Modality *unknown*, confidence 0.
Computed targets, static leading part	Prefixes come from template-literal heads, concatenation's left operands, or commands arrays up to trailing variable elements. Earlier variable elements and interpolations in an opening template literal each represent one segment. Edges connect to every declared route extending the prefix by at least one segment; an empty prefix reaches every route except the root.	*May*, with incomplete over-approximation: static prefix segments require identical route segments, excluding routes with parameters at those positions.
Computed targets, commands array with trailing constants	An array with constants after a variable element becomes a pattern with one edge to its first matching route.	*May*. Variable elements match parameter segments only, silently omitting routes with constants in those positions.

Construct	How it is treated	Consequence (tier, missing edges, note)
Injection tokens and provider bindings	The call graph resolves field calls through declared types (Section 3.3). Bindings are application-wide: different providers for one token are joined rather than resolved by injector. Factory/value providers and externally returned provider arrays are unsupported. Calls fall back to all components injecting the class, or empty scope if none does.	Joins only widen scope; unresolved tokens add no edges. The corpus has 5574 calls resolved directly through application types, 4937 through provider bindings and 18 through subclasses, with 1111 unresolved across 2 of six applications.
Configuration-driven route registration	Array tables passed to `RouterModule.forRoot`, `forChild` or `provideRouter` are parsed wherever registered, including application-called provider factories. Routes created by startup configuration are omitted. Declared route objects with configuration-supplied `path` produce unresolved-path records without nodes.	Unresolved paths mark the route set as a lower bound. Function-computed tables, `ROUTES` providers, `Router.resetConfig` calls and JSON extension manifests (Section 5.5.3) silently omit routes without a lower-bound note. The independent parser behaves identically, so both agree on omissions. This approximation also caused one corpus rejection (Section 4.1).
Guards that would let the navigation through	State-dependent guards are not evaluated; expression text identifies them, so equivalent expressions remain distinct. Projected `<ng-content>` follows its host template's placement. Only `mat-menu` and `mat-tab` are recognised as lazy containers. Conditional outlets demote their route components, but nested components use their own placement; the root outlet is not inspected.	Any executed guard keeps an edge at *may*, even if it always passes. Conditional rendering inside a projection recipient does not demote projected links.
Navigations a later one may cancel	Claim 3.1's execution model excludes this case and modifier-key activation. Code that may start another navigation before event handling completes demotes the earlier one (Section 3.1). Calls are followed to application bodies: class members, file-level functions or methods resolved through typed receivers.	Library calls, untyped receivers and runtime-loaded code are assumed not to navigate, potentially leaving cancelled navigation at must. Scheduled navigation and other listeners for the event are not followed.
Children of a route a function returns, at the empty path	The first empty-path child under a screen and its first empty-path child render at the screen's address; their links originate there. For function-returned routes such as `withShell([...])`, traversal does not place the returned child components under the enclosing screen.	Links remain at unplaced origins with *may*: screen edges are missing, without false assertions.

Construct	How it is treated	Consequence (tier, missing edges, note)
Activation with a modifier key	Anchor `routerLink` leaves modifier-key and non-primary clicks to the browser, opening another browsing context, as does `target` naming another context. The model treats all activation as application navigation.	Must applies to plain activation, excluding these cases.
Order in which Angular matches routes	Angular matches routes and redirects in declaration order: an earlier parameter route can shadow a literal route. The extractor instead prefers exact paths. Separately registered tables follow provider order, which is unresolved; the extractor assumes their catch-alls follow all other tables' routes and records each dismissed catch-all in a note.	Overlapping routes or redirects keep edges at *may*, potentially naming Angular's unchosen route. The catch-all exception assumes unresolved registration order; Section 3.1 counts the must edges relying on it with Claim 3.1.
Ambiguous selectors in the nesting relation	Shared selectors are narrowed to components imported by the rendering file; if none is imported, all candidates are joined.	A note records the ambiguity; sites may be attributed to screens that never render their components.

Note. "May" is the tier without a soundness claim.